대회 / 코딩테스트 알고리즘 정리

Seojin Kim[1]

March 11, 2026

[1]Department of Computer Engineering, Sejong University, Seoul, Republic of Korea. GitHub: github.com/rlatjwls7882

머리말

이 문서는 **GitHub 저장소**에서 제작·배포되며, 최신 버전은 항상 해당 저장소에서 확인할 수 있습니다.

본 문서는 **알고리즘의 동작 방식과 구현**을 중심으로 가능한 한 간단하게 설명하여, 학습 및 복습 시 빠르게 참고할 수 있는 자료를 목표로 합니다.

정확한 증명이나 자세한 설명은 본 문서의 범위를 벗어나므로, 필요한 경우 다른 자료를 참고하시기 바랍니다.

또한 **차례의 순서는 정확한 난이도 순서를 의미하지 않습니다.** 각 알고리즘 제목 옆에 표시된 난이도는 작성자의 경험을 바탕으로 임의로 설정한 것이므로, 실제 난이도와 다를 수 있습니다.

난이도는 해당 알고리즘이 사용되는 문제의 **최소 티어**를 기준으로 설정하였으며, 실제로는 더 높은 난이도를 요구할 수도 있습니다.

© 2026 Seojin Kim

CC BY-NC-SA 4.0

차례

머리말 **i**

1 자료구조 **1**

1.1 DSU (Disjoint Set Union, 분리 집합) 1

2 기본 알고리즘 **5**

2.1 Binary Search (이분 탐색) 5

2.2 Prefix Sum (누적 합) 8

2.3 Two Pointer (두 포인터) 9

2.4 Backtracking (백트래킹) 12

3 그래프 **20**

3.1 경로 탐색 20

3.1.1 DFS (Depth First Search, 깊이 우선 탐색) 20

3.1.2 BFS (Breadth First Search, 너비 우선 탐색) 27

3.2 최단 경로 33

3.2.1 Dijkstra's Algorithm 33

3.2.2 Bellman-Ford Algorithm 39

3.2.3 Floyd-Warshall Algorithm 48

3.2.4 SPFA (Shortest Path Faster Algorithm) 52

3.3 DAG (Directed Acyclic Graph) 60

3.3.1 Kahn's Algorithm (Topological Sort, 위상 정렬) 60

3.4 최소 스패닝 트리 67

3.4.1 Kruskal's Algorithm 67

3.5 유량 72

3.5.1 Kuhn's Algorithm (Maximum Bipartite Matching, 이분 매칭) 72

© 2026 Seojin Kim

CC BY-NC-SA 4.0

1장

자료구조

1.1 DSU (Disjoint Set Union, 분리 집합) Gold V

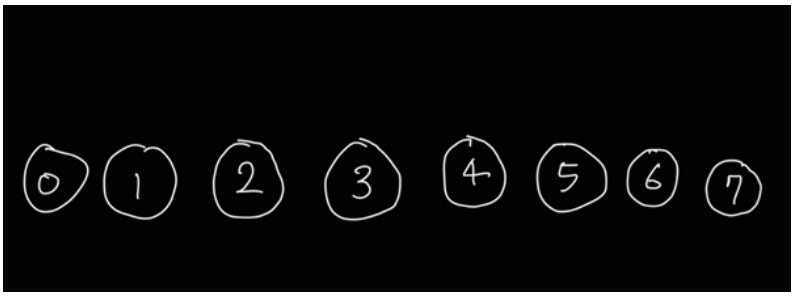
서로 겹치지 않는(서로소) 집합을 관리하여 합치기와 대표 원소 찾기 연산을 효율적으로 처리하는 자료구조

시간복잡도: $O(\alpha(N))$ ($\alpha(x)$: 역아커만 함수 $\approx$ 상수 시간, N: 데이터 개수)

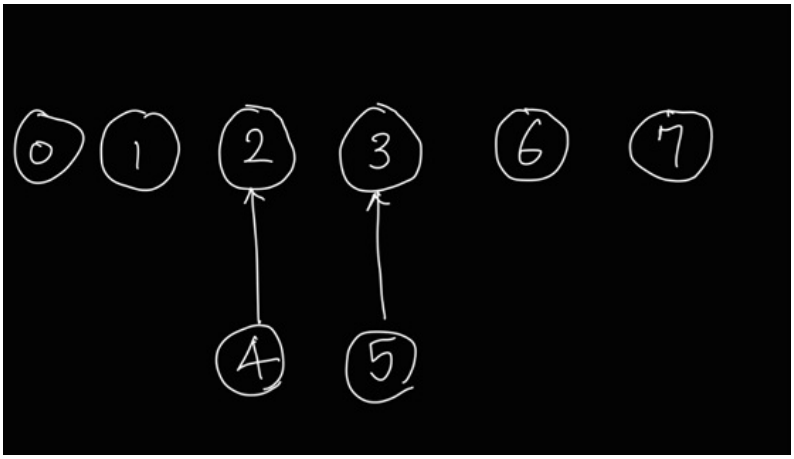
DSU는 서로소 집합을 관리하기 위한 자료구조로 **Find** 연산과 **Union** 연산을 제공한다.

Find 연산은 특정 원소가 속한 집합의 대표 원소를 찾는 연산이며, 이 과정에서 경로 압축을 적용할 수 있다.

Union 연산은 두 원소가 속한 집합의 대표 원소를 찾아 두 집합을 하나로 합치는 연산이다.

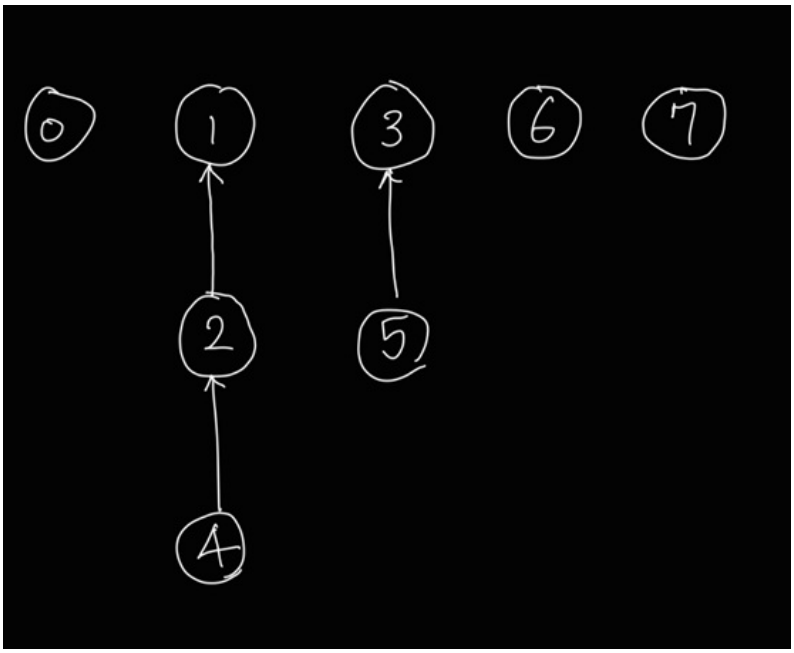


다음과 같이 7개의 원소가 각각 독립된 집합으로 존재한다고 하자.

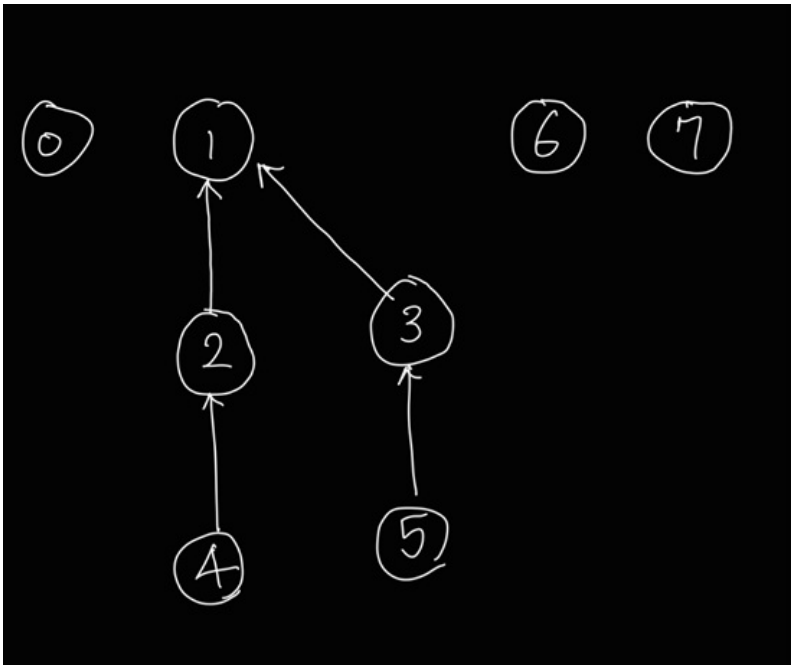


 CC BY-NC-SA 4.0

먼저 2와 4를 Union하고, 3과 5를 Union하면 다음과 같은 구조가 된다. (여기서는 각 집합에서 가장 작은 원소를 대표 원소로 사용한다고 가정한다.)



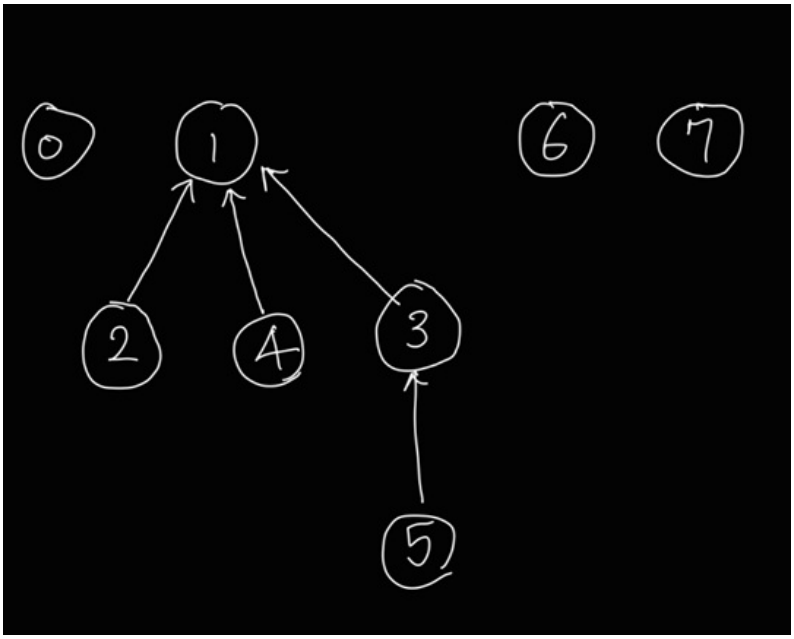
이후 1과 4를 Union하면 다음과 같이 집합이 하나로 합쳐진다.



이 상태에서 4와 5를 Union하면 단순한 트리 구조에서는 다음과 같이 연결될 수 있다. 하지만 이러한 방식은 트리의 높이가 계속 증가할 수 있으며, Find 연산의 시간 복잡도가 최악의 경우 $O(N)$까지 커질 수 있다.

이를 방지하기 위해 **경로 압축**을 사용한다. 경로 압축은 Find 연산을 수행하는 과정에서 탐색 경로에 있는 모든 노드가 직접 대표 원소를 가리키도록 만드는 방법이다.

 CC BY-NC-SA 4.0



이렇게 하면 트리의 높이가 크게 줄어들어 Union과 Find 연산을 거의 상수 시간에 가까운 $O(\alpha(N))$에 수행할 수 있다.

연습 문제: 백준 1717번

```
/** http://boj.kr/61d6f3b974084fc3ab1aca437121bcb9 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int parent[1'000'000]; // parent[i] : 원소 i가 속한 집합의 대표 원소

/**
 * x와 같은 집합에 속한 가장 작은 원소를 parent로 설정(경로 압축)하고 추출
 */
int _find(int x) {
    if(parent[x]==x) return x;
    return parent[x] = _find(parent[x]);
}

/**
 * x가 속한 집합과 y가 속한 집합을 합치는 연산
 * - x가 속한 집합의 가장 작은 원소 추출
 * - y가 속한 집합의 가장 작은 원소 추출
 * - x와 y중 작은 원소를 부모로 설정
 */
void _union(int x, int y) {
    x = _find(x);
    y = _find(y);
    if(x<y) parent[y]=x;
    else parent[x]=y;
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;
```

 CC BY-NC-SA 4.0

```
    for(int i=0;i<=n;i++) parent[i]=i; // i의 부모를 i로 설정 -> 각각이 고유의 집합이 됨

    while(m--) {
        int op, a, b; cin >> op >> a >> b;
        if(op==0) _union(a, b); // a가 속한 집합과 b가 속한 집합 합치기
        else cout << (_find(a) == _find(b) ? "YES\n" : "NO\n"); // a가 속한 집합의 가장
        ↪  작은 원소와 b가 속한 집합의 가장 작은 원소가 같은지 확인
    }
}
```

 CC BY-NC-SA 4.0

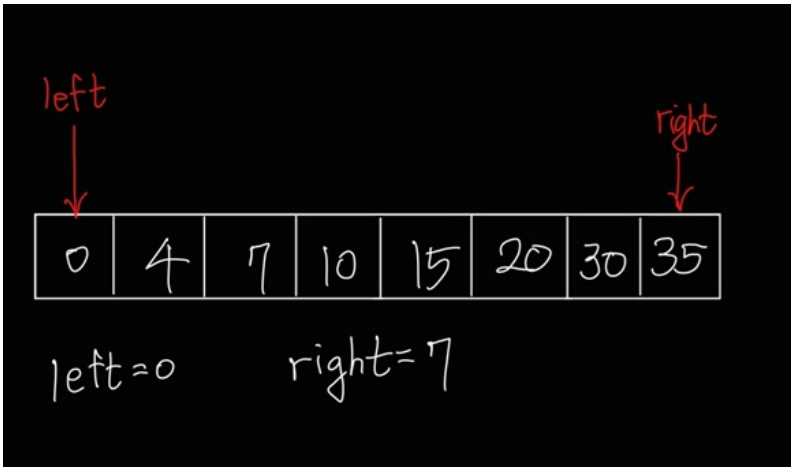
2장

기본 알고리즘

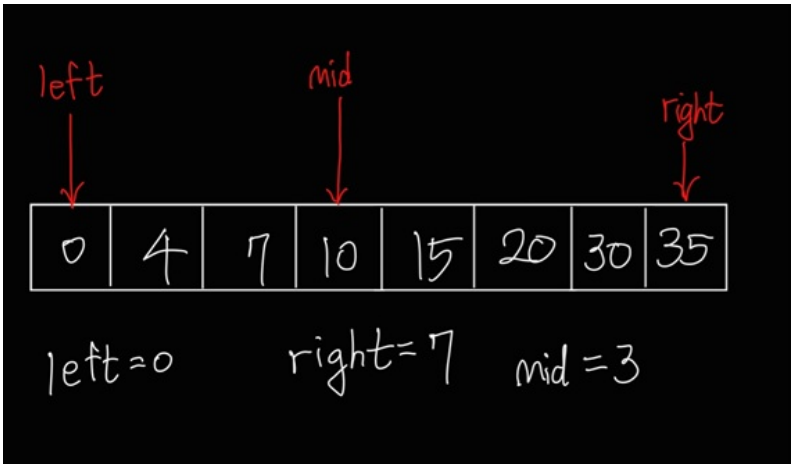
2.1 Binary Search (이분 탐색) Silver IV

정렬된 데이터에서 탐색 범위를 절반씩 줄여가며 원하는 값을 찾는 알고리즘

시간복잡도: $O(M \log N)$ (N: 데이터 개수, M: 탐색 횟수)

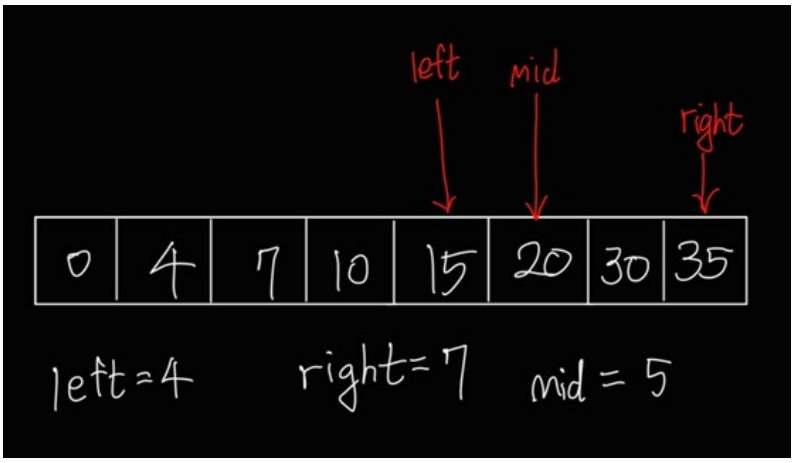


정렬된 길이 7의 배열 A에서 값 21을 찾는다고 가정하자.
초기에 21이 존재할 수 있는 인덱스 범위는 $0 \leq i \leq 6$이다.

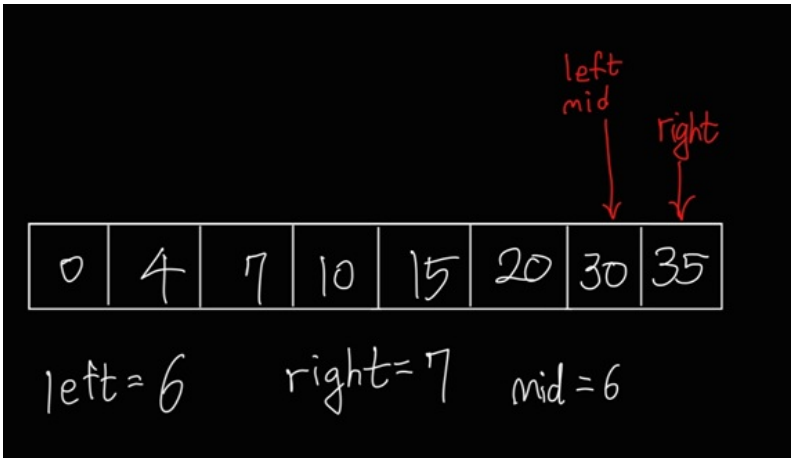


 CC BY-NC-SA 4.0

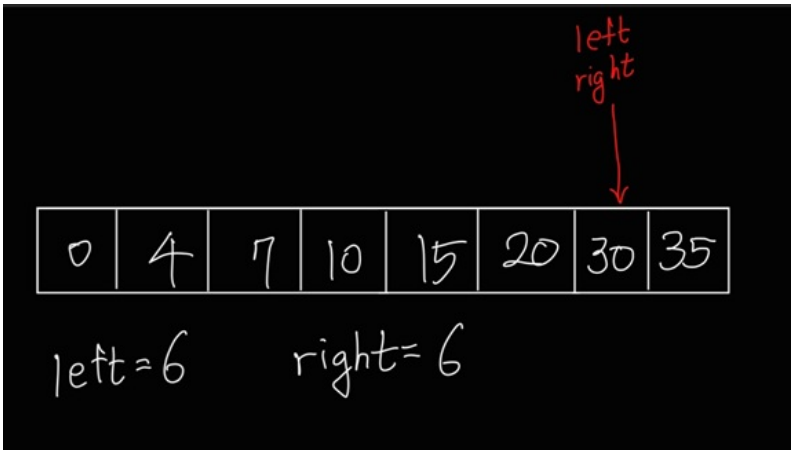
현재 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 3$
이때 $A_{mid} < 21$이므로 $left \le i \le mid$인 모든 A_i가 21보다 작다는 것을 알 수 있다.
따라서 탐색 범위를 $left = mid + 1$로 갱신한다.



다시 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 5$
이번에도 $A_{mid} < 21$이므로 $left \le i \le mid$인 모든 A_i가 21보다 작다는 것을 알 수 있다.
따라서 탐색 범위를 $left = mid + 1$로 갱신한다.



다시 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 6$
이번에는 $A_{mid} \ge 21$이므로 $mid \le i \le right$인 모든 A_i가 21보다 크거나 같다는 것을 알 수 있다.
따라서 탐색 범위를 $right = mid$로 갱신한다.



이제 $left = right$이므로 더 이상 탐색 범위를 줄일 수 없다.
마지막으로 $A_{left} = A_{right}$ 값을 확인한다.
이 값이 21이 아니므로 배열 A에는 21이 존재하지 않는다.

연습 문제: 백준 1920번

```
/** http://boj.kr/13586b0c1d744fe8b0c4f6a8393ed65c 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int A[100'001];

/**
 * Binary Search
 * 처음 범위 : 0 ~ n-1
 *
 * right가 left보다 큰 동안 :
 * - mid를 (left+right)/2로 설정
 * - mid 위치의 값이 현재 탐색중인 값보다 작다면 left = mid+1
 * - 그렇지 않다면 right = mid
 */
bool binary_search(int val) {
    int left=0, right=n-1;
    while(left<right) {
        int mid = (left+right)/2;
        if(A[mid]<val) left=mid+1;
        else right=mid;
    }
    return A[left] == val;
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n; cin >> n;
    for(int i=0;i<n;i++) cin >> A[i];
    sort(A, A+n);

    int m; cin >> m;
    while(m--) {
        int val; cin >> val;
```

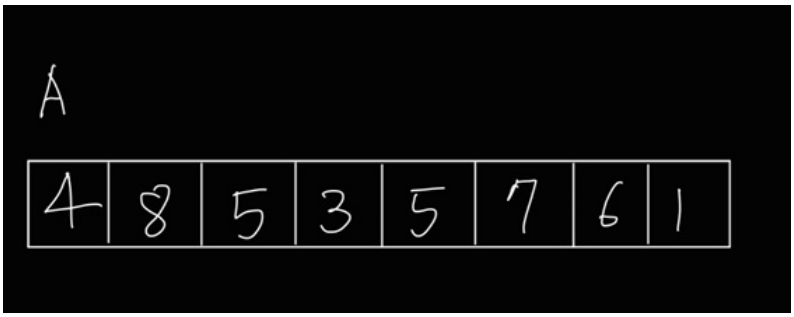
 CC BY-NC-SA 4.0

```
        cout << binary_search(val) << '\n';
    }
}
```

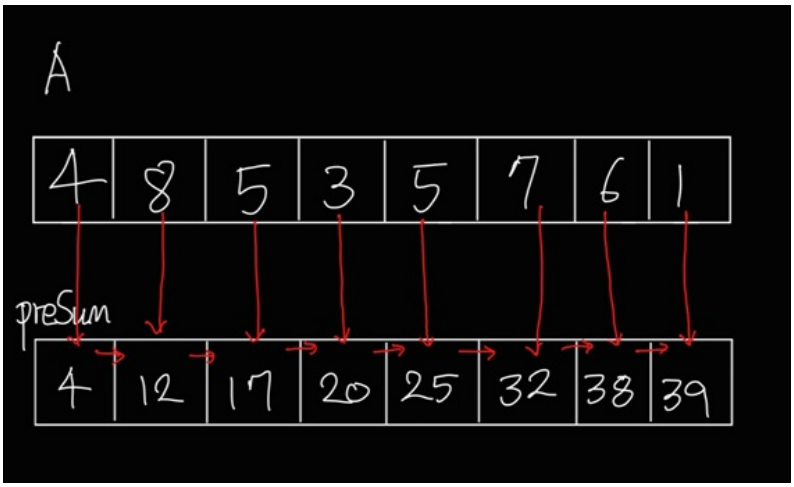
2.2 Prefix Sum (누적 합) Silver III

배열의 누적 합을 미리 계산하여 임의 구간의 합을 $O(1)$에 구하는 알고리즘

시간복잡도: 전처리 $O(N)$, 쿼리 $O(1)$ (N: 데이터 개수)



배열 A가 다음과 같다고 하자.



이때 배열 A의 누적 합 배열 $preSum$을 다음과 같이 정의한다.

$$preSum_i = A_0 + A_1 + \cdots + A_i$$

즉, $preSum_i$는 배열의 첫 번째 원소부터 i번째 원소까지의 합을 의미한다.
이때 구간 $A_i + A_{i+1} + \cdots + A_j$의 합은 다음과 같이 계산할 수 있다.

$$(A_0 + A_1 + \cdots + A_j) - (A_0 + A_1 + \cdots + A_{i-1})$$

이는 누적 합 배열을 이용하면 $preSum_j - preSum_{i-1}$로 간단하게 구할 수 있다.

연습 문제: 백준 11659번

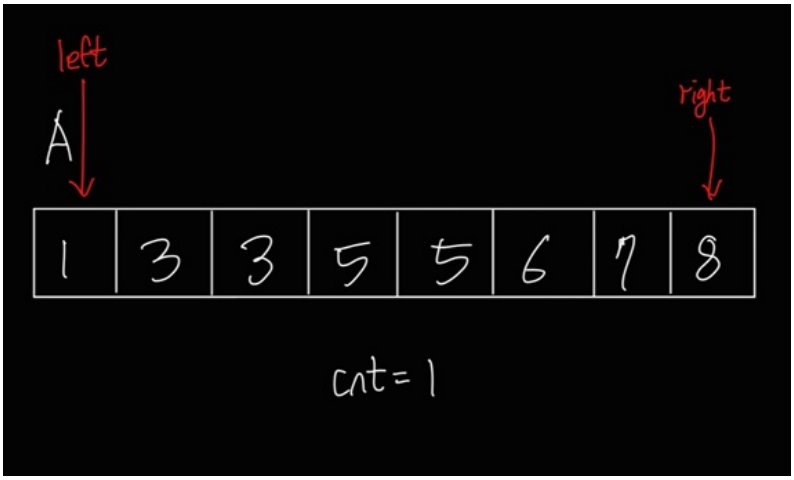
```cpp
/** http://boj.kr/8a8bf787cca8473fa1b0c13219b94918 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int preSum[100'001]; // preSum[i] : 1번 수부터 i번 수까지의 누적 합

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;
    for(int i=1;i<=n;i++) {
        cin >> preSum[i]; // i번째 수 입력
        preSum[i] += preSum[i-1]; // i번째 수에 0 ~ i-1 수의 합 더하기 -> 0 ~ i 수의 합
        ↪ 완성
    }

    while(m--) {
        int i, j; cin >> i >> j;
        cout << preSum[j] - preSum[i-1] << '\n'; // i ~ j 수의 합 = (0 ~ j 수의 합) -
        ↪ (0 ~ i-1 수의 합)
    }
}
```

2.3 Two Pointer (두 포인터) Silver III

두 개의 포인터를 움직이며 배열에서 조건을 만족하는 쌍을 효율적으로 찾는 알고리즘

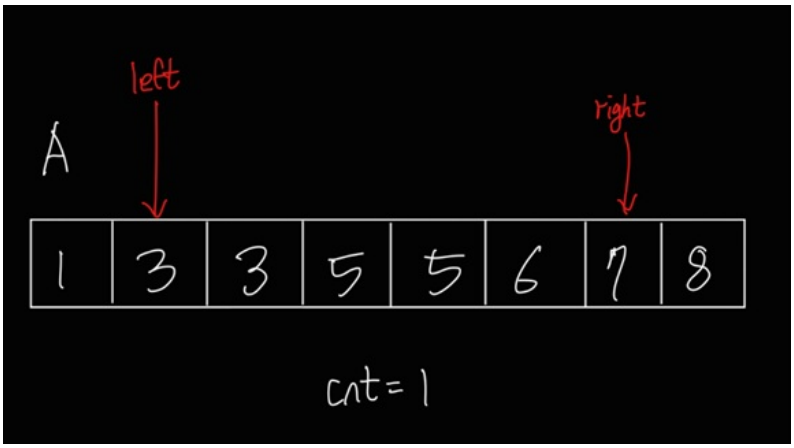
시간복잡도: $O(N)$ (N: 데이터 개수)

다음과 같은 배열 A에서 두 수의 합이 9가 되는 쌍을 찾는다고 하자.

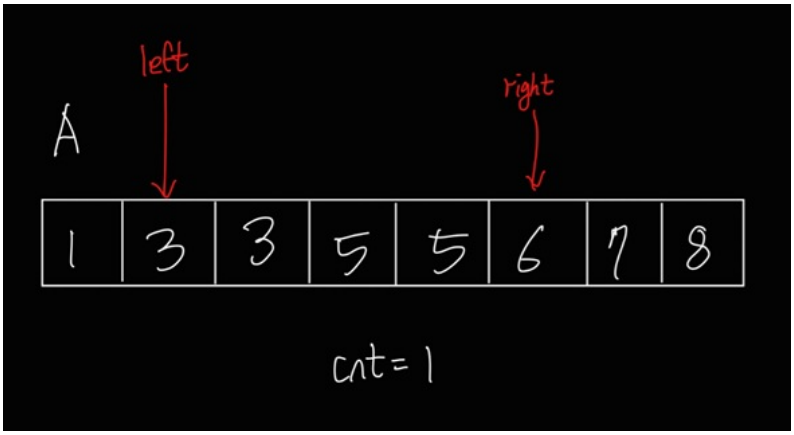


처음에는 왼쪽 포인터 $left$를 배열의 왼쪽 끝에, 오른쪽 포인터 $right$를 배열의 오른쪽 끝에 두고 탐색을 시작한다.

$A_{\text{left}} + A_{\text{right}} = 9$이므로 cnt를 1 증가시키고, 해당 조합을 다시 사용하지 않도록 $left$를 1 증가시키고 $right$를 1 감소시킨다.

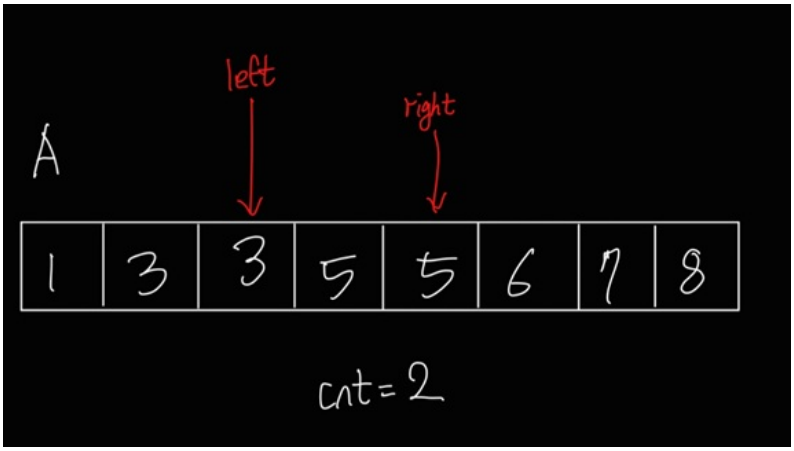


$A_{\text{left}} + A_{\text{right}} > 9$이므로 합을 줄이기 위해 $right$를 1 감소시킨다.

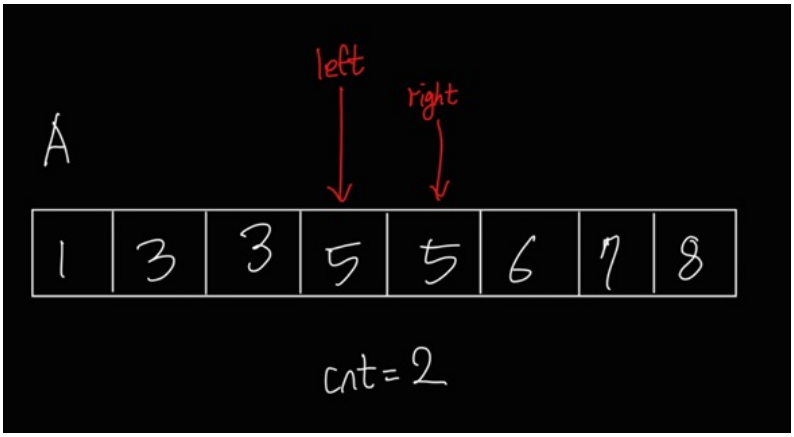


$A_{\text{left}} + A_{\text{right}} = 9$이므로 cnt를 1 증가시키고, 해당 조합을 다시 사용하지 않도록 $left$를 1 증가시키고 $right$를 1 감소시킨다.

 CC BY-NC-SA 4.0



$A_{\text{left}} + A_{\text{right}} < 9$이므로 합을 늘리기 위해 $left$를 1 증가시킨다.



$A_{\text{left}} + A_{\text{right}} > 9$이므로 $right$를 1 감소시킨다. $left \geq right$가 되었으므로 탐색을 종료한다.
이와 같이 $left < right$인 동안 위 과정을 반복하면 두 수의 합이 9가 되는 모든 쌍을 찾을 수 있다.

연습 문제: 백준 3273번

```
/** http://boj.kr/bba07bce85944877bf61c2508cd6a062 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int a[100'000];

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n; cin >> n;
    for(int i=0;i<n;i++) cin >> a[i];
    int val; cin >> val;
    sort(a, a+n); // 정렬

    int cnt=0, left=0, right=n-1; // left와 right는 각각 왼쪽 끝에 위치한 포인터, 오른쪽
    ↪ 끝에 위치한 포인터
    while(left<right) {
        if(a[left]+a[right]<val) { // 두 원소의 합이 val보다 작으면
```

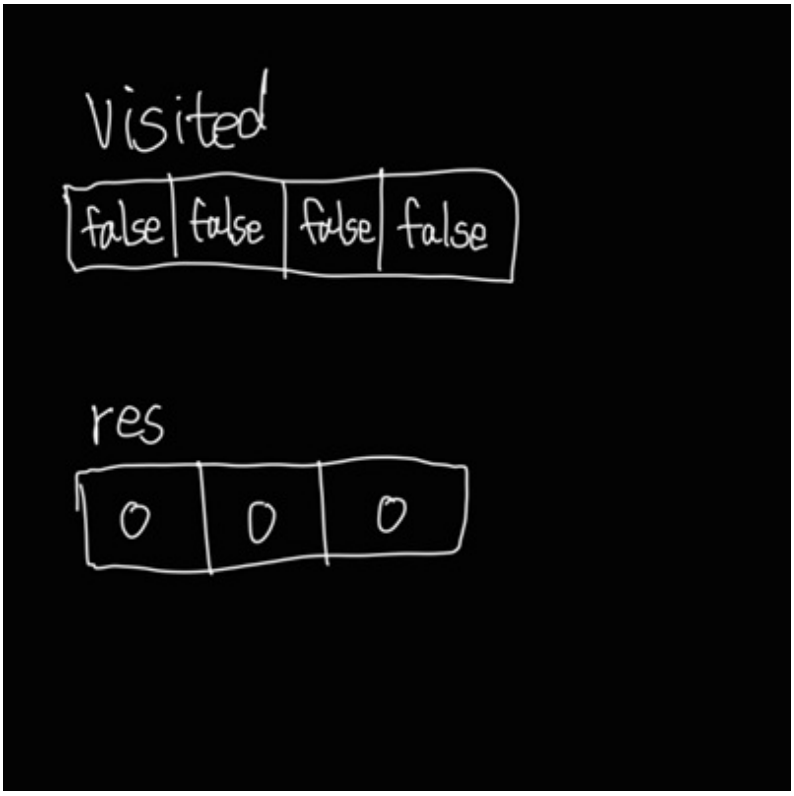
```
            left++; // 왼쪽 포인터를 오른쪽으로 이동
        } else if(a[left]+a[right]>val) { // 두 원소의 합이 val보다 크면
            right--; // 오른쪽 포인터를 왼쪽으로 이동
        } else { // 두 원소의 합이 val과 동일하면
            cnt++;
            left++; // 왼쪽 포인터를 오른쪽으로, 오른쪽 포인터를 왼쪽으로 이동 (사용한 원소
            ↪ 제외)
            right--;
        }
    }
    cout << cnt;
}
```

2.4 Backtracking (백트래킹) Silver III

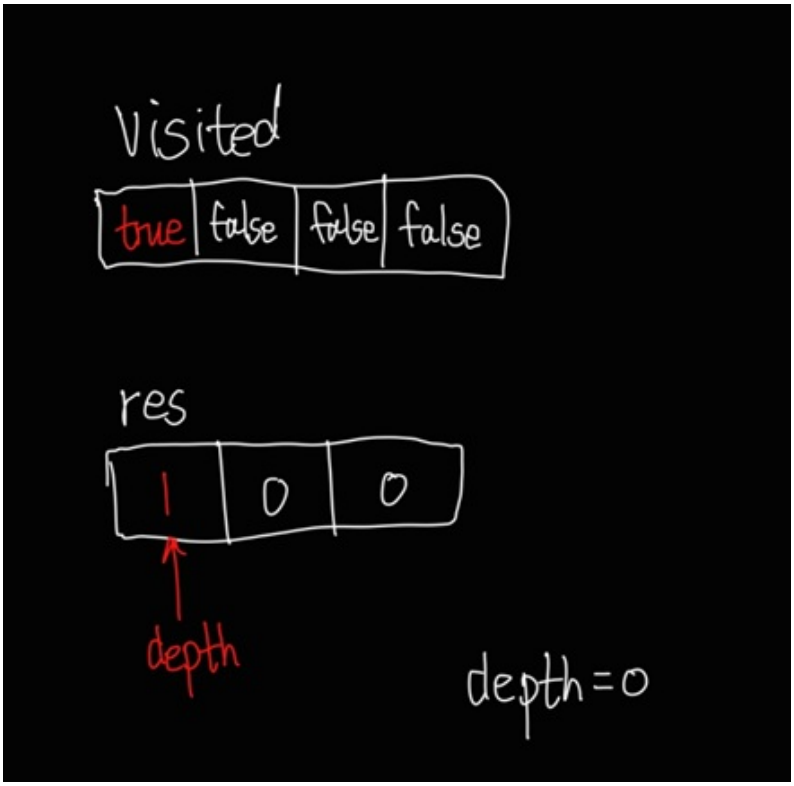
모든 경우를 탐색하면서 해답이 될 수 없는 경로는 가지치기하여 탐색을 줄이는 완전 탐색 기법

시간복잡도: 최악 $O(K^n)$ (K: 선택지 개수, n: 깊이)

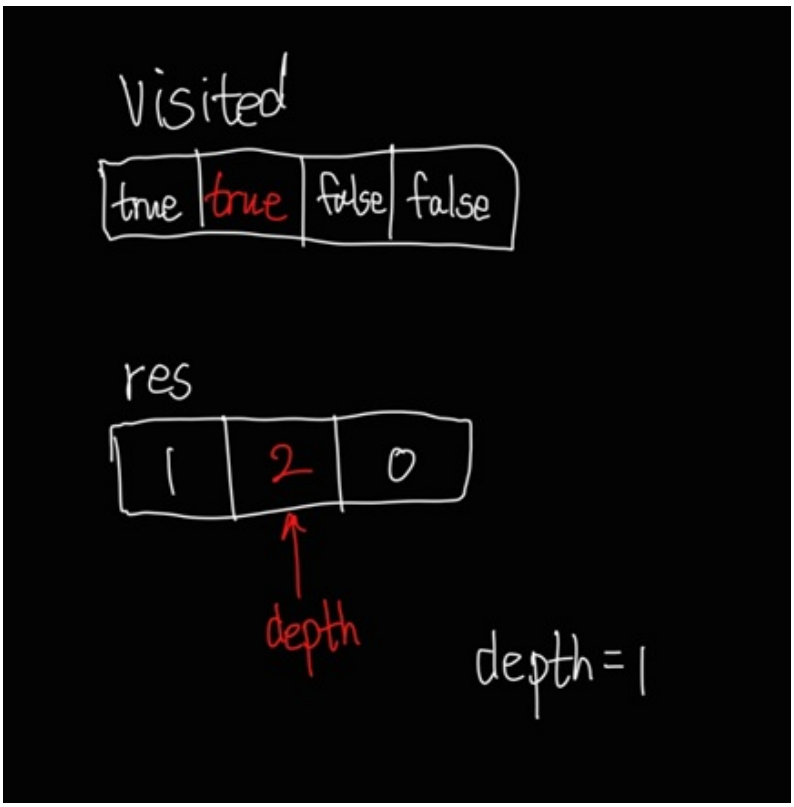
${}_nP_m$의 모든 경우를 오름차순으로 출력하는 문제를 생각해보자.
이 문제는 백트래킹을 이용해 해결할 수 있다. 각 깊이에서 하나의 숫자를 선택하여 결과 배열을 채우고 이미 사용한 숫자는 다시 선택하지 않도록 제외한다.



예를 들어 $n = 4$, $m = 3$인 경우를 살펴보자. 초기 상태에서는 아직 어떤 숫자도 선택되지 않았다.

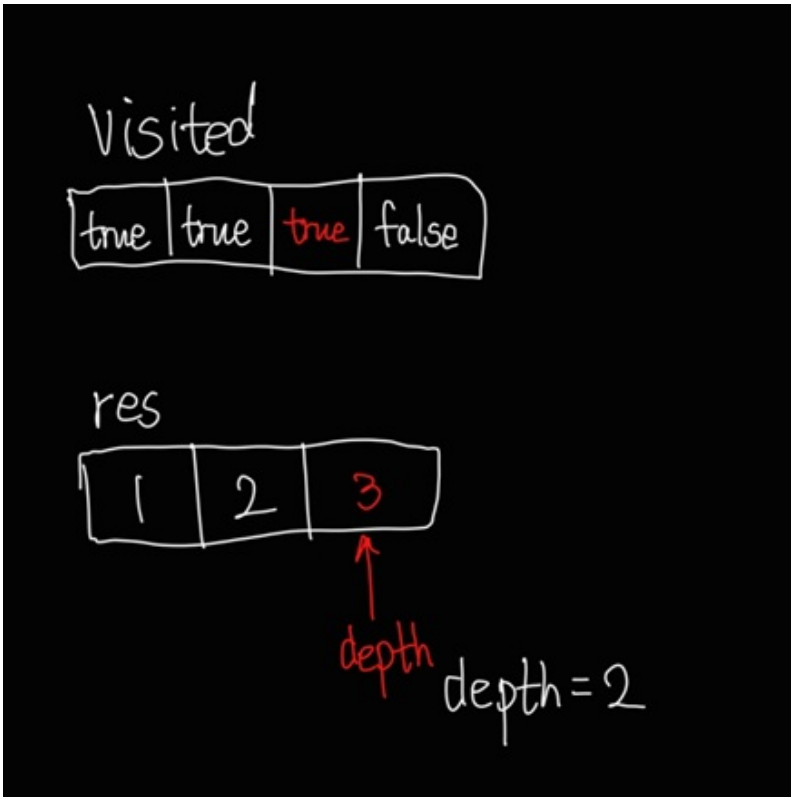


깊이 0에서는 첫 번째 칸에 들어갈 숫자를 결정한다. 1부터 4까지 차례대로 확인하면서 아직 사용하지 않은 숫자를 선택한다. 먼저 1이 선택된다.

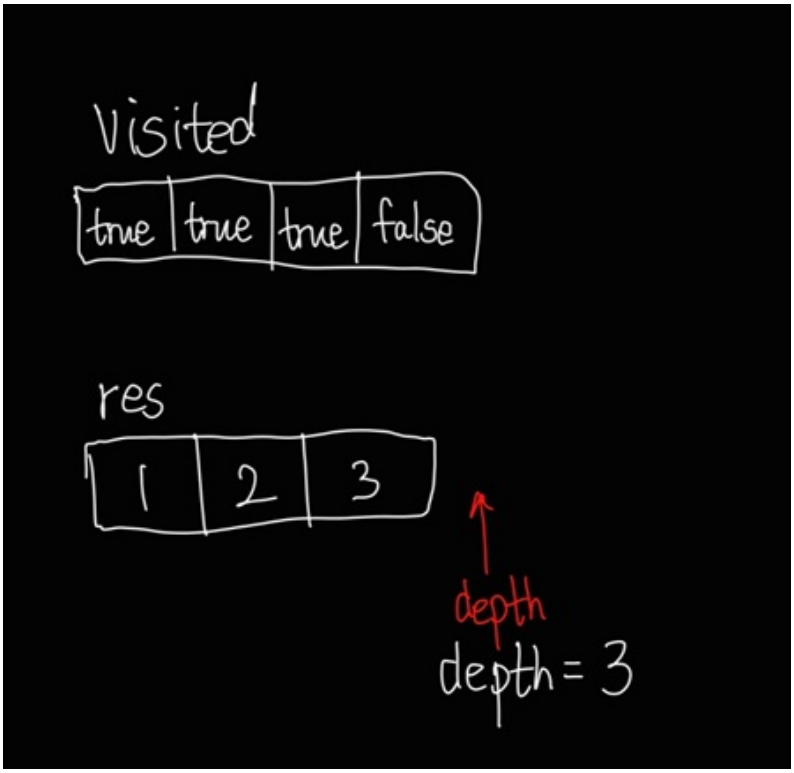


이후 깊이 1에서는 두 번째 칸에 들어갈 숫자를 결정한다. 이미 1은 사용되었으므로 남아 있는 숫자 중 가장 작은 2가 선택된다.

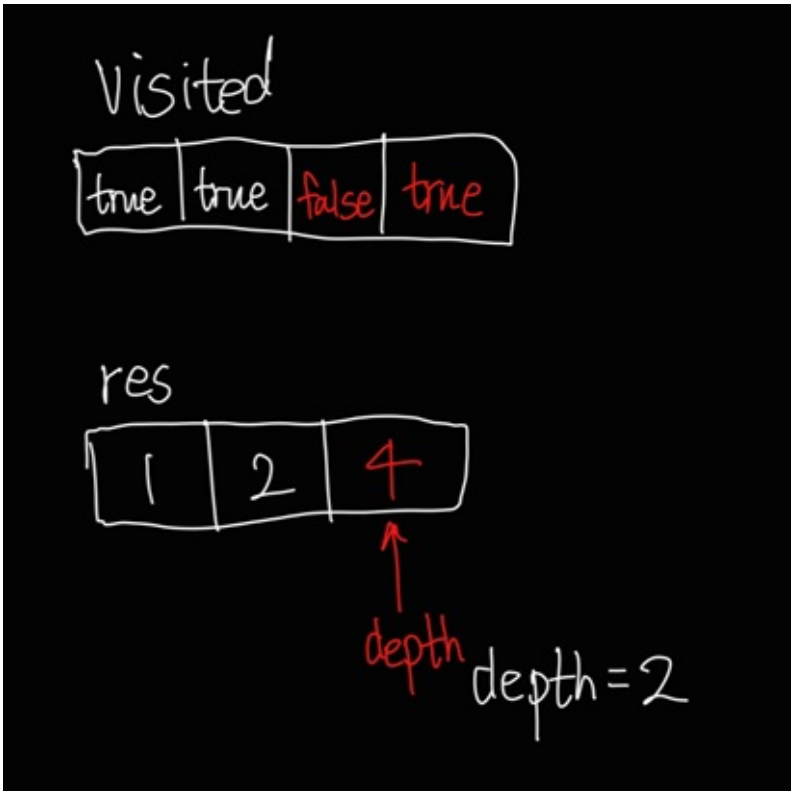
 CC BY-NC-SA 4.0



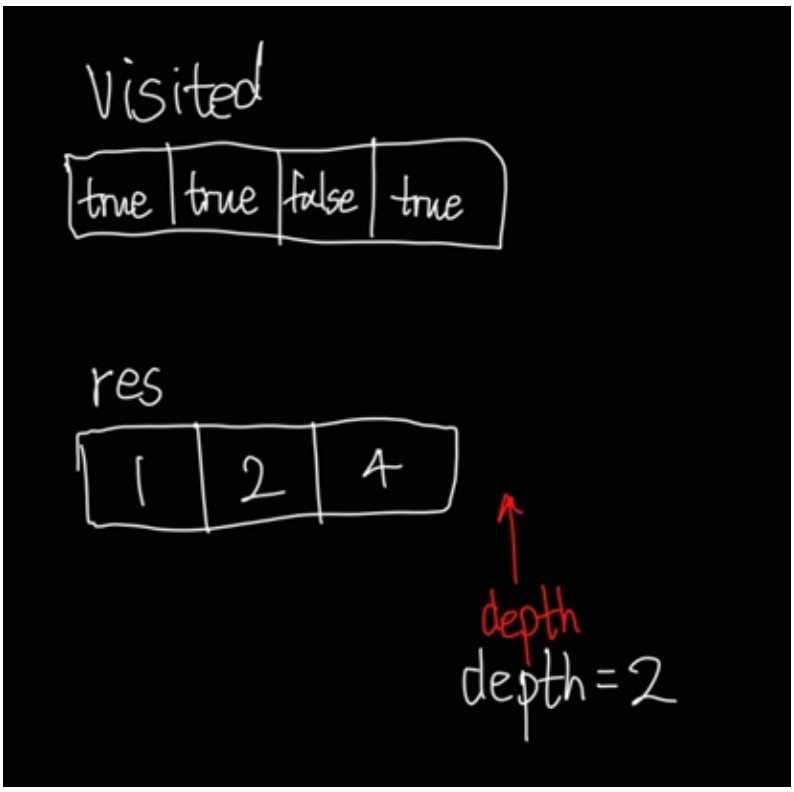
다음으로 깊이 2에서는 세 번째 칸에 들어갈 숫자를 결정한다. 사용하지 않은 숫자 중 가장 작은 3이 선택된다.



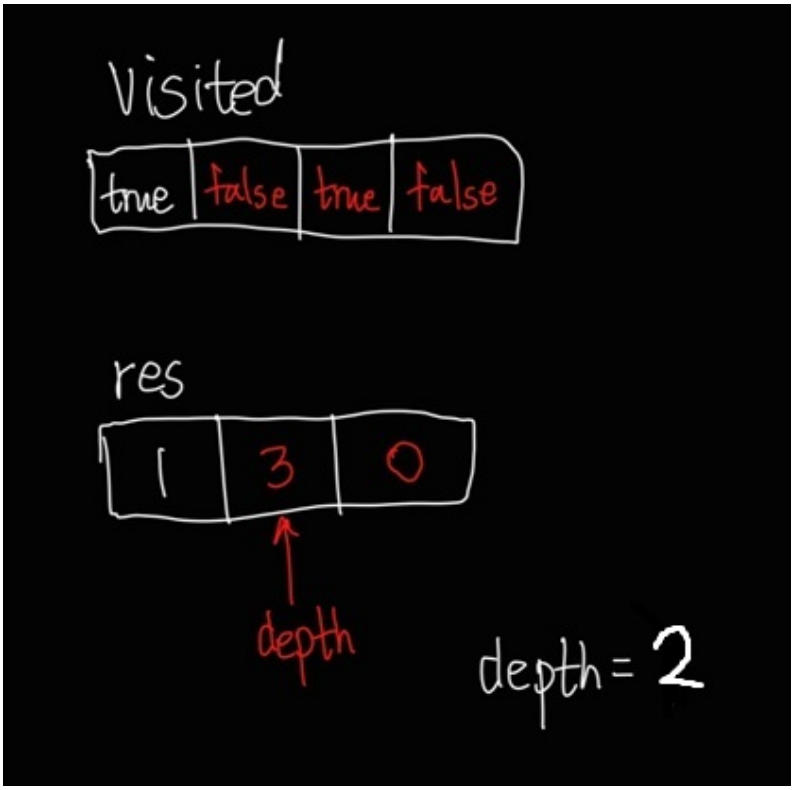
이제 깊이가 $m = 3$에 도달했으므로 현재까지 선택한 값인 **1 2 3**을 출력한다.



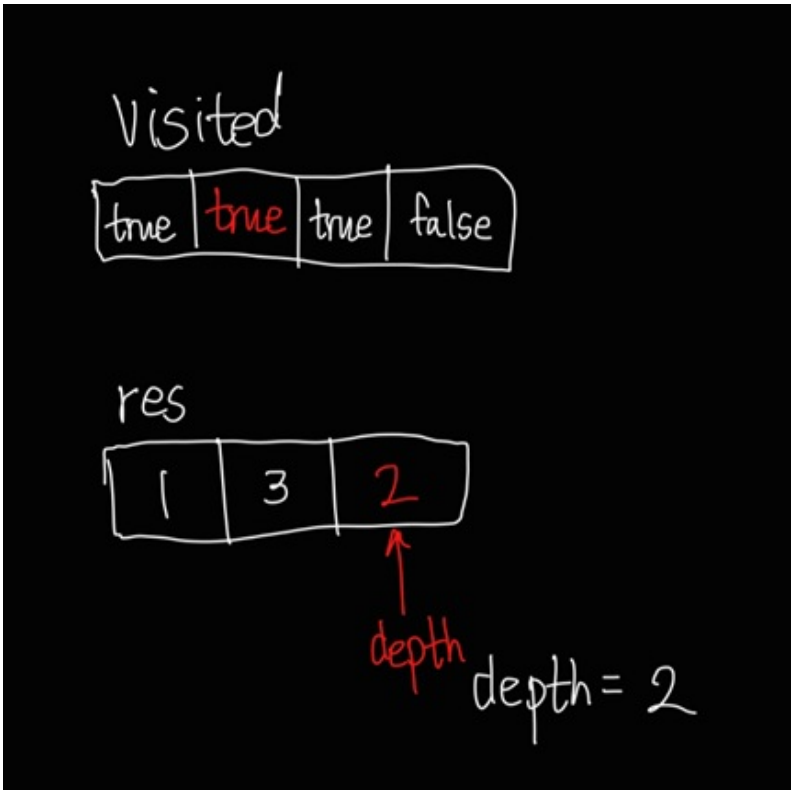
출력을 마친 뒤에는 백트래킹을 통해 이전 상태로 되돌아간다. 깊이 2에서 다음으로 선택할 수 있는 숫자는 4이므로 세 번째 칸의 값을 4로 바꾼다.



다시 깊이 3에 도달했으므로 이번에는 **1 2 4**를 출력한다.

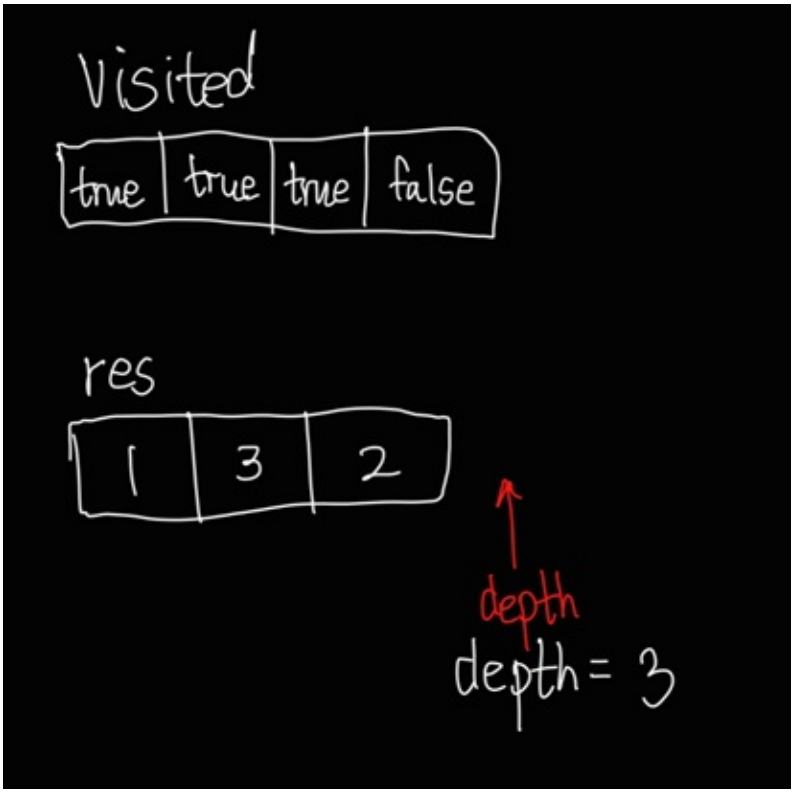


이후 다시 이전 상태로 돌아가 깊이 1에서 다음 가능한 숫자를 선택한다. 두 번째 칸에는 3이 들어간다.



그 다음 깊이 2에서는 아직 사용하지 않은 숫자 중 2가 선택된다.

 CC BY-NC-SA 4.0



따라서 이번에는 **1 3 2**가 출력된다.
이와 같은 과정을 반복하면 가능한 모든 순열이 차례대로 출력된다.

1 2 3

1 2 4

1 3 2

1 3 4

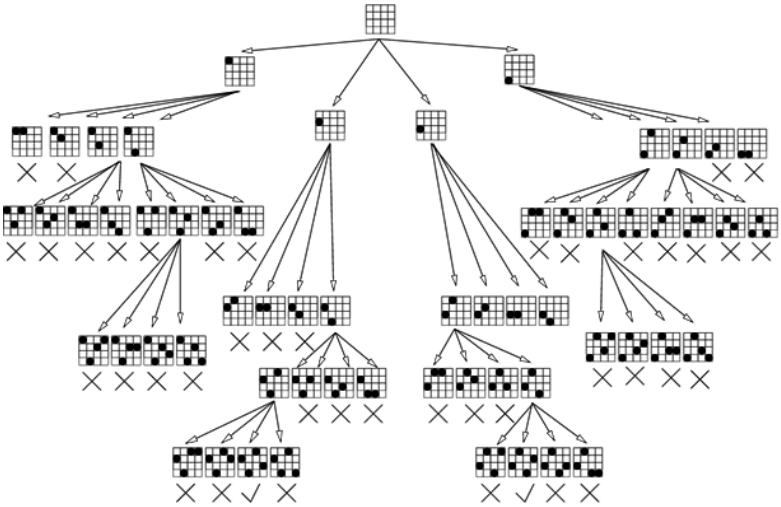
2 1 3

2 1 4

...

이처럼 백트래킹은 현재 깊이에서 가능한 값을 하나 선택한 뒤 하나의 경우를 완성하면 이전 단계로 되돌아가 다음 선택지를 탐색하는 방식으로 동작한다.

 CC BY-NC-SA 4.0



연습 문제: 백준 15649번

```
/** http://boj.kr/1a898dba13d74c9fb524af5ca7770877 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

/**
 * n : 선택지의 개수
 * m : 최대 깊이
 * arr[i] : 깊이 i일때의 저장된 값
 * visited[i] : i번째 선택지가 사용되었는지 여부
 */
int n, m, arr[8];
bool visited[8];

/**
 * Backtracking
 * - 최대 깊이에 도달한 경우
 * - - 문제에서 요구하는 것을 수행하고 return으로 빠져나오기
 * - 최대 깊이에 도달하지 않은 경우
 * - - n개의 선택지 중 사용 안된 것을 사용하고 다음 깊이로 이동
 */
void backtracking(int depth=0) {
    if(depth==m) {
        // 문제 요구사항 : 출력
        for(int i=0;i<m;i++) cout << arr[i] << ' ';
        cout << '\n';
        return;
    }

    for(int i=0;i<n;i++) {
        if(!visited[i]) { // i번째 선택지를 사용 안한 경우
            visited[i]=true; // 사용 마킹
            arr[depth]=i+1;
            back(depth+1); // 다음 깊이로 이동
            visited[i]=false; // 사용 마킹 해제
        }
    }
```

 CC BY-NC-SA 4.0

```
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin >> n >> m;
    backtracking();
}
```

 CC BY-NC-SA 4.0

3장

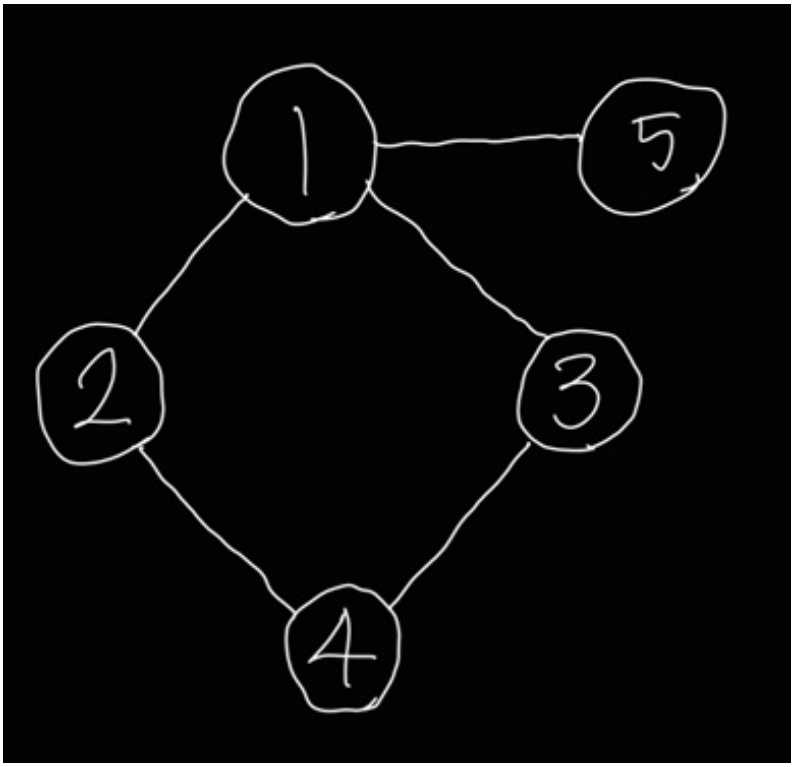
그래프

3.1 경로 탐색

3.1.1 DFS (Depth First Search, 깊이 우선 탐색) Silver II

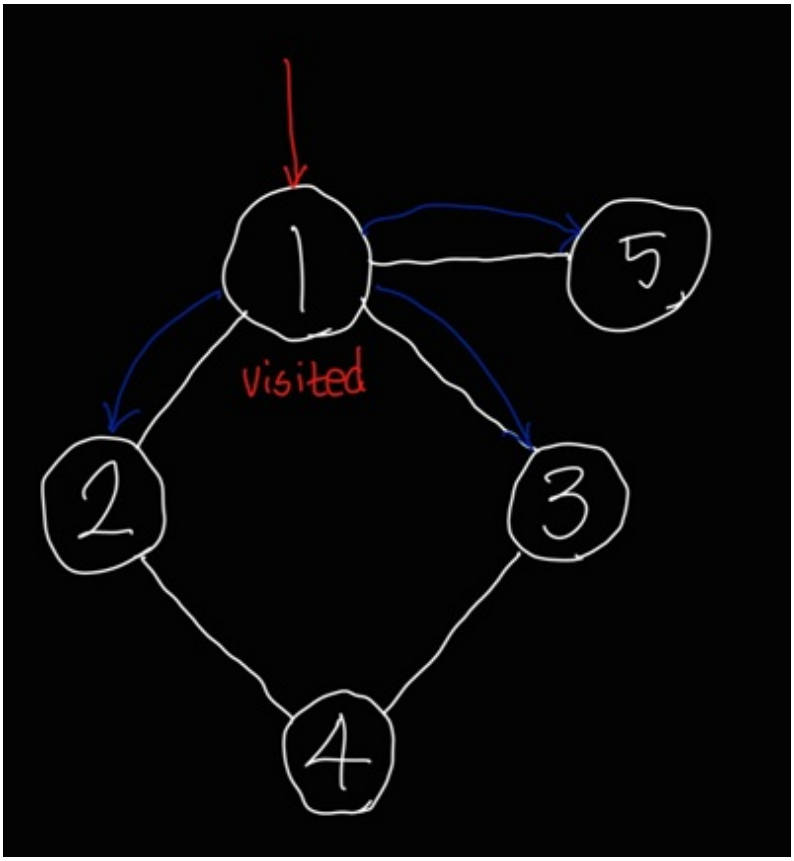
그래프에서 한 경로를 가능한 깊게 탐색하다가 막히면 이전 분기점으로 돌아가 다른 경로를 탐색하는 알고리즘

시간복잡도: $O(V + E)$ (V: 정점 수, E: 간선 수)

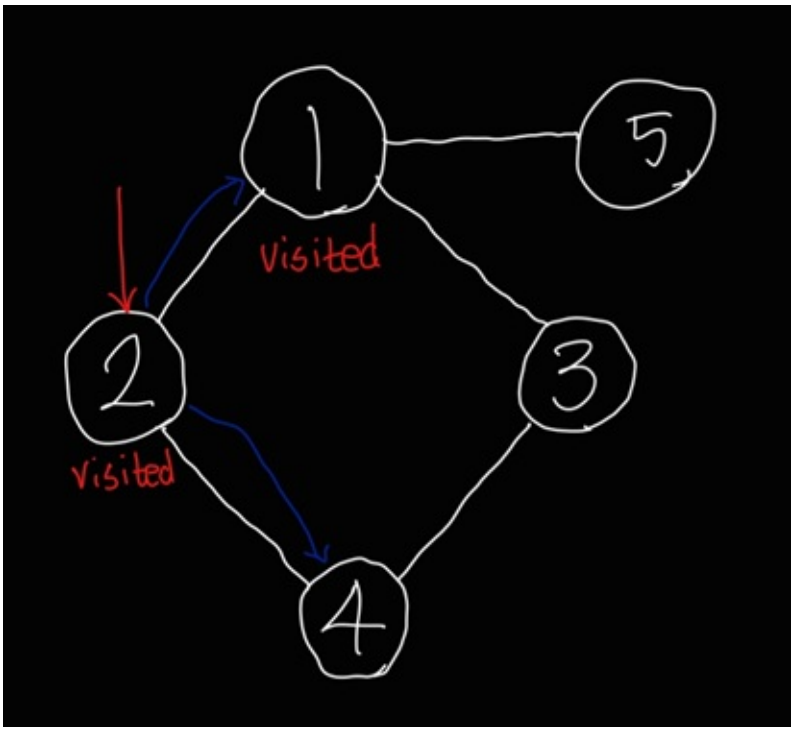


위와 같이 연결된 그래프가 있다고 하자. 탐색은 **1번 정점에서 시작**하며 동시에 여러 정점으로 이동할 수 있는 경우 **정점 번호가 작은 순서**로 이동한다고 가정한다.

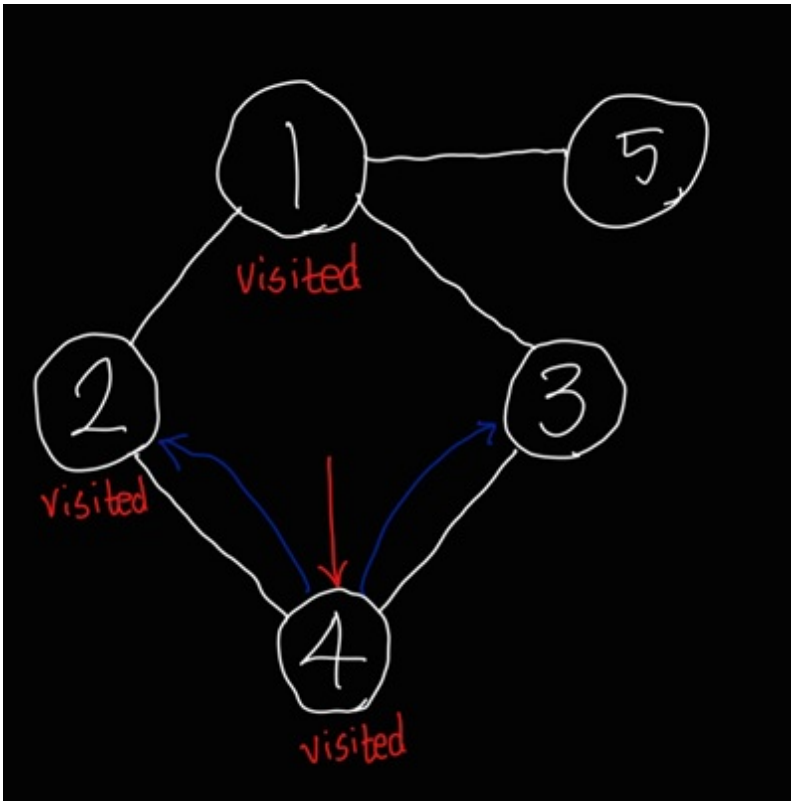
 CC BY-NC-SA 4.0



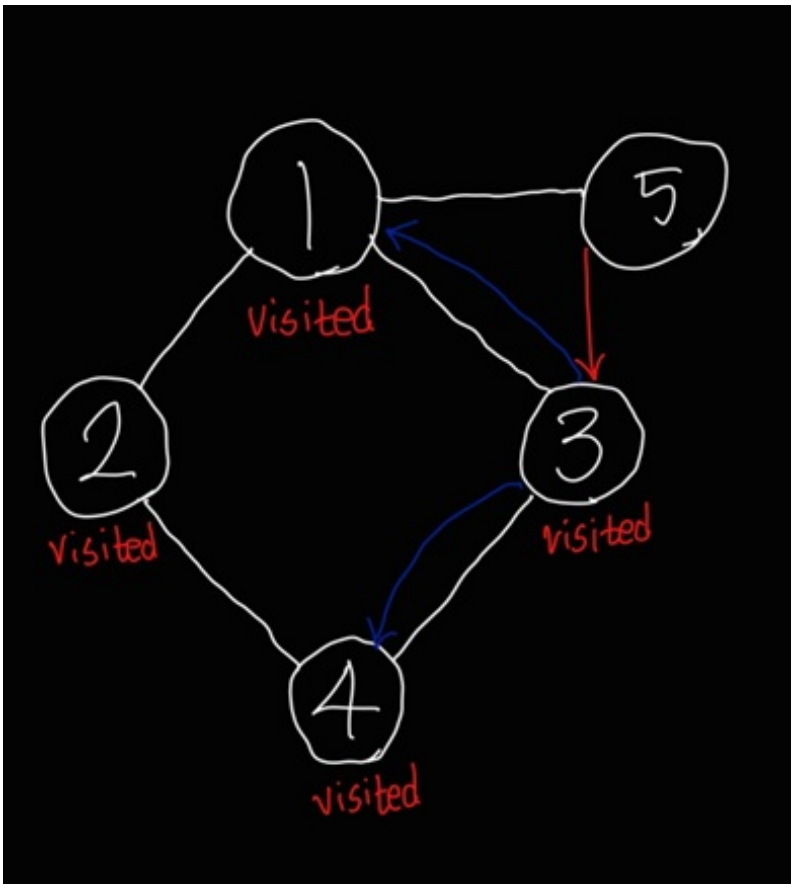
처음에는 시작 정점인 **1번 정점**에 위치한다. 1번 정점과 연결된 정점은 **2번, 3번, 5번**이다.



이 중 가장 번호가 작은 **2번 정점으로 이동**한다. 2번 정점과 연결된 정점은 **1번, 4번**이다.

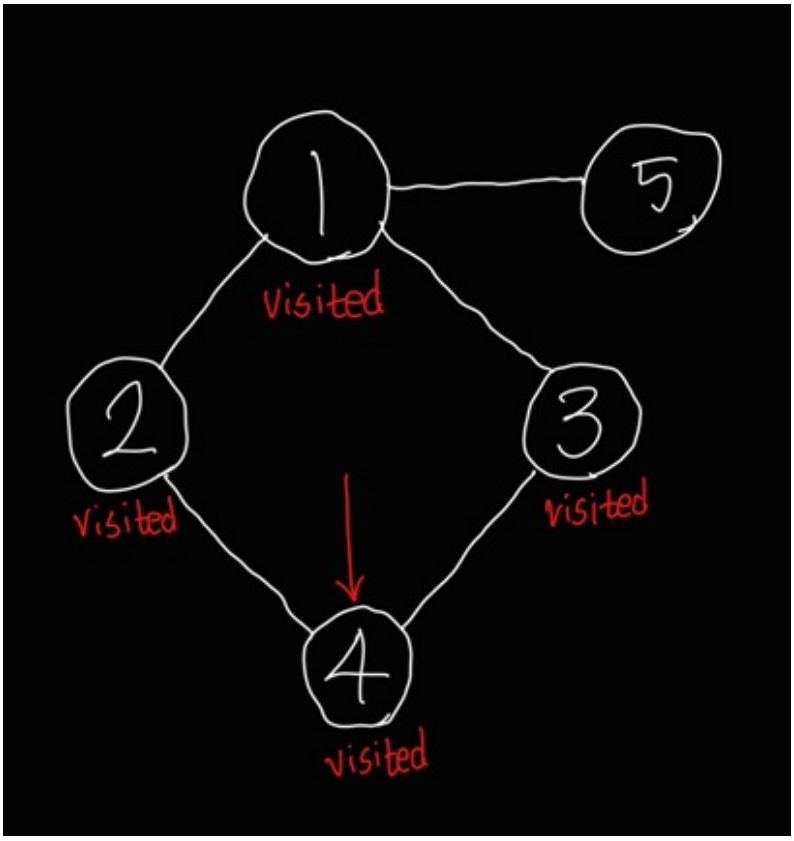


이미 방문한 **1번 정점은 제외**하고 **4번 정점으로 이동**한다. 4번 정점과 연결된 정점은 **2번, 3번**이다.

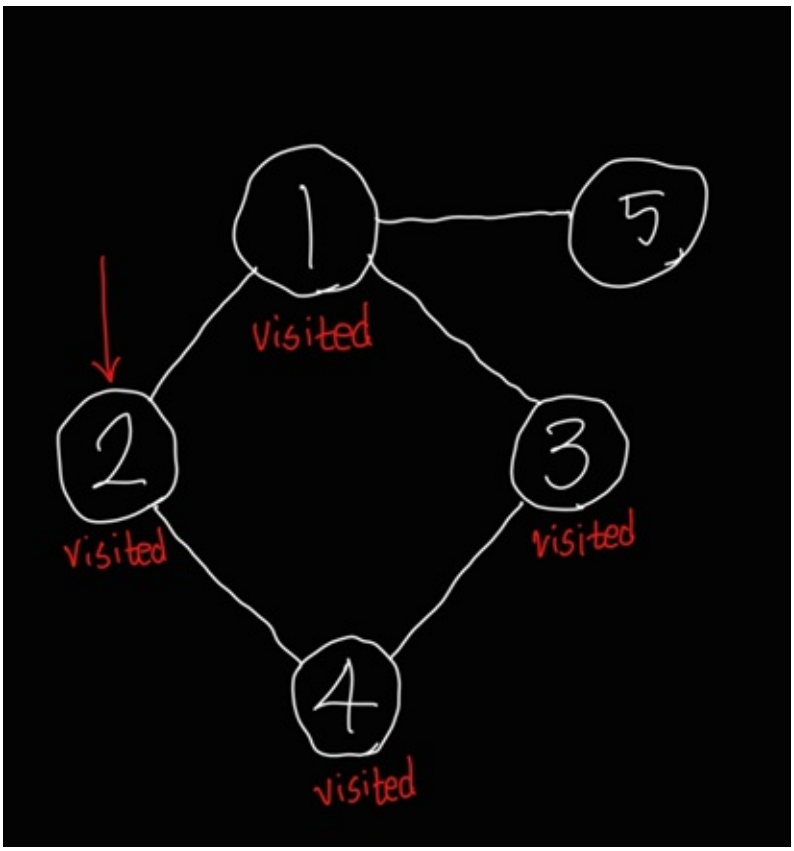


이미 방문한 **2번 정점을 제외**하고 **3번 정점으로 이동**한다. 3번 정점과 연결된 정점은 **1번, 4번**이다.

 CC BY-NC-SA 4.0

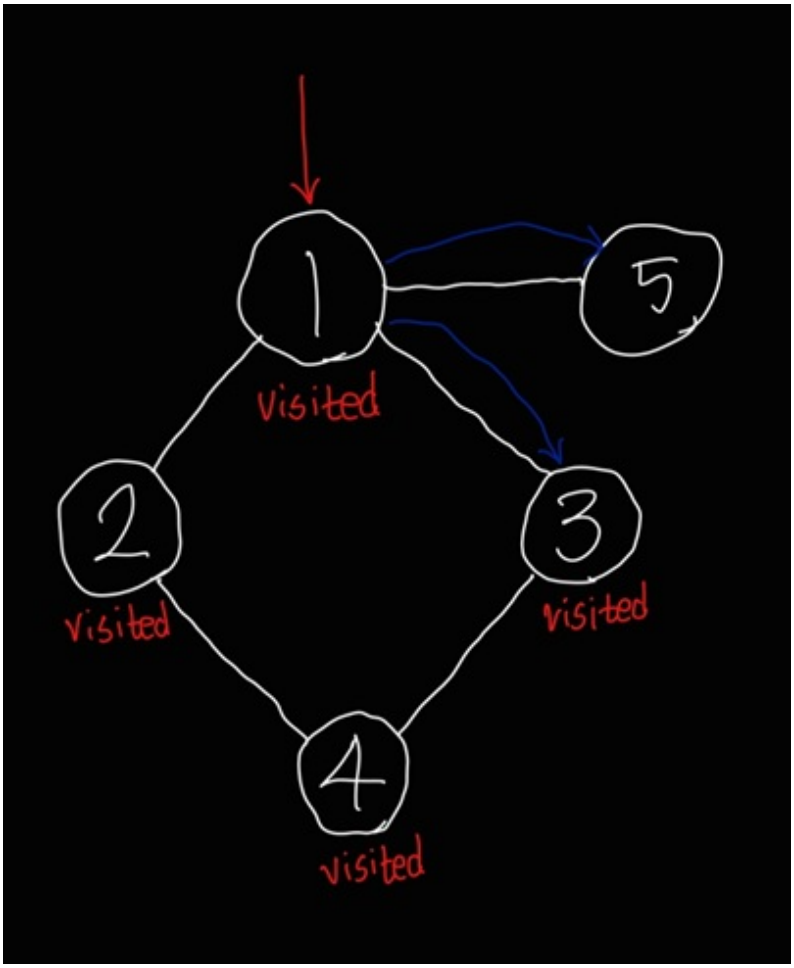


3번 정점에서 **방문하지 않은 정점이 없으므로** 이전에 방문했던 **4번 정점으로 되돌아간다.**



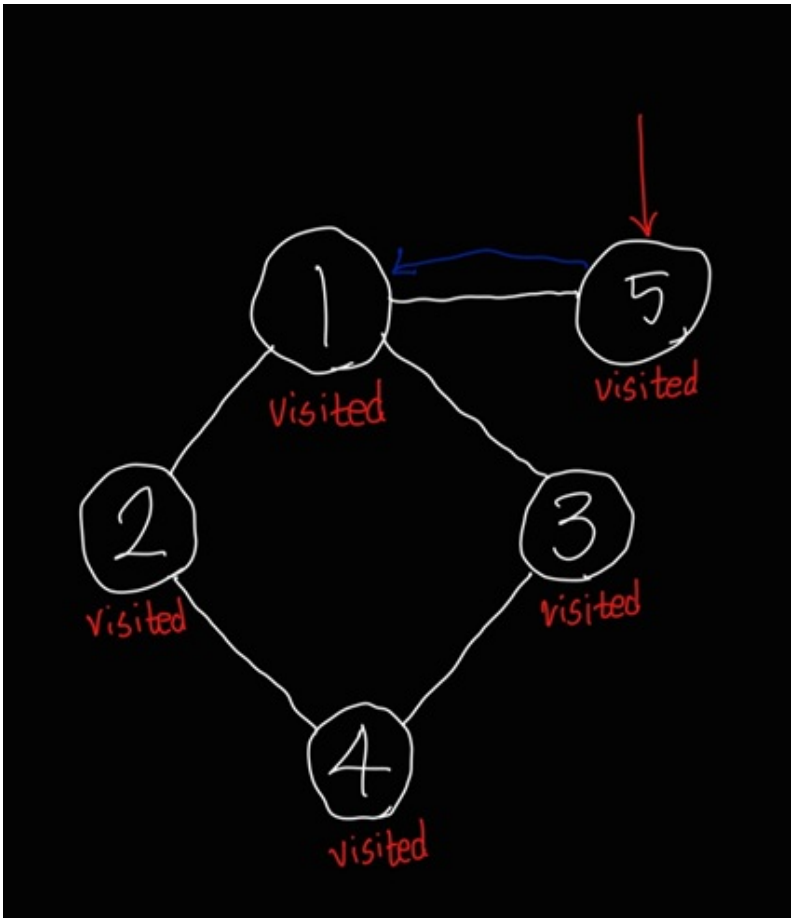
4번 정점에서도 더 이상 방문할 정점이 없으므로 이전에 방문했던 **2번 정점으로 되돌아간다.**

 CC BY-NC-SA 4.0



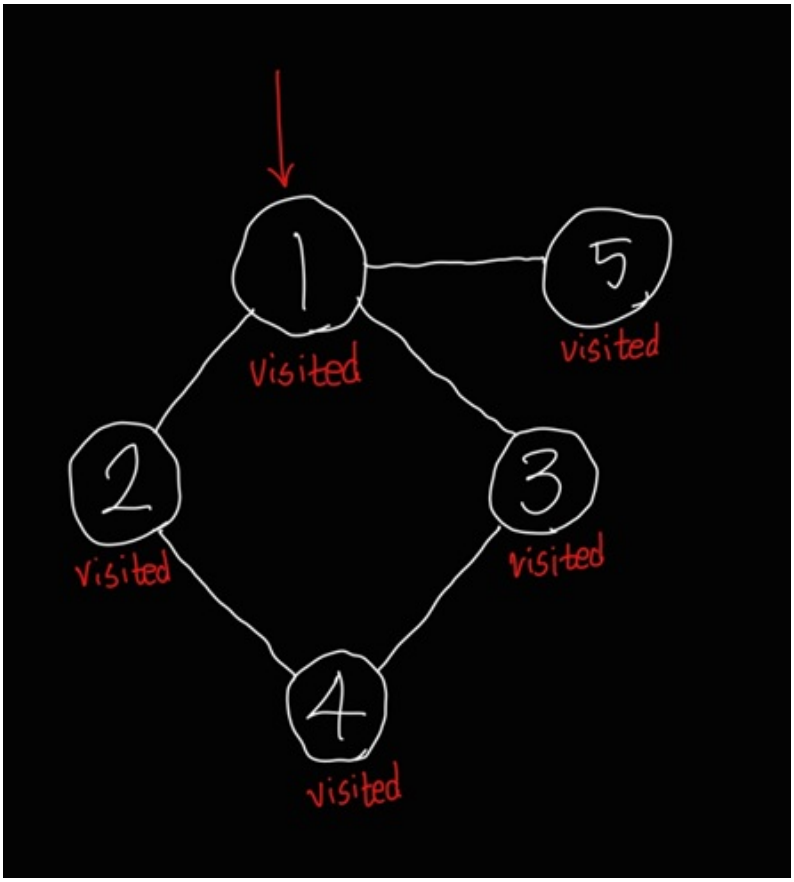
2번 정점에서도 더 이상 방문할 정점이 없어 **1번 정점으로 되돌아간다.**

 CC BY-NC-SA 4.0



1번 정점으로 돌아왔을 때 **3번 정점은 이미 방문된 상태**이므로 이동하지 않고 다음으로 번호가 작은 **5번 정점으로 이동**한다.

 CC BY-NC-SA 4.0



5번 정점에서 방문하지 않은 정점이 없으므로 **1번 정점으로 되돌아온다.** 이후 1번 정점에서도 더 이상 방문할 정점이 없으므로 **탐색이 종료된다.**

이와 같이 DFS는 한 경로를 가능한 한 깊게 탐색하다가 더 이상 방문할 수 있는 정점이 없으면 이전 정점으로 되돌아가 다른 경로를 탐색하는 방식으로 동작한다.

연습 문제: 백준 24479번

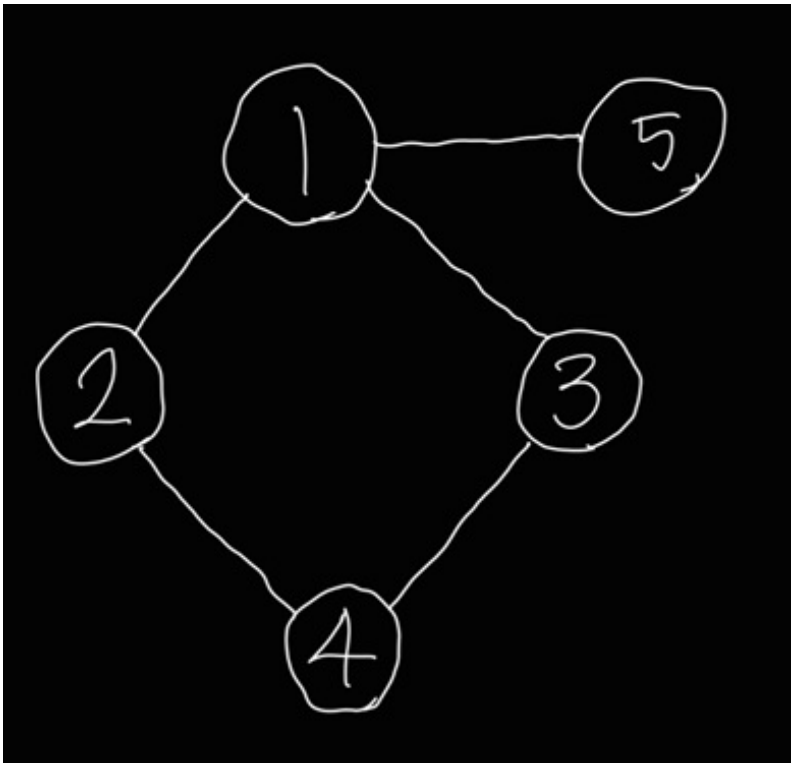
```
/** http://boj.kr/ca29bf44e6ec44d7ab2d37e0264c9879 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int cnt, visited[200'001];
vector<vector<int>> conn(200'001);

/**
 * DFS
 * 방문할 수 있는 노드가 보이면 이동.
 * 방문할 수 있는 노드가 보이지 않으면 다시 뒤로 되돌아감.
 * 미로 찾기에서 왼손의 법칙을 쓰는 것과 유사함.
 */
void dfs(int cur) {
    visited[cur] = ++cnt; // 방문 순서 마킹
    for(int next:conn[cur]) {
        if(!visited[next]) { // 방문하지 않은 정점이면 방문
            dfs(next);
        }
    }
```

```
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m, r; cin >> n >> m >> r;
    while(m--) {
        int u, v; cin >> u >> v;
        conn[u].push_back(v); // u번 노드에서 v번 노드로 이동 가능
        conn[v].push_back(u); // v번 노드에서 u번 노드로 이동 가능
    }
    for(int i=1;i<=n;i++) sort(conn[i].begin(), conn[i].end()); // 문제의 조건: 방문할
    ↪  수 있는 노드가 여러 개일 때, 오름차순으로 방문

    dfs(r); // DFS 시작
    for(int i=1;i<=n;i++) cout << visited[i] << '\n';
}
```

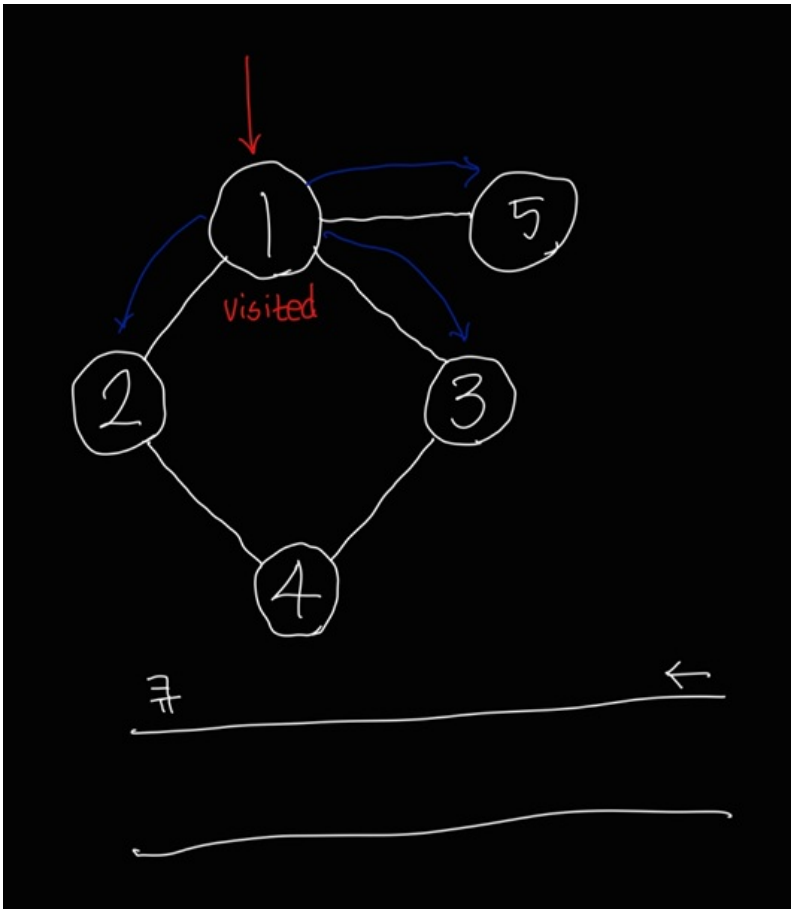
3.1.2 BFS (Breadth First Search, 너비 우선 탐색) Silver II

그래프에서 시작 정점으로부터 가까운 정점부터 차례대로 탐색하는 알고리즘

시간복잡도: $O(V+E)$ (V: 정점 수, E: 간선 수)

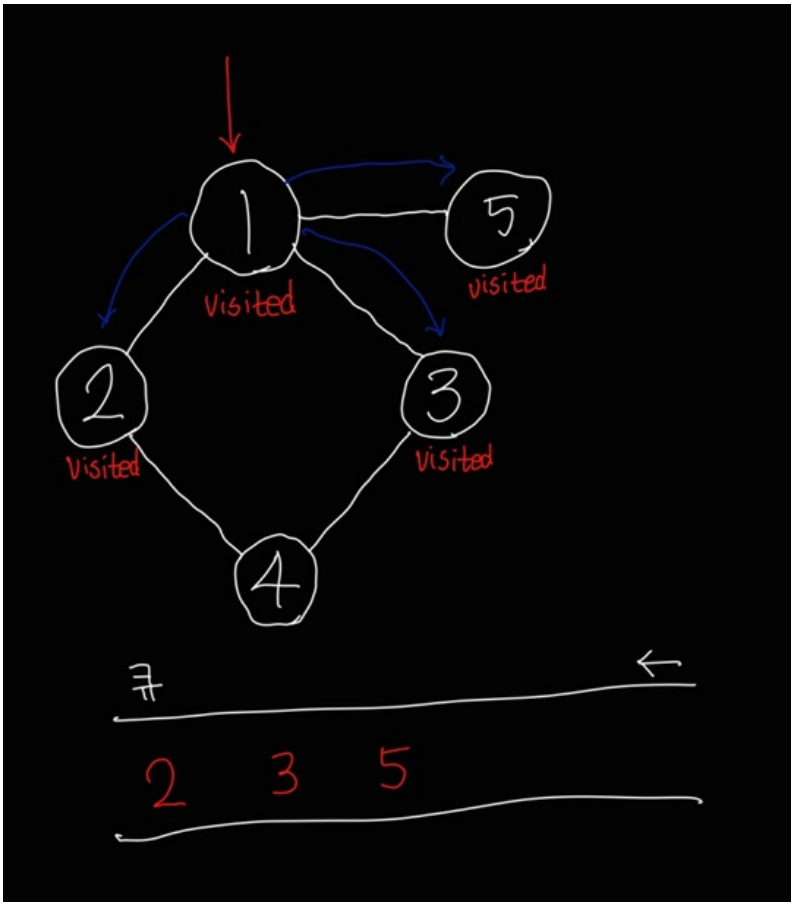


위와 같이 연결된 그래프가 있다고 하자. 탐색은 **1번 정점에서 시작**하며 동시에 여러 정점으로 이동할 수 있는 경우 **정점 번호가 작은 순서**로 방문한다고 가정한다.

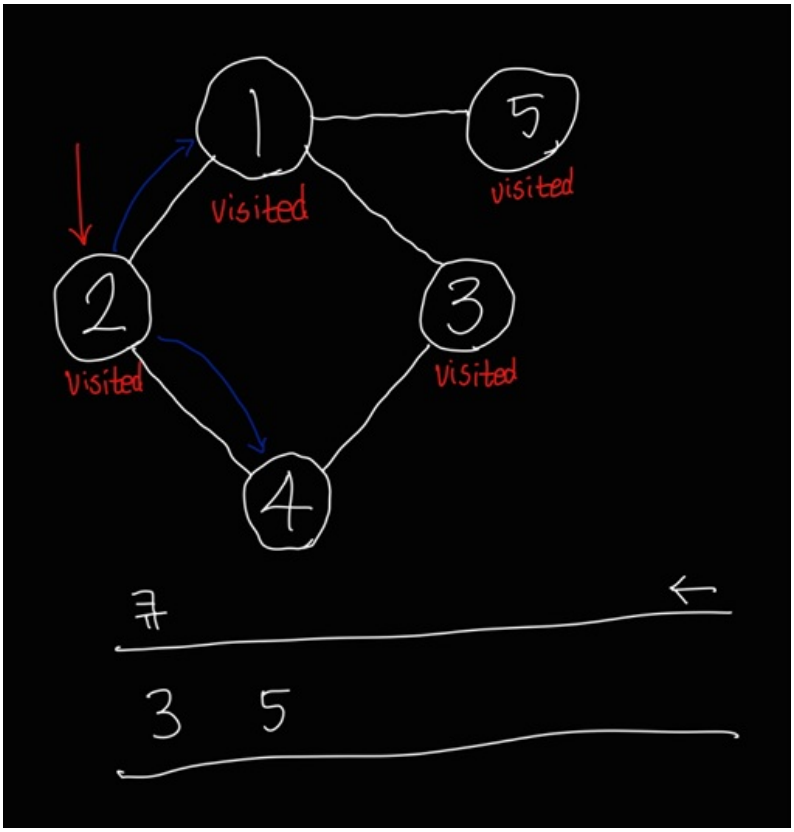


처음에는 **1번 정점**에서 탐색을 시작한다. 1번 정점과 연결된 정점은 **2번, 3번, 5번**이다.

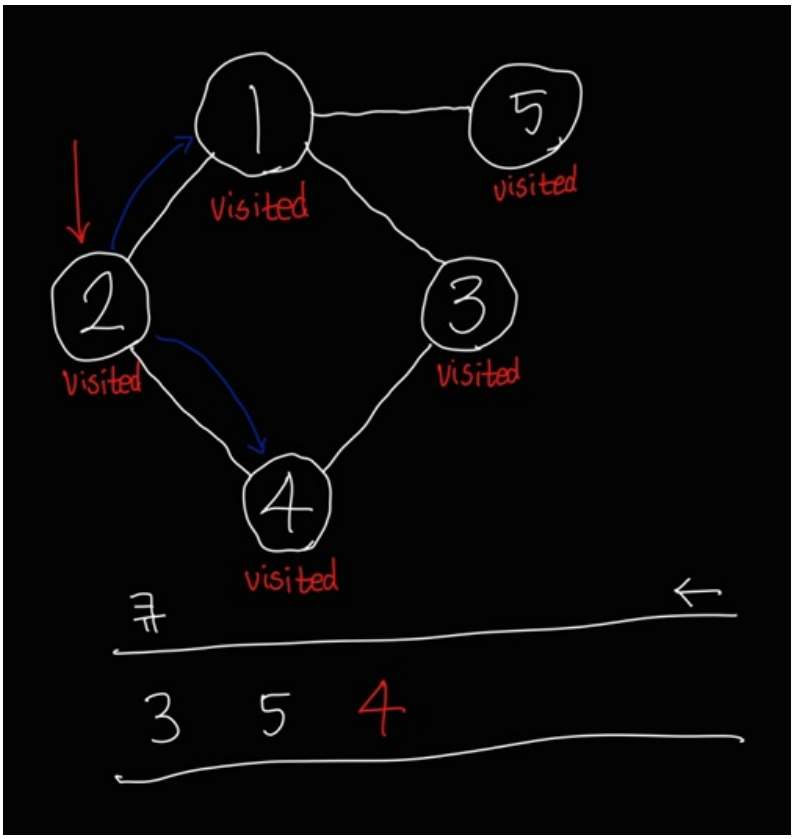
 CC BY-NC-SA 4.0



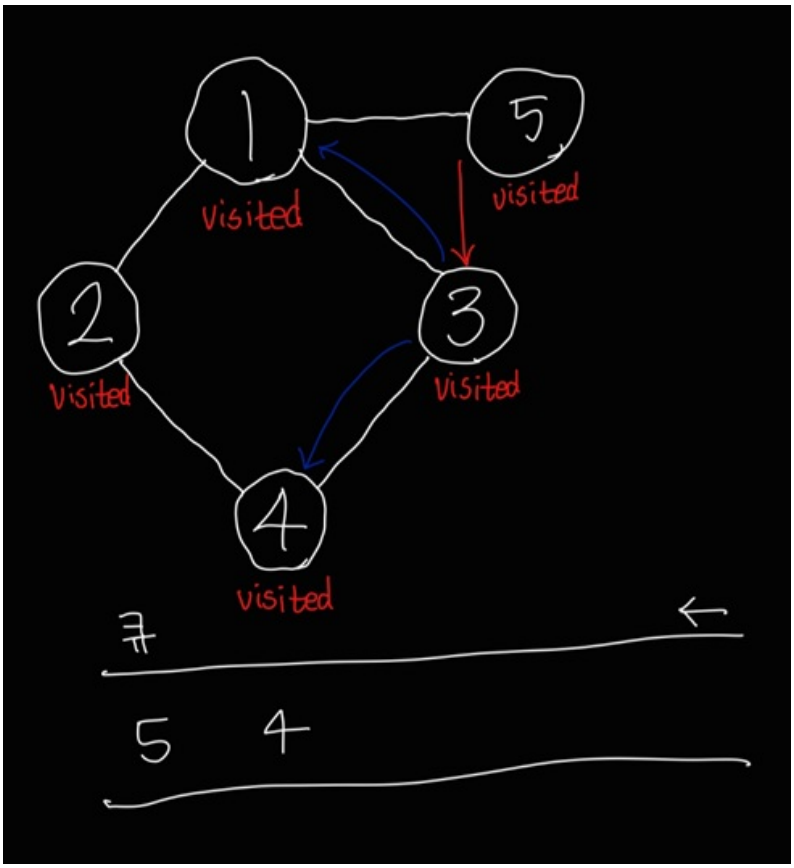
아직 방문하지 않은 정점인 **2번, 3번, 5번 정점을 큐에 순서대로 넣는다.**



큐의 맨 앞에 있는 **2번 정점**을 꺼내 방문한다. 2번 정점과 연결된 정점은 **1번, 4번**이다.

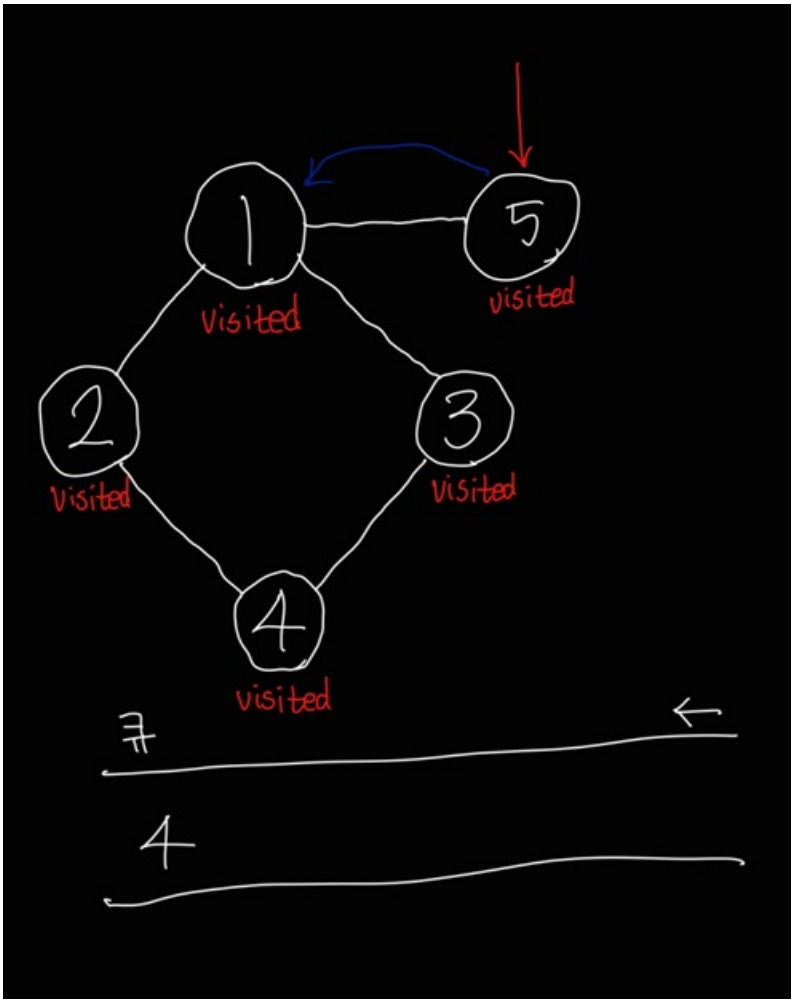


이미 방문한 **1번 정점을 제외**하고 **4번 정점을 큐에 넣는다.**



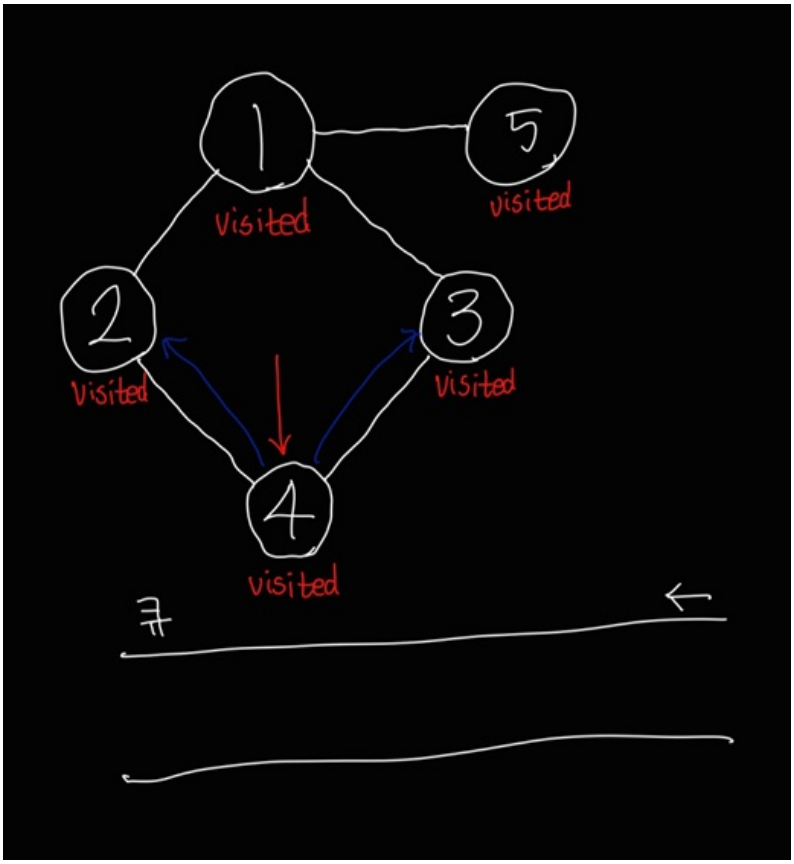
 CC BY-NC-SA 4.0

다음으로 큐의 맨 앞에 있는 **3번 정점**을 방문한다. 3번 정점과 연결된 정점은 **1번, 4번**이지만 모두 이미 방문했거나 큐에 들어 있으므로 추가되는 정점은 없다.



다음으로 큐의 맨 앞에 있는 **5번 정점**을 방문한다. 5번 정점과 연결된 정점은 **1번 정점**뿐이며 이미 방문했으므로 큐에 추가되는 정점은 없다.

 CC BY-NC-SA 4.0



마지막으로 큐의 맨 앞에 있는 **4번 정점**을 방문한다. 4번 정점과 연결된 정점은 **2번, 3번**이지만 이미 방문했으므로 큐에 추가되는 정점은 없다.

이후 **큐가 비게 되므로 탐색이 종료된다.**

연습 문제: 백준 24444번

```
/** http://boj.kr/5a29153331384b3cb8dc9d68127f6b34 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int cnt, visited[200'001];
vector<vector<int>> conn(200'001);

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m, r; cin >> n >> m >> r;
    while(m--) {
        int u, v; cin >> u >> v;
        conn[u].push_back(v); // u번 노드에서 v번 노드로 이동 가능
        conn[v].push_back(u); // v번 노드에서 u번 노드로 이동 가능
    }
    for(int i=1;i<=n;i++) sort(conn[i].begin(), conn[i].end()); // 문제의 조건: 방문할
    ↪  수 있는 노드가 여러 개일 때, 오름차순으로 방문

    /**
     * BFS
     * 가까운 위치의 정점부터 탐색하는 알고리즘
```

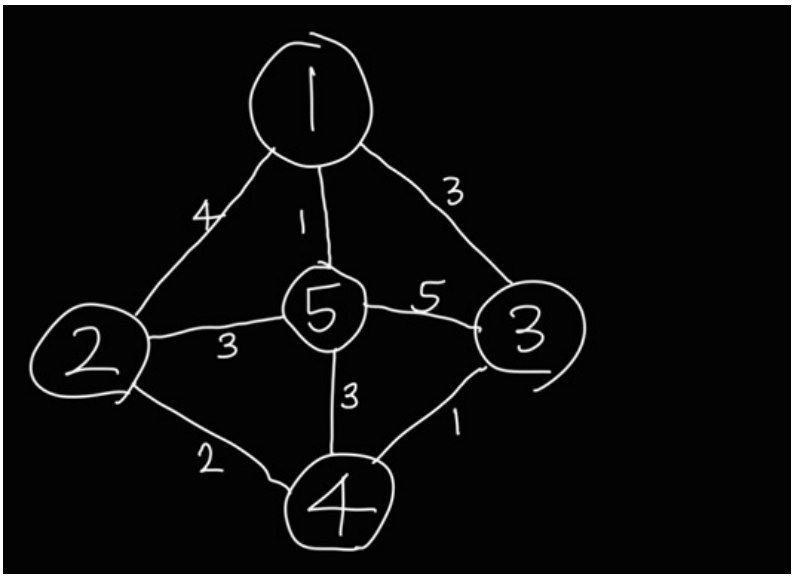
```
     * 방문할 수 있는 노드가 보이면 큐에 넣는다.
     */
    queue<int> q; q.push(r);
    visited[r] = ++cnt;
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        for(int next: conn[cur]) {
            if(!visited[next]) { // 방문하지 않은 정점이면 방문
                visited[next] = ++cnt;
                q.push(next);
            }
        }
    }
    for(int i=1;i<=n;i++) cout << visited[i] << '\n';
}
```

3.2 최단 경로

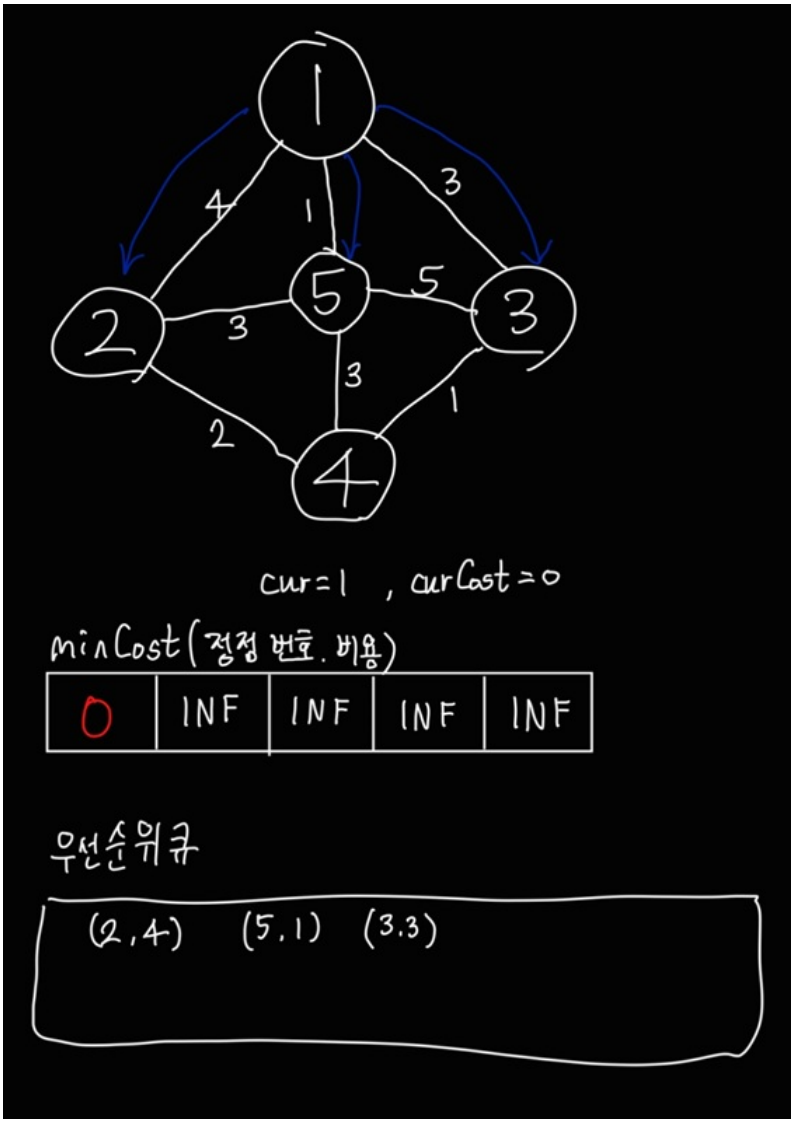
3.2.1 Dijkstra's Algorithm Gold IV

가중치가 음수가 없는 그래프에서 시작 정점으로부터의 최단 거리를 구하는 알고리즘

시간복잡도: $O(ElogV)$ (V: 정점 수, E: 간선 수)

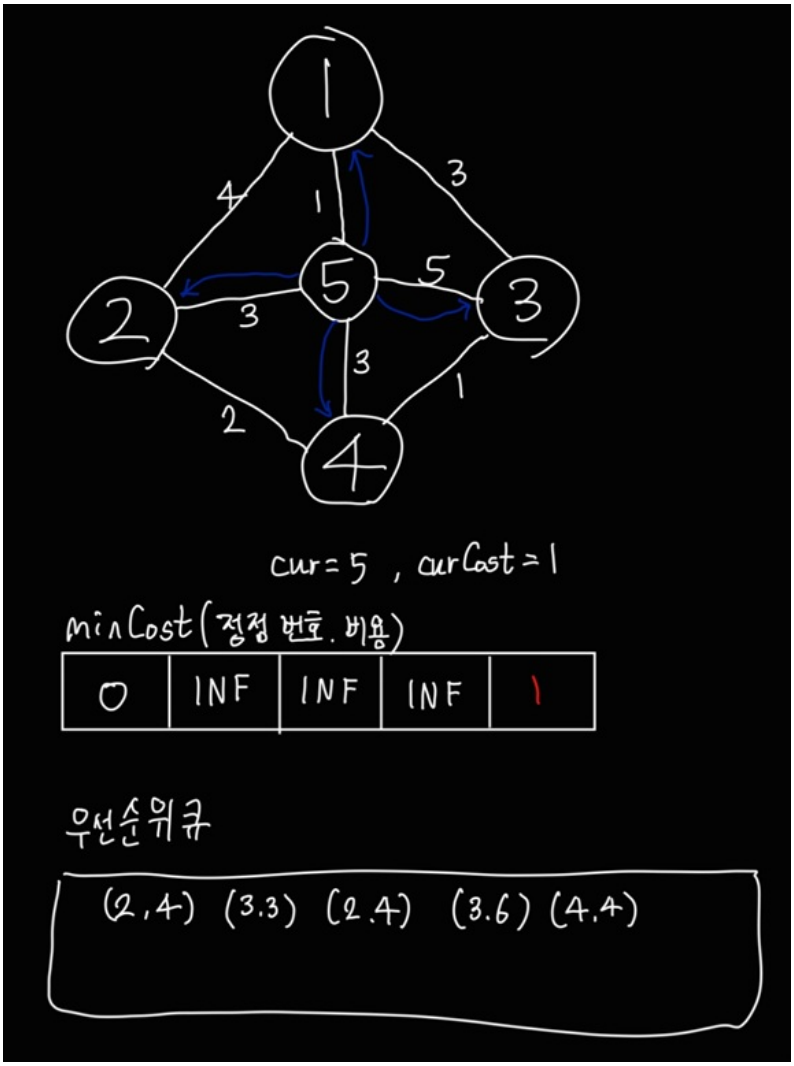


위와 같이 연결된 그래프가 있고, **1번 정점에서 탐색을 시작**한다고 가정하자.

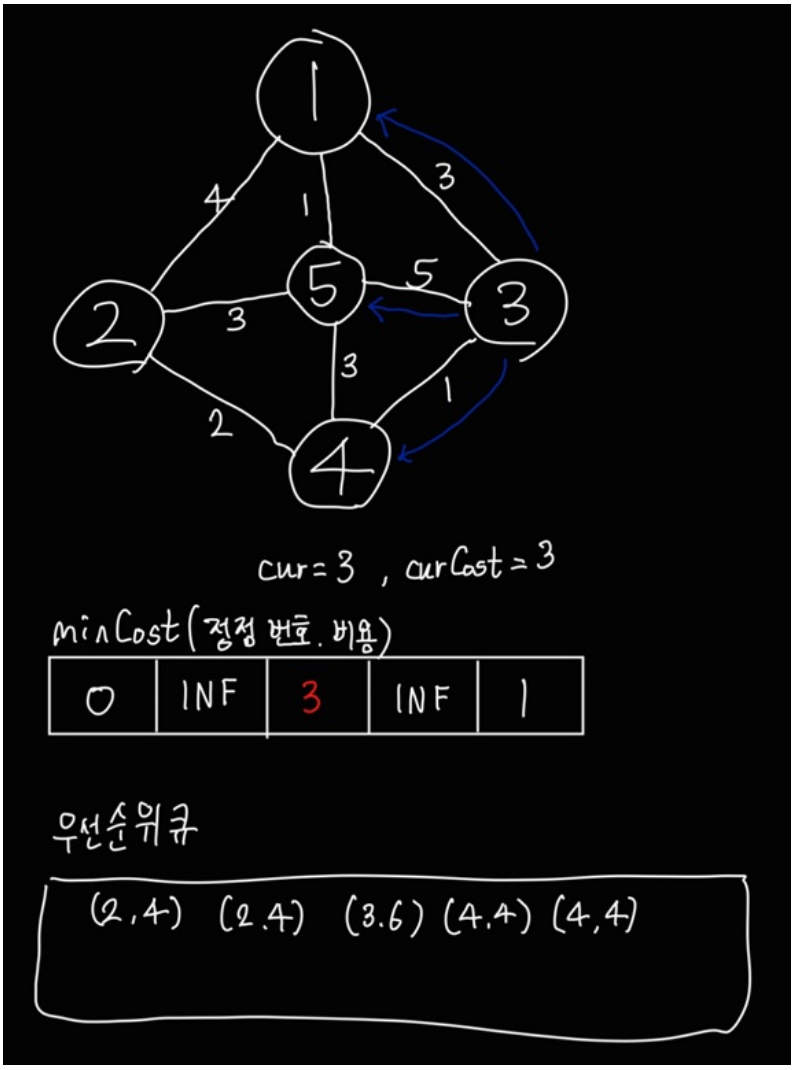


먼저 **1번 정점**을 방문한다. 1번 정점과 연결된 정점은 **2번, 3번, 5번**이다. 이때 만들 수 있는 (정점, 비용) 쌍은 **(2, 4), (5, 1), (3, 3)**이며 이를 **우선순위 큐에 넣는다.**

 CC BY-NC-SA 4.0

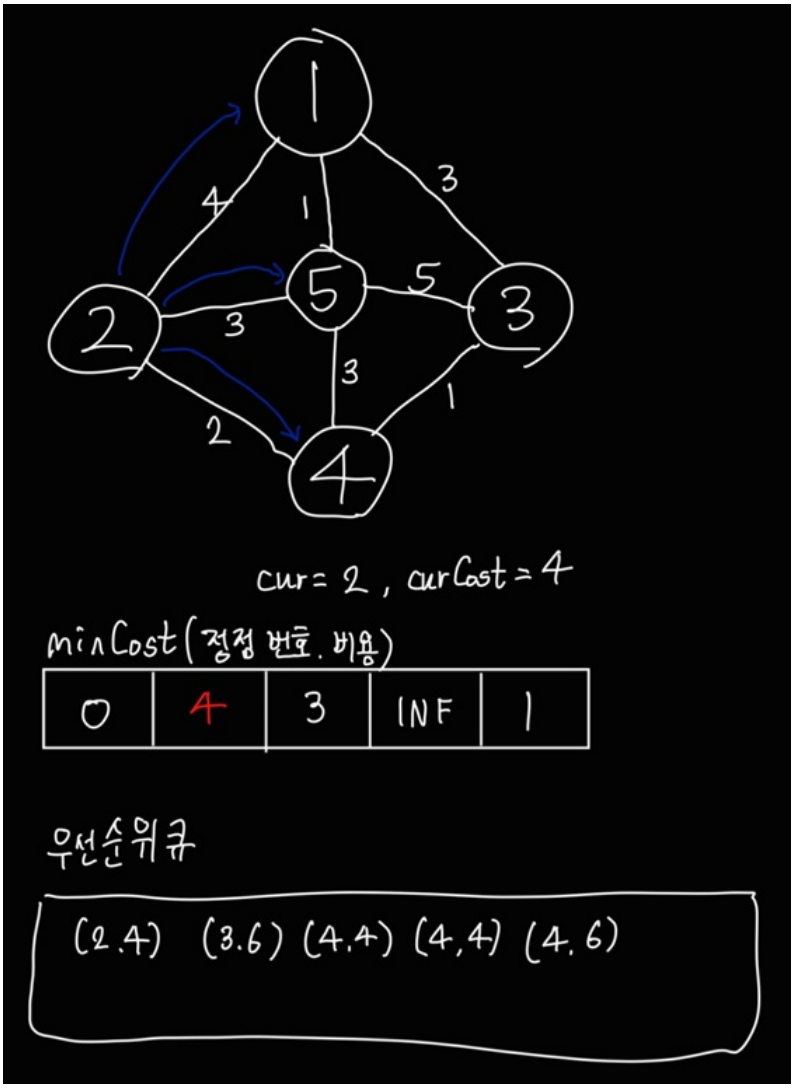


우선순위 큐에서 비용이 가장 작은 쌍인 **(5, 1)**을 꺼내고 **5번 정점의 최단 거리를 1로 기록한다.** 5번 정점에서 갈 수 있는 정점은 **1번, 2번, 3번, 4번**이며 새로 계산되는 비용은 **(1, 2), (2, 4), (3, 6), (4, 4)**이다. 이미 기록된 최단 거리보다 **더 작은 경우만** 우선순위 큐에 넣으므로 **(2, 4), (3, 6), (4, 4)**를 큐에 추가한다.

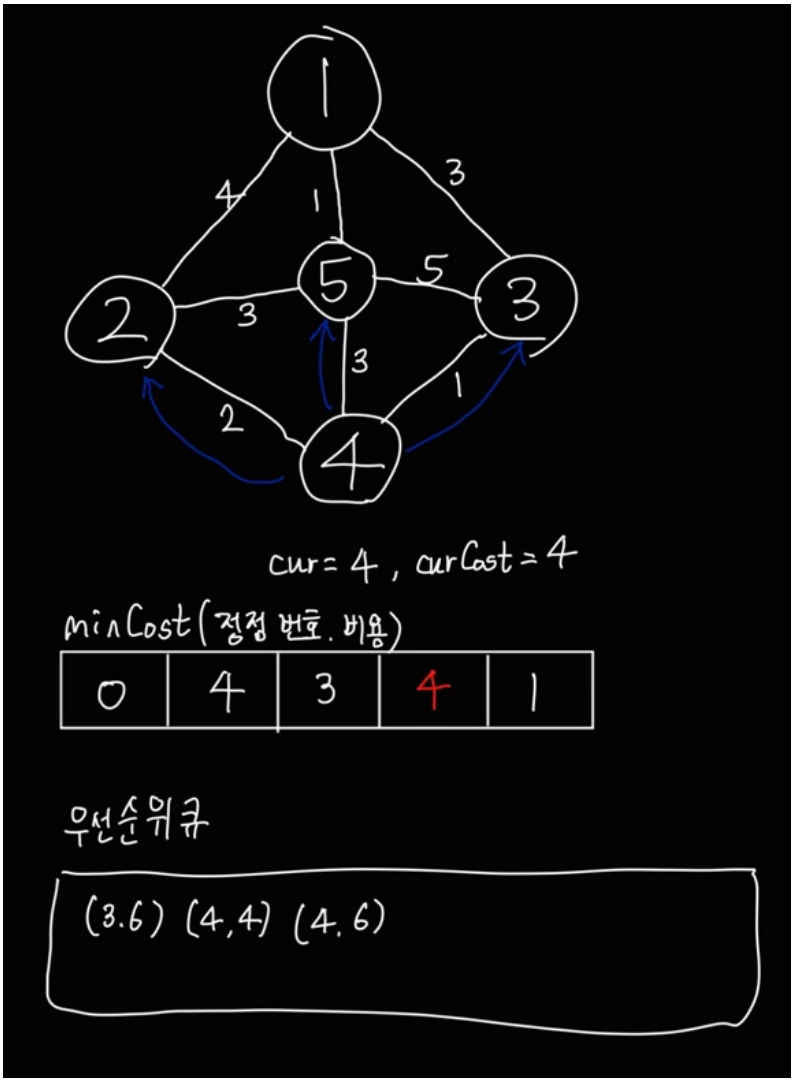


다음으로 우선순위 큐에서 **(3, 3)**을 꺼내고 **3번 정점의 최단 거리를 3으로 기록한다.** 3번 정점에서 갈 수 있는 정점은 **1번, 4번, 5번**이며 새로 계산되는 비용은 **(1, 6), (4, 4), (5, 8)**이다. 이 중 기존 최단 거리보다 작은 경우만 큐에 넣으므로 **(4, 4)**만 추가된다.

 CC BY-NC-SA 4.0



다음으로 **(2, 4)**를 꺼내고 **2번 정점의 최단 거리를 4로 기록한다.** 2번 정점에서 갈 수 있는 정점은 **1번, 4번, 5번**이며 새로 계산되는 비용은 **(1, 8), (4, 6), (5, 7)**이다. 이 중 기존 최단 거리보다 작은 경우는 **(4, 6)**뿐이므로 이를 우선순위 큐에 넣는다.



다음으로 큐에서 (2, 4)가 나오지만 이미 더 짧은 거리로 처리된 상태이므로 **무시한다.** 그 다음 (4, 4)를 꺼내고 **4번 정점의 최단 거리를 4로 기록한다.** 4번 정점에서 갈 수 있는 정점은 **2번, 3번, 5번**이며 새로 계산되는 비용은 (2, 6), (3, 5), (5, 7)이다. 모두 이미 기록된 최단 거리보다 크므로 **우선순위 큐에 추가하지 않는다.**

이후 더 이상 최단 거리를 갱신할 수 있는 정점이 없으므로 **탐색이 종료된다.**

연습 문제: 백준 1753번

```
/** http://boj.kr/e655a4c35f9b4f9b8862d49ff4009992 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;
const int MAX = 20001;

struct element {
    int pos, cost;
    bool operator<(const element e) const {
        return cost > e.cost; // 우선순위 큐라 일반 정렬과 부호 반대
    }
```

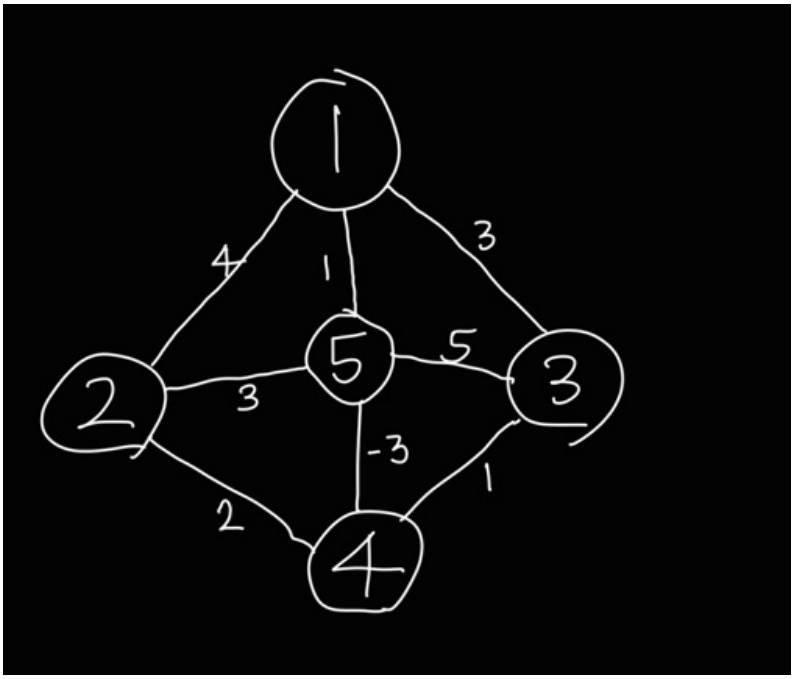
```
};

int minCost[MAX]; // 최단 거리를 저장하는 배열
vector<vector<element>> conn(MAX);

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int v, e, k; cin >> v >> e >> k;
    while(e--) {
        int u, v, w; cin >> u >> v >> w;
        conn[u].push_back({v, w}); // 단방향 간선
    }

    /**
     * Dijkstra Algorithm
     * 우선순위 큐에서 비용이 작은 경로부터 꺼내 탐색함
     */
    fill(minCost, minCost+MAX, INF); // 기본값 초기화
    priority_queue<element> pq; pq.push({k, 0}); // 시작 정점 넣기
    while(!pq.empty()) {
        auto cur = pq.top(); pq.pop();
        if(minCost[cur.pos]<=cur.cost) continue; // 현재 경로가 비용이 더 크다면 제외
        minCost[cur.pos] = cur.cost;

        for(auto next:conn[cur.pos]) {
            int nextCost = cur.cost + next.cost;
            if(nextCost<minCost[next.pos]) { // 현재 경로로 다음 정점에 도달하는 비용이
            ↪ 더 적다면
                pq.push({next.pos, nextCost});
            }
        }
    }

    for(int i=1;i<=v;i++) {
        if(minCost[i]==INF) cout << "INF\n";
        else cout << minCost[i] << '\n';
    }
}
```

3.2.2 Bellman-Ford Algorithm Gold IV

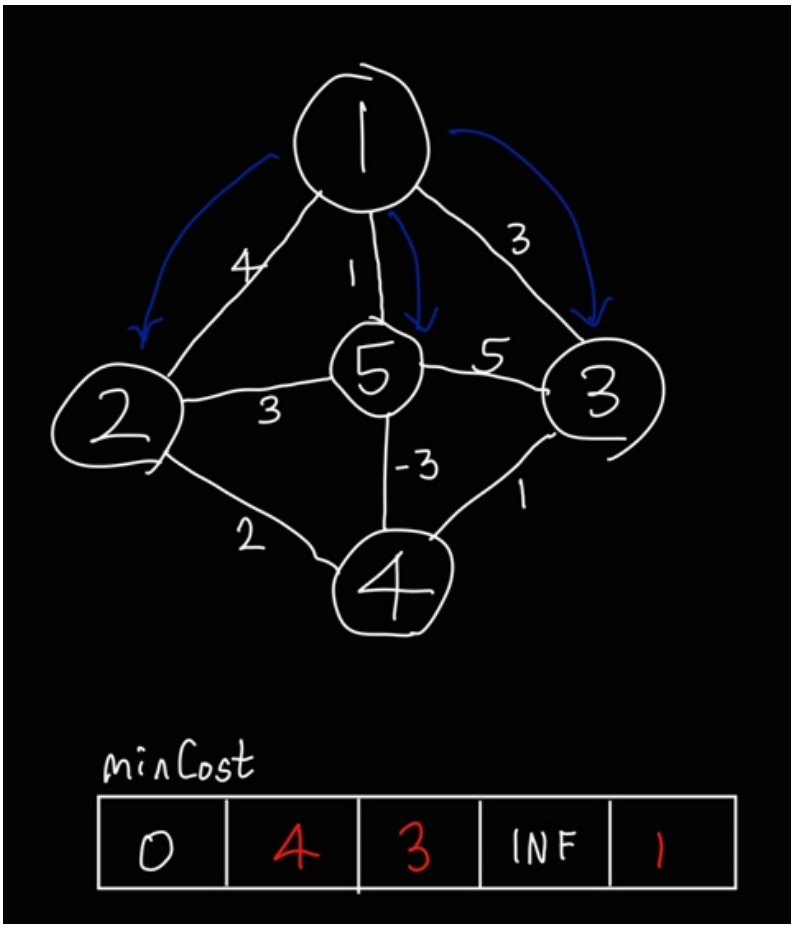
가중치가 음수인 그래프에서도 최단 거리를 구하고 음수 사이클의 존재 여부를 판별하는 알고리즘

시간복잡도: $O(VE)$ (V: 정점 수, E: 간선 수)

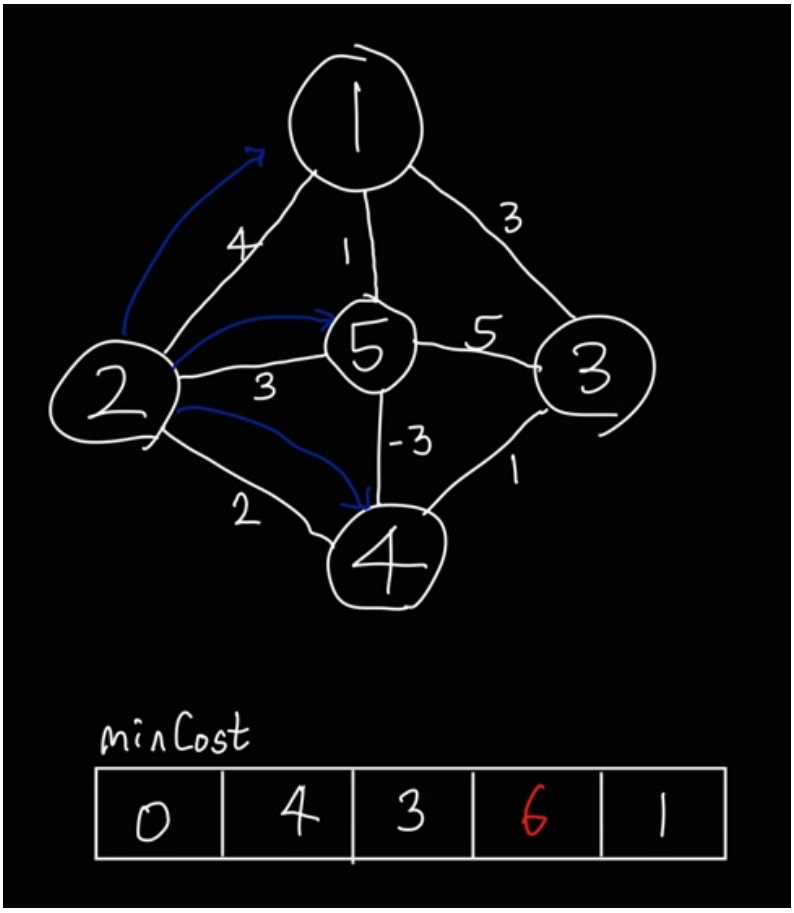
 CC BY-NC-SA 4.0



위와 같이 연결된 그래프가 있고 **1번 정점에서 탐색을 시작**한다고 가정하자.

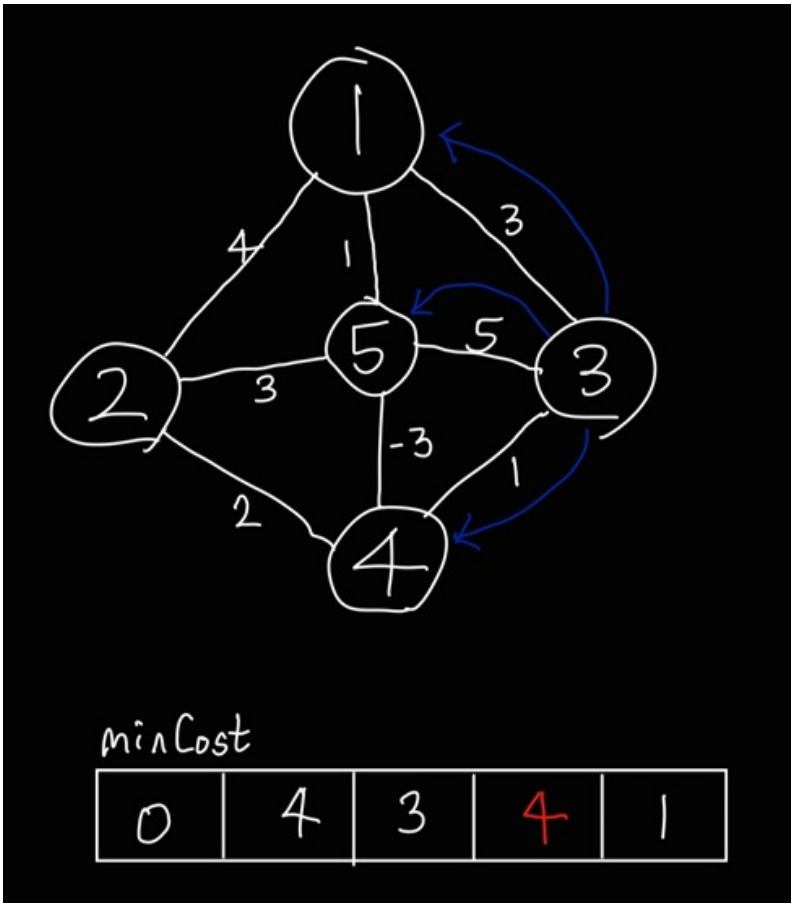


먼저 1번 정점에서 시작하여 연결된 모든 간선을 확인하며 다른 정점들의 최소 비용을 갱신한다.



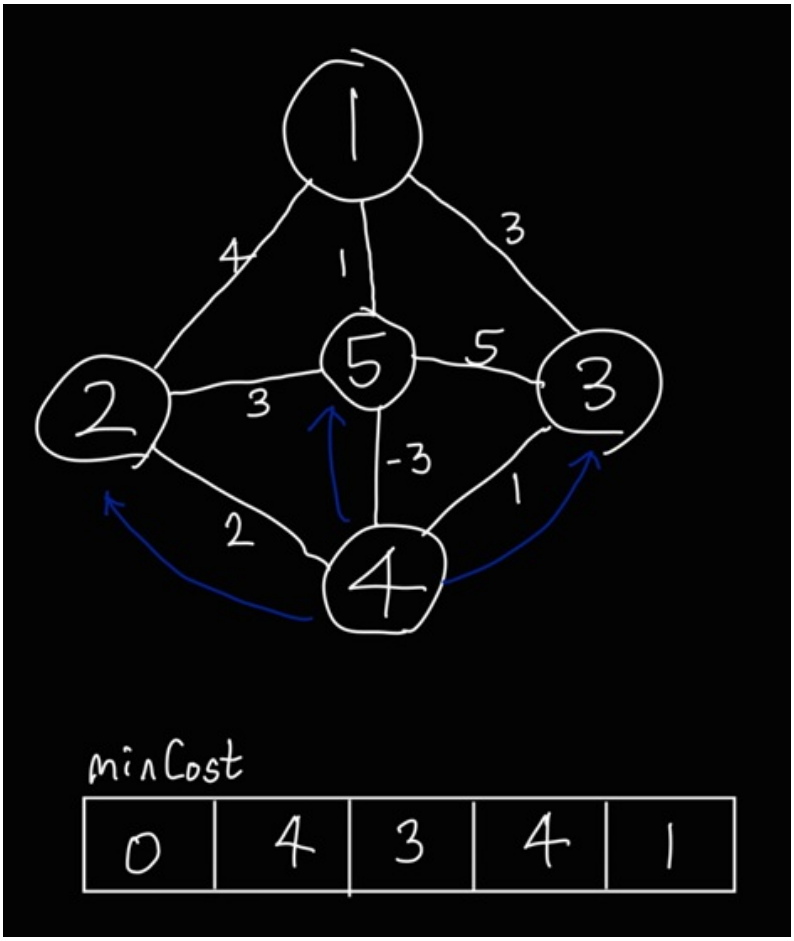
다음으로 2번 정점에서도 연결된 모든 간선을 확인하며 최소 비용을 갱신한다.

 CC BY-NC-SA 4.0



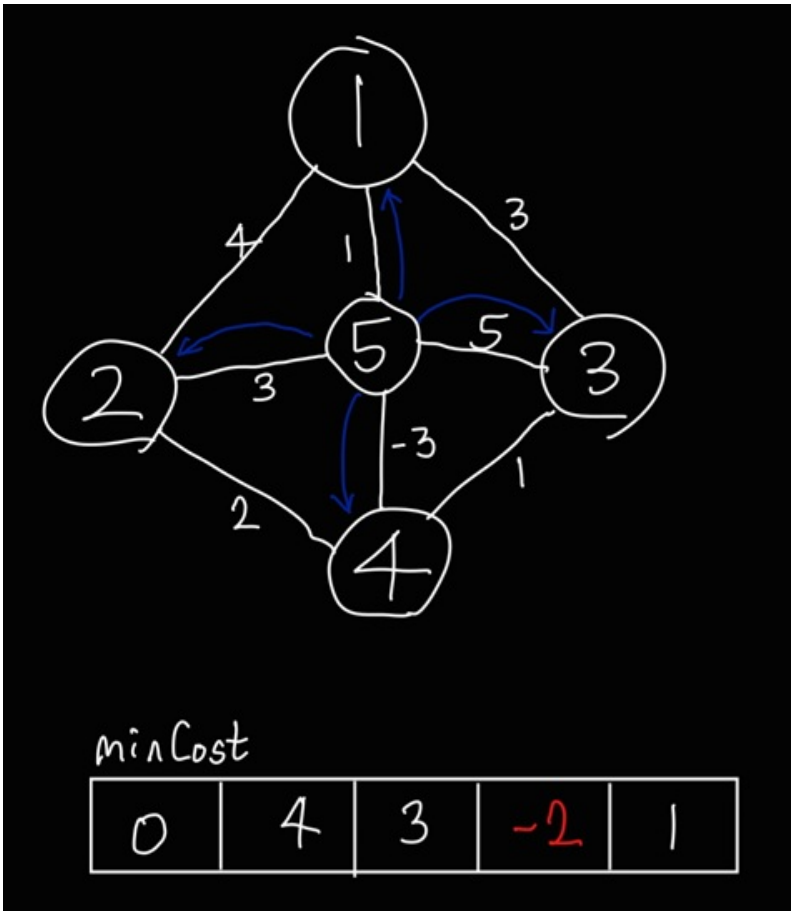
같은 방식으로 3번 정점에서도 연결된 모든 간선을 확인하며 최소 비용을 갱신한다.

 CC BY-NC-SA 4.0



이어서 4번 정점에서도 연결된 모든 간선을 확인하며 최소 비용을 갱신한다.

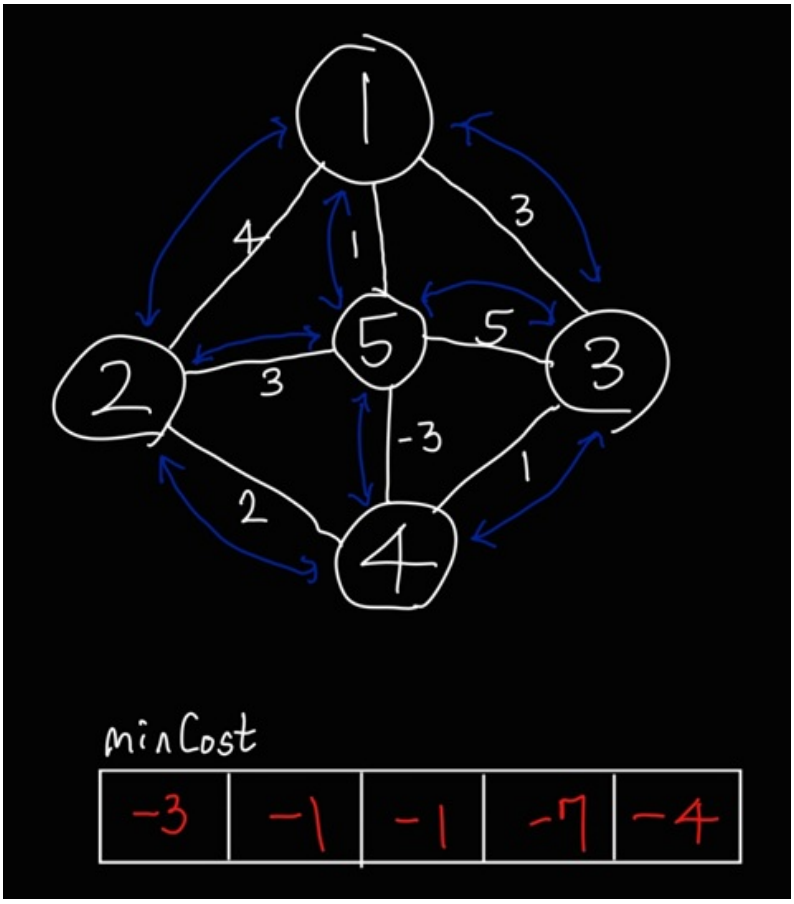
 CC BY-NC-SA 4.0



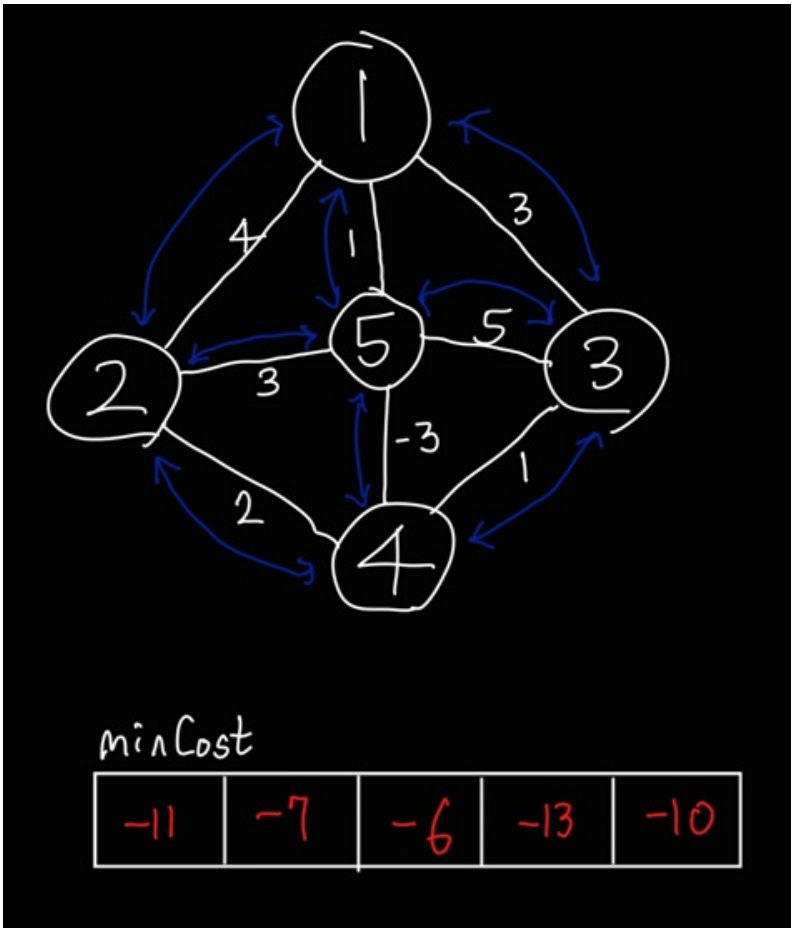
마지막으로 5번 정점에서도 연결된 모든 간선을 확인하며 최소 비용을 갱신한다.

이로써 **첫 번째 반복**이 끝난다. 이번 반복에서 최단 거리 갱신이 발생했으므로 다음 반복을 계속 진행한다. (한 번의 반복에서 갱신이 전혀 발생하지 않으면 알고리즘을 종료한다.)

 CC BY-NC-SA 4.0



두 번째 반복의 결과는 다음과 같다. 이 반복에서도 최단 거리 갱신이 발생했으므로 다음 반복을 진행한다.



세 번째 반복의 결과는 다음과 같다. 이처럼 모든 간선을 반복적으로 확인하며 최소 비용을 계속 갱신한다.

⋮

이 과정을 반복하다가 n**번째 반복에서도 최단 거리 갱신이 발생한다면**, 그래프에는 **음수 사이클이 존재한다고 판단한다.**

그 이유는 한 번의 반복에서 최소 한 개의 정점의 거리는 반드시 갱신되기 때문이다. (갱신이 전혀 발생하지 않으면 알고리즘이 종료된다.)

시작 정점을 제외하면 거리가 갱신될 수 있는 정점은 최대 $n-1$개이므로 정상적인 경우라면 n**번째 반복에서는 더 이상 갱신이 발생할 수 없다.**

따라서 n번째 반복에서도 갱신이 발생한다면 이는 **음수 사이클로 인해 거리가 계속 감소하고 있기 때문**이다.

연습 문제: 백준 11657번

```
/** http://boj.kr/42f4592a0ec44609b3ede32dfe7aa19f 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;
const int MAX = 501;
```

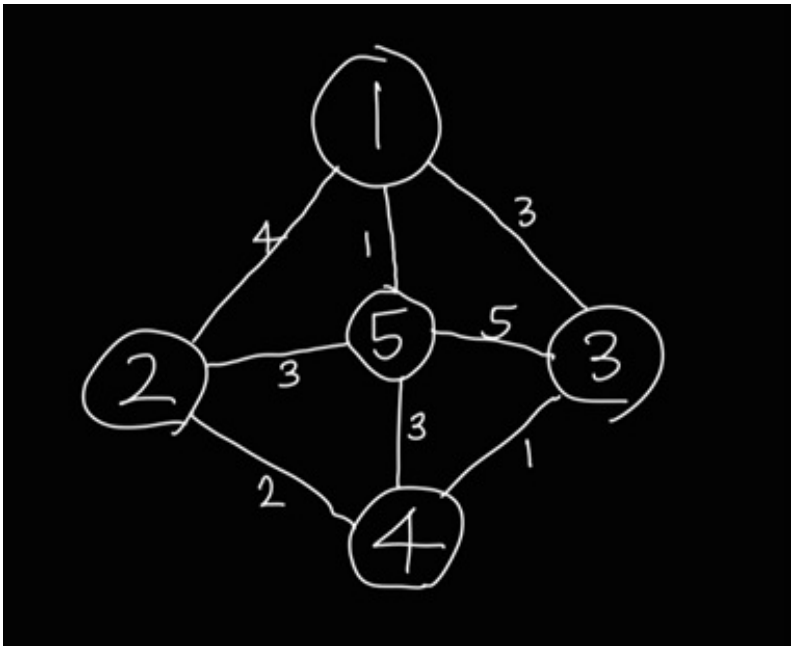
```
struct element {
    int pos, cost;
};

int n, m;
long long minCost[MAX];
vector<vector<element>> conn(MAX);

/**
 * Bellman-Ford Algorithm
 * - 음수 사이클이 존재하는 경우 return false
 */
bool bellman_ford() {
    fill(minCost, minCost+MAX, INF);
    minCost[1]=0;
    for(int i=0;i<n;i++) {
        bool change=false;
        for(int cur=1;cur<=n;cur++) { // 모든 정점에 대해 모든 간선을 살펴보기
            if(minCost[cur]==INF) continue; // 도달 못한 정점은 패스
            for(auto next:conn[cur]) {
                long long nextCost = minCost[cur] + next.cost;
                if(nextCost<minCost[next.pos]) {
                    minCost[next.pos] = nextCost;
                    change=true;
                }
            }
        }
        if(!change) return true;
    }
    return false; // n-1번의 업데이트를 진행하고도 변화가 있으면 사이클 존재
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin >> n >> m;
    while(m--) {
        int a, b, c; cin >> a >> b >> c;
        conn[a].push_back({b, c});
    }

    if(!bellman_ford()) { // 사이클이 존재하는 경우
        cout << -1;
        return 0;
    }

    for(int i=2;i<=n;i++) {
        if(minCost[i]==INF) cout << "-1\n";
        else cout << minCost[i] << '\n';
    }
}
```

 CC BY-NC-SA 4.0

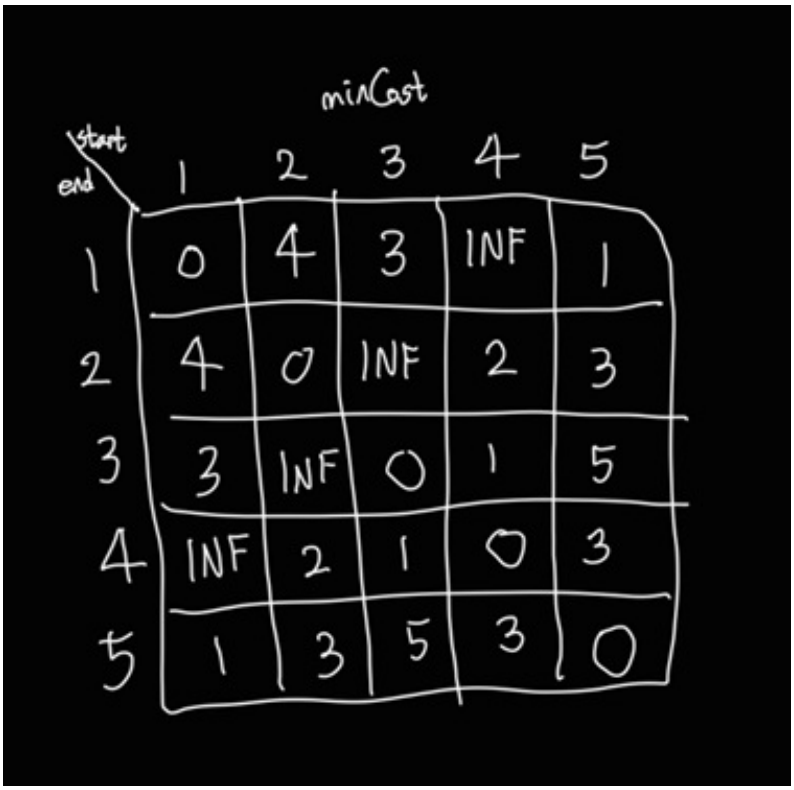
3.2.3 Floyd-Warshall Algorithm Gold IV

DP를 이용해 모든 정점 쌍 사이의 최단 거리를 구하는 알고리즘

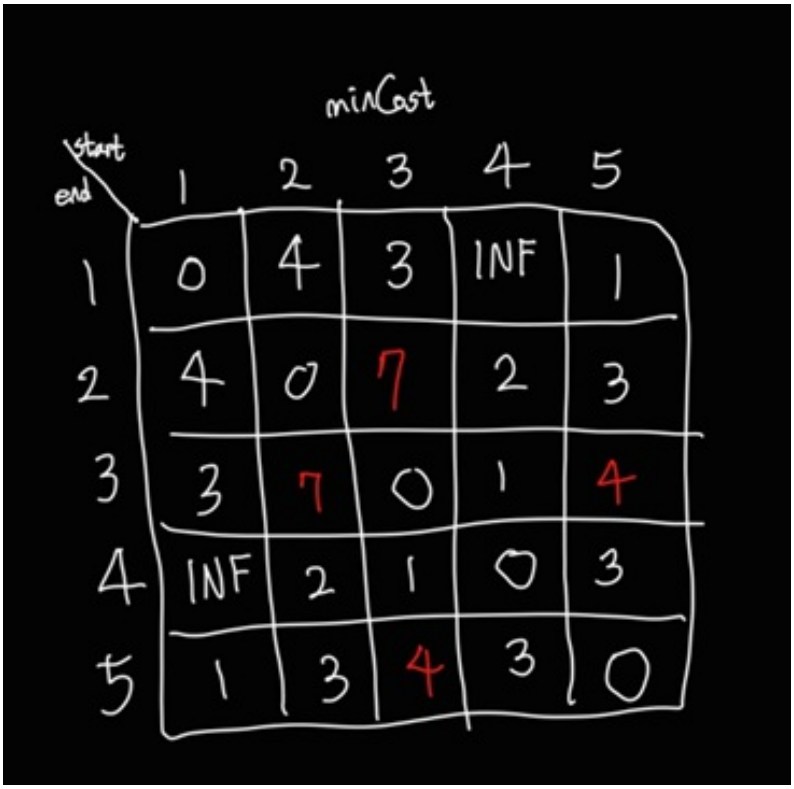
시간복잡도: $O(V^3)$, **공간복잡도**: $O(V^2)$ (V: 정점 수)



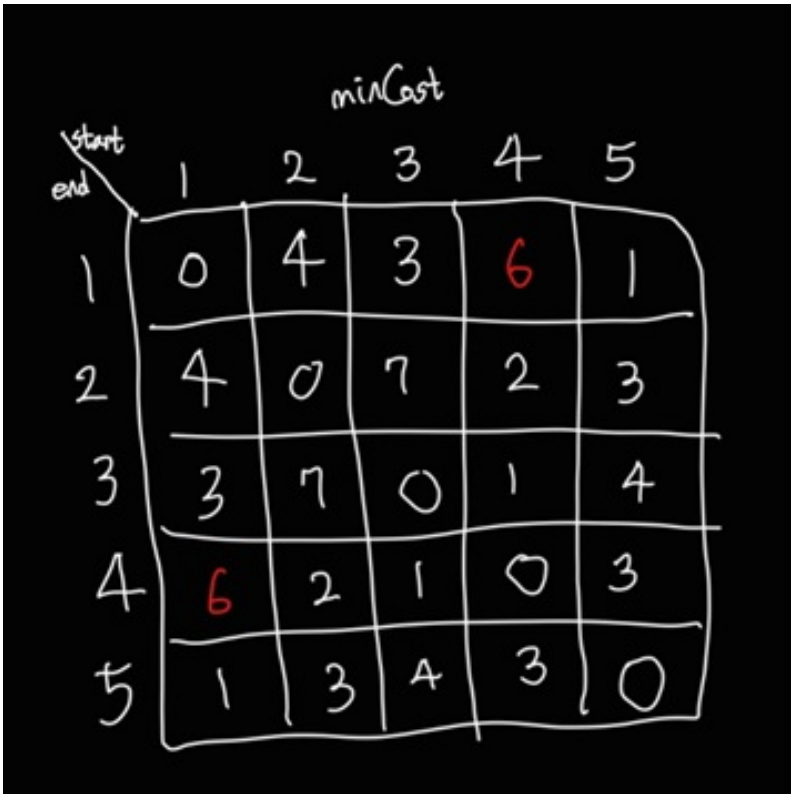
위와 같은 그래프를 예시로 사용하겠다.



그래프의 간선을 이용해 인접 행렬을 만들면 다음과 같다. 이때 i번 행, j번 열의 원소는 **i번 정점에서 j번 정점까지의 현재 최단 거리**를 의미한다.

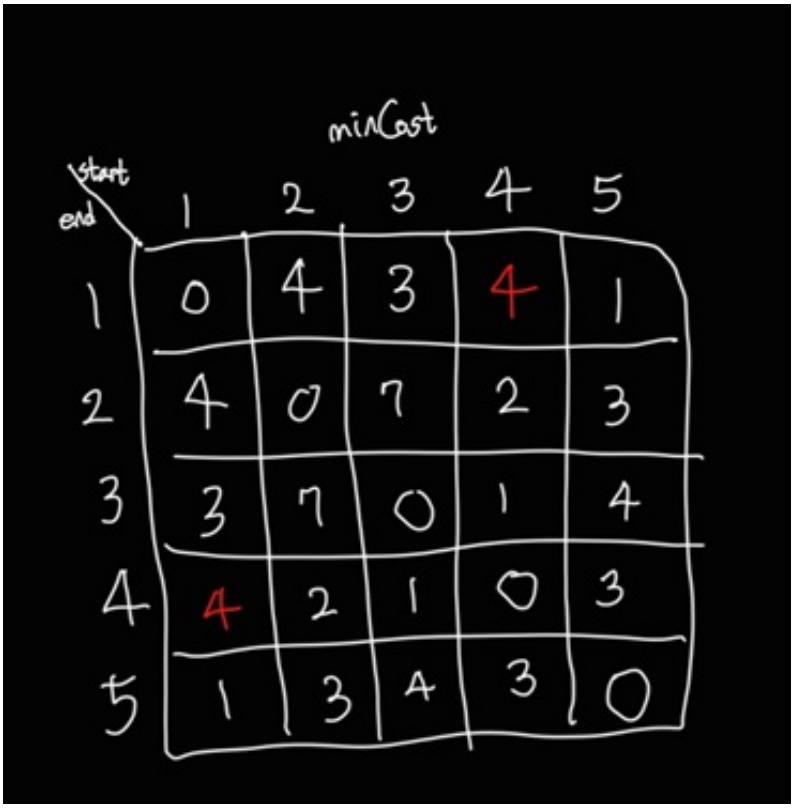


이후 **mid(경유지)**를 1로 두고 모든 $(start, end)$ 쌍에 대해 최단 거리를 갱신한다.
이 과정에서 $2 \to 1 \to 3$, $3 \to 1 \to 2$, $3 \to 1 \to 5$, $5 \to 1 \to 3$ 경로의 최단 거리가 단축된다.

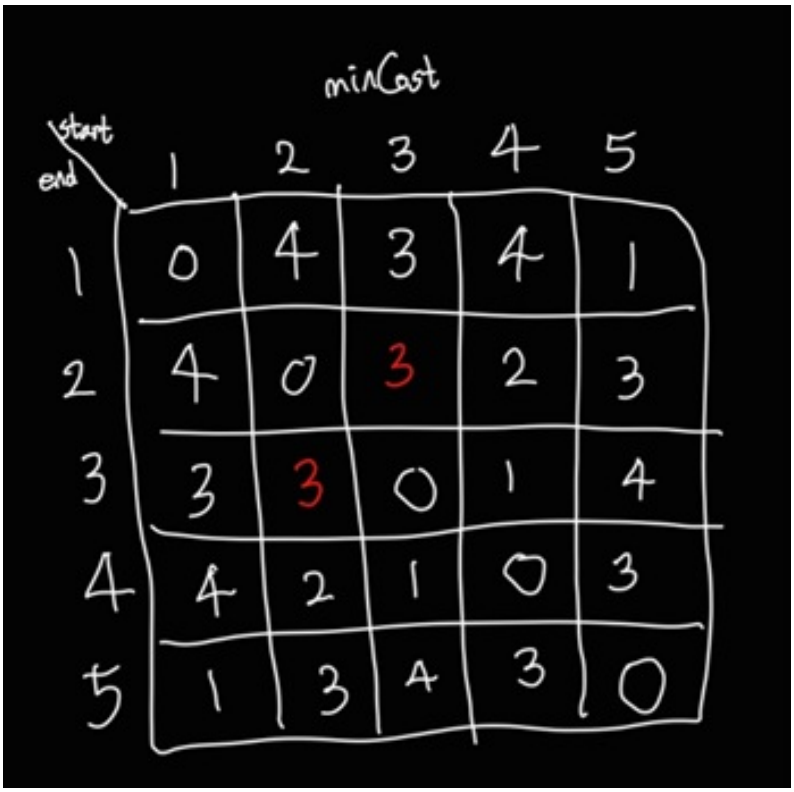


다음으로 **mid를 2로 두고** 모든 $(start, end)$ 쌍에 대해 최단 거리를 갱신한다.
이 과정에서 $1 \to 2 \to 4$, $4 \to 2 \to 1$ 경로의 최단 거리가 단축된다.

 CC BY-NC-SA 4.0

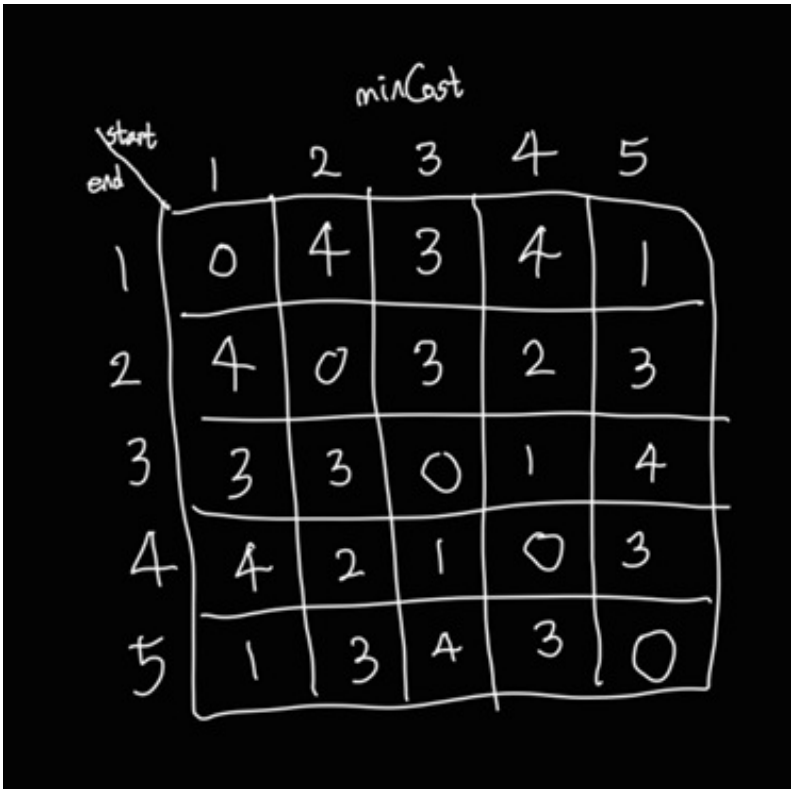


이제 **mid를** 3**으로 두고** 모든 $(start, end)$ 쌍에 대해 최단 거리를 갱신한다.
이 과정에서 $1 \to 3 \to 4$, $4 \to 3 \to 1$ 경로의 최단 거리가 단축된다.



다음으로 **mid를** 4**로 두고** 모든 $(start, end)$ 쌍에 대해 최단 거리를 갱신한다.
이 과정에서 $2 \to 4 \to 3$, $3 \to 4 \to 2$ 경로의 최단 거리가 단축된다.

 CC BY-NC-SA 4.0



마지막으로 **mid를 5로 두고** 모든 $(start, end)$ 쌍에 대해 최단 거리를 갱신한다.
하지만 $mid = 5$를 경유하는 경우에는 더 짧아지는 경로가 없으므로 행렬은 더 이상 변하지 않는다.

연습 문제: 백준 11404번

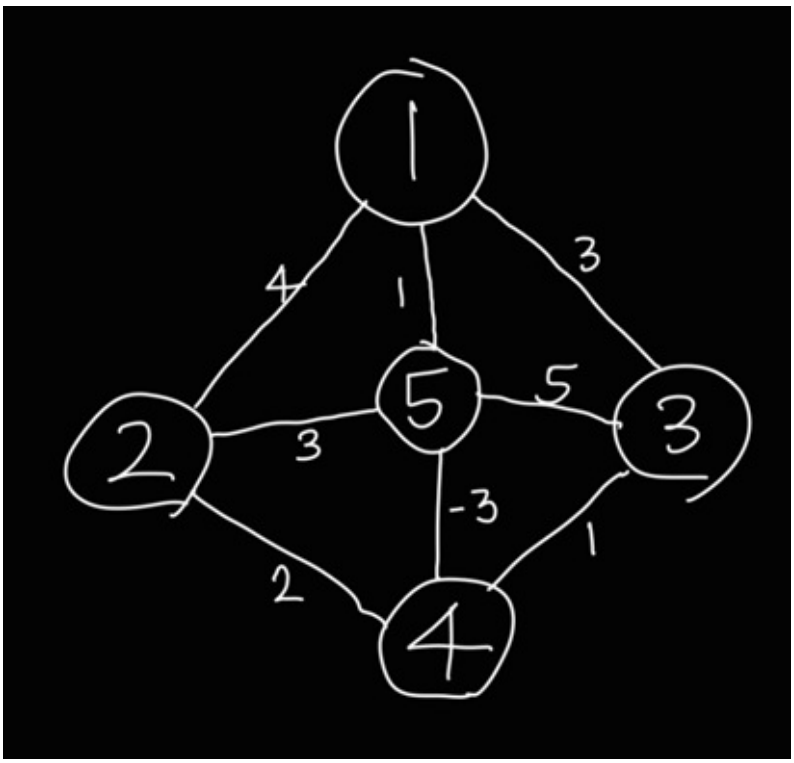
```
/** http://boj.kr/16aa77515ca84d378b5b10dabfa5ba2a 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;
const int MAX = 101;

int minCost[MAX][MAX]; // minCost[i][j] : i에서 j로 이동할 때의 최소 비용

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;

    fill(&minCost[0][0], &minCost[MAX-1][MAX], INF);
    for(int i=1;i<=n;i++) minCost[i][i]=0;
    while(m--) {
        int a, b, c; cin >> a >> b >> c;
        minCost[a][b] = min(minCost[a][b], c);
    }

    /**
     * Floyd-Warshall Algorithm
     * 모든 정점 쌍 최단거리 계산
     * - mid : 경유지
     * - start : 출발지
```

```
     * - end : 도착지
     *
     * mid를 하나씩 늘려가며 start → end 경로와 start → mid → end 경로 중 더 짧은
     ↪  경로를 선택
     */
    for(int mid=1;mid<=n;mid++) {
        for(int start=1;start<=n;start++) {
            for(int end=1;end<=n;end++) {
                minCost[start][end] = min(minCost[start][end],
                ↪  minCost[start][mid]+minCost[mid][end]);
            }
        }
    }

    for(int i=1;i<=n;i++) {
        for(int j=1;j<=n;j++) {
            if(minCost[i][j]==INF) cout << "0 ";
            else cout << minCost[i][j] << ' ';
        }
        cout << '\n';
    }
}
```

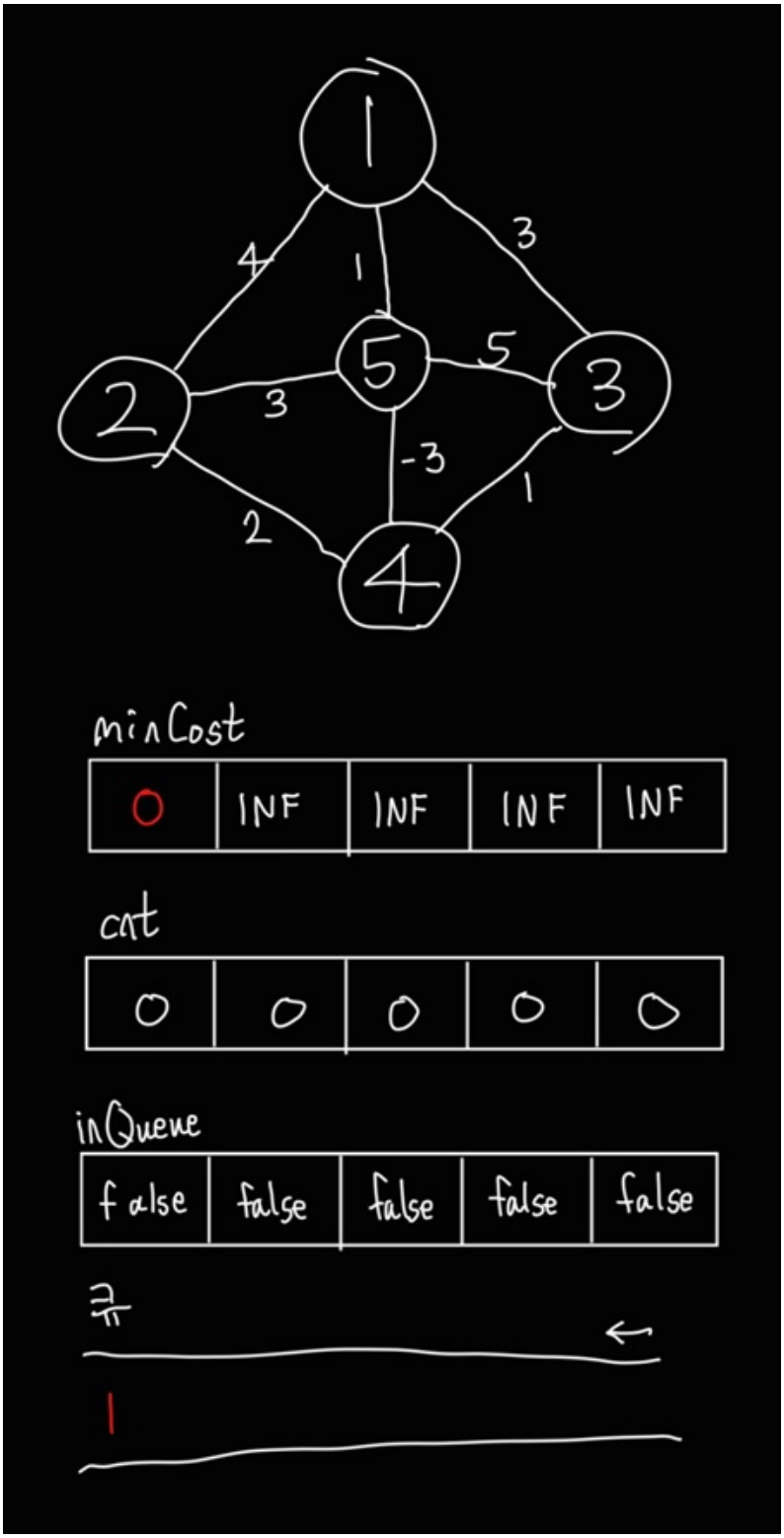
3.2.4 SPFA (Shortest Path Faster Algorithm) Gold IV

Bellman-Ford를 기반으로 큐를 사용해 갱신이 필요한 정점만 처리하여 최단 거리를 구하는 알고리즘

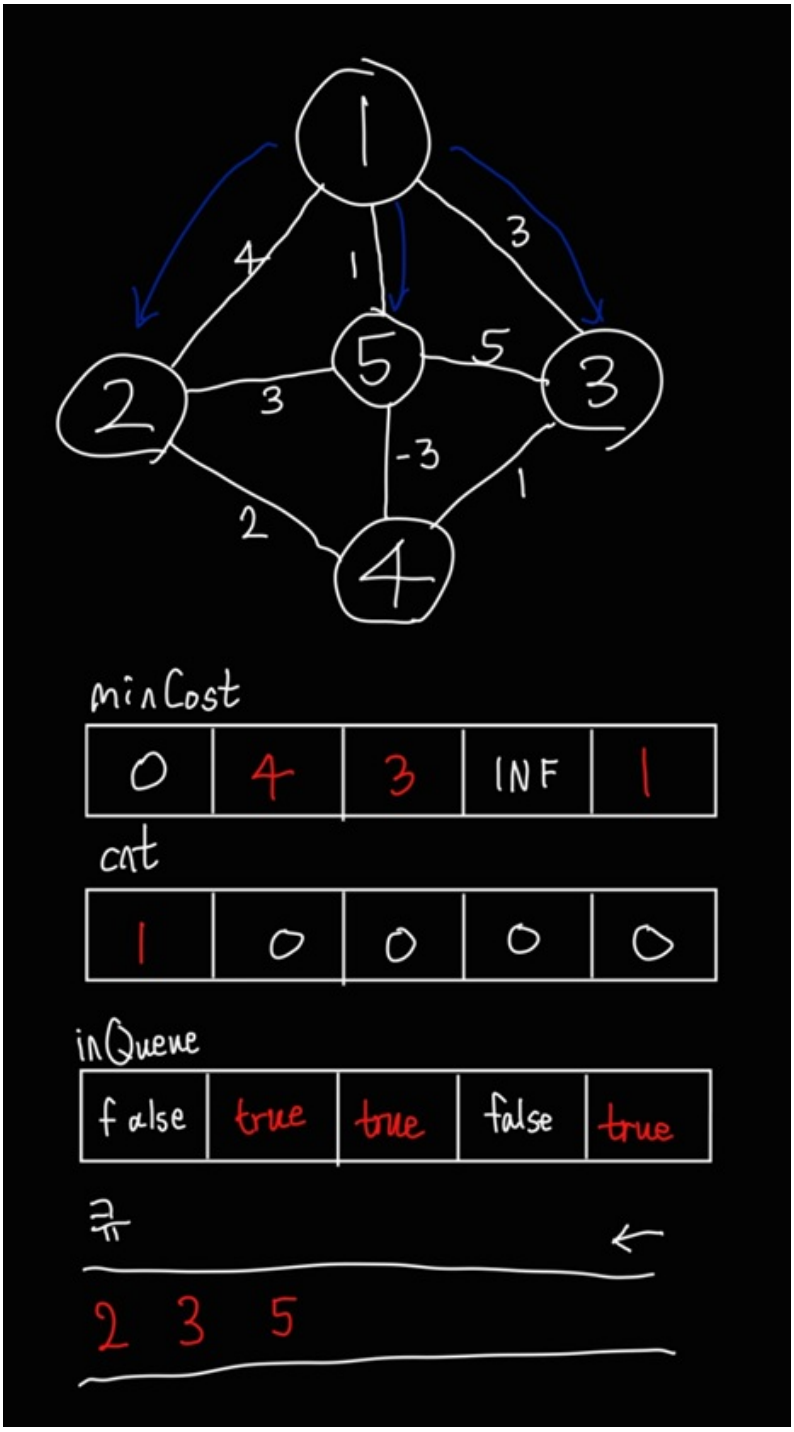
시간복잡도: 경험적 평균 $O(V+E)$, 최악 $O(VE)$ (V: 정점 수, E: 간선 수)



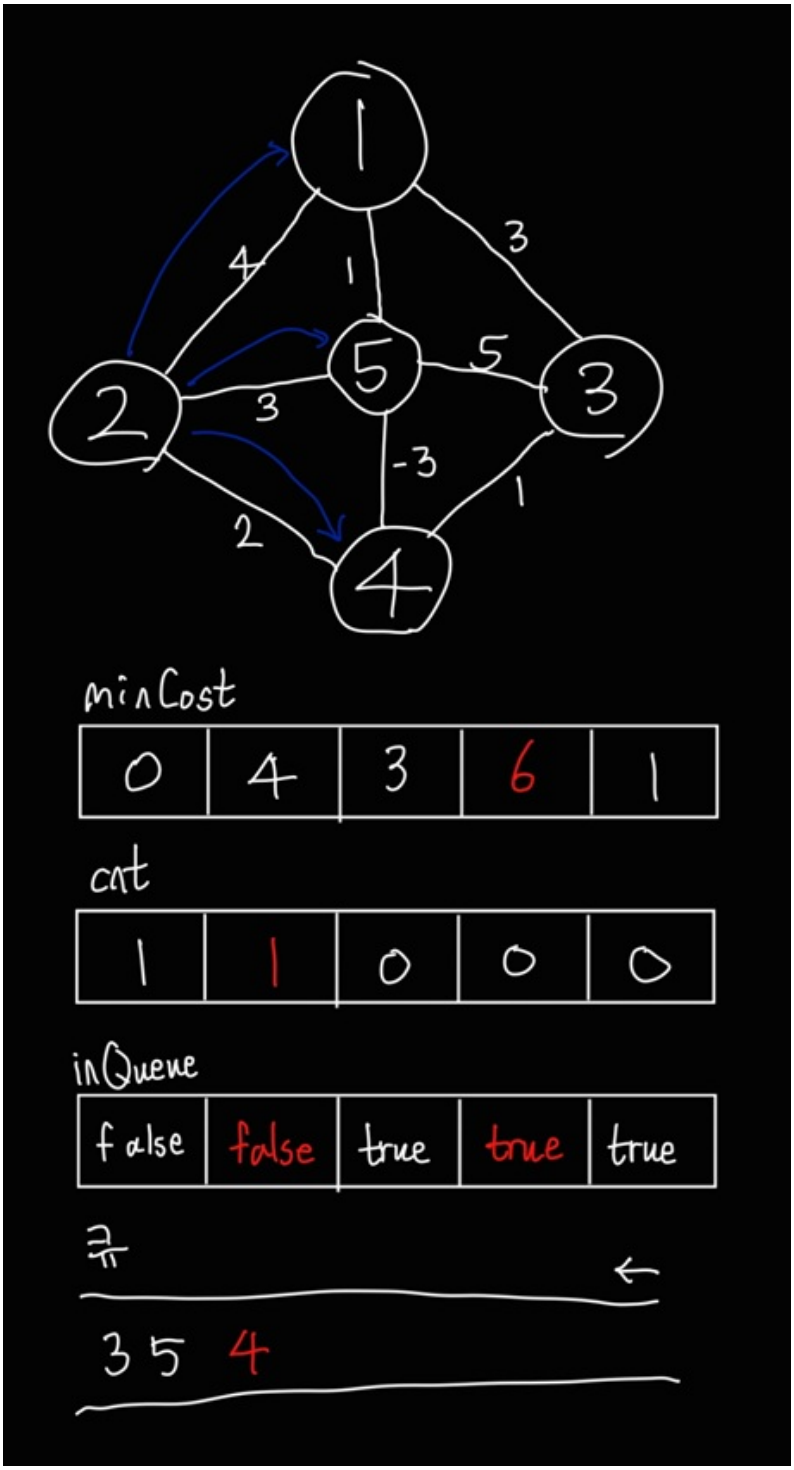
벨만-포드와 같은 그래프를 사용하고 **1번 정점에서 탐색을 시작**한다고 가정하자.



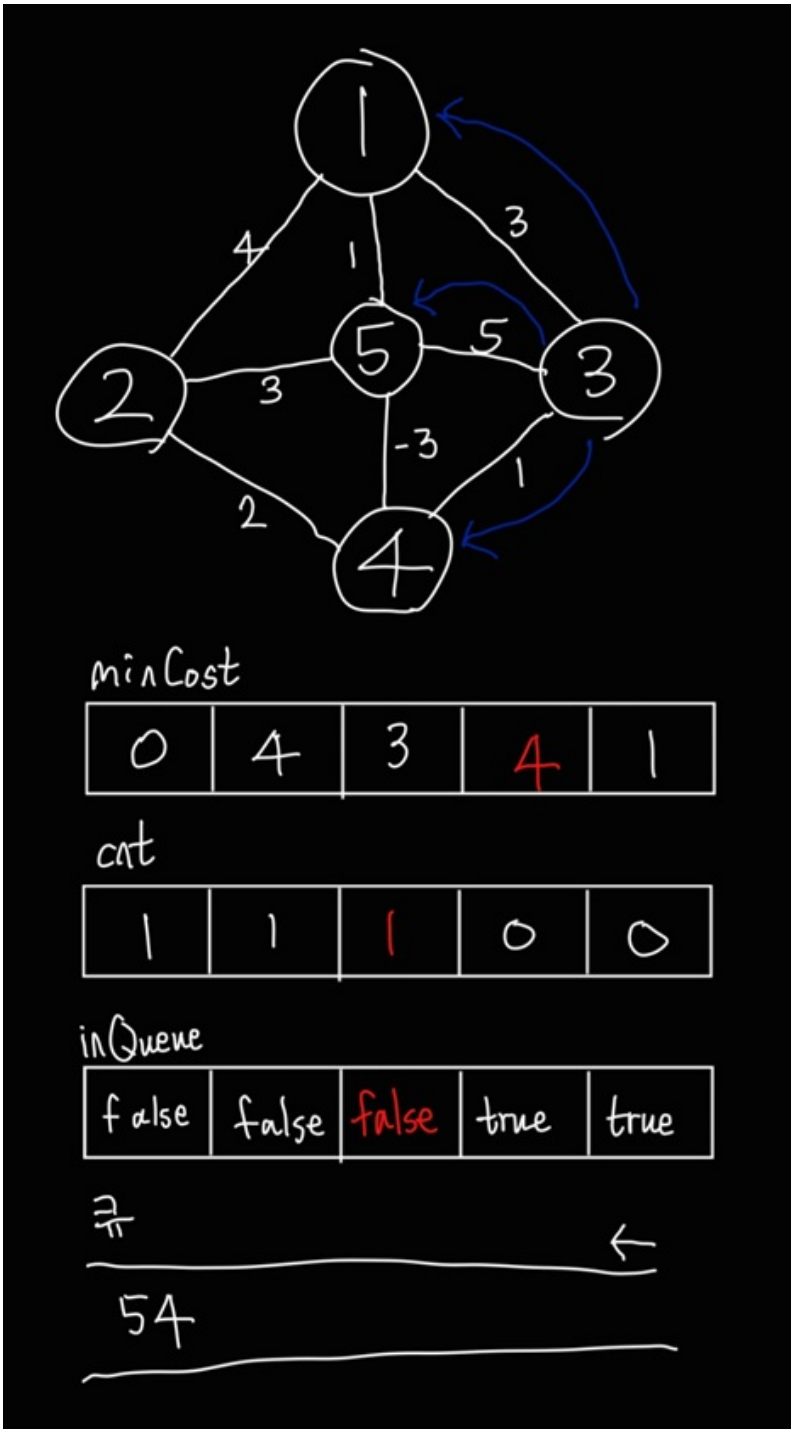
탐색을 시작하기 전에 **시작 정점인 1번 정점을 큐에 넣는다.**



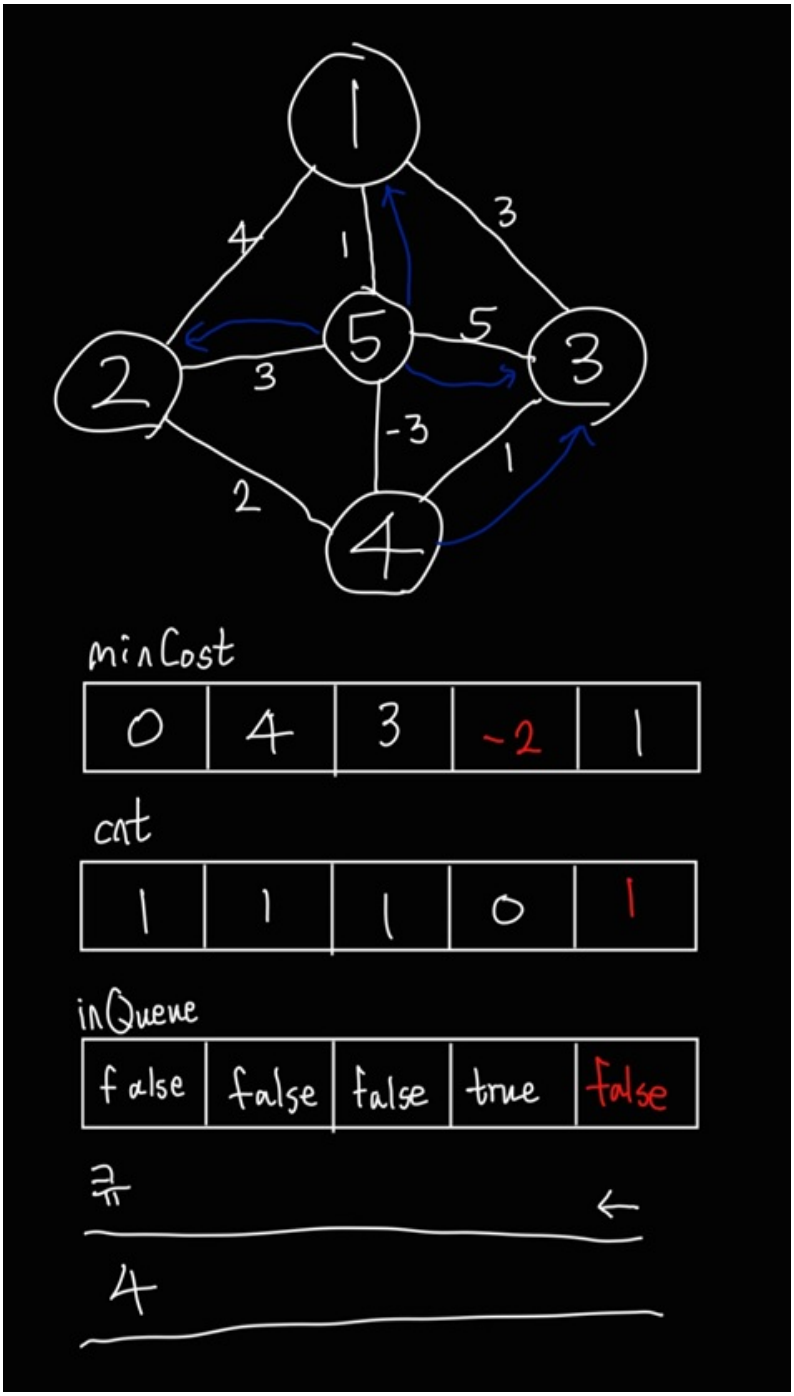
큐의 맨 앞 원소인 **1번 정점**을 꺼낸다. 1번 정점에서 접근 가능한 정점은 **2번, 3번, 5번**이다. 각 정점에 대해 간선을 완화하여 거리를 갱신한다. 2번 정점의 거리는 **4**, 3번 정점의 거리는 **3**, 5번 정점의 거리는 **1**로 갱신된다. 따라서 **2번, 3번, 5번 정점이 큐에 들어간다.**



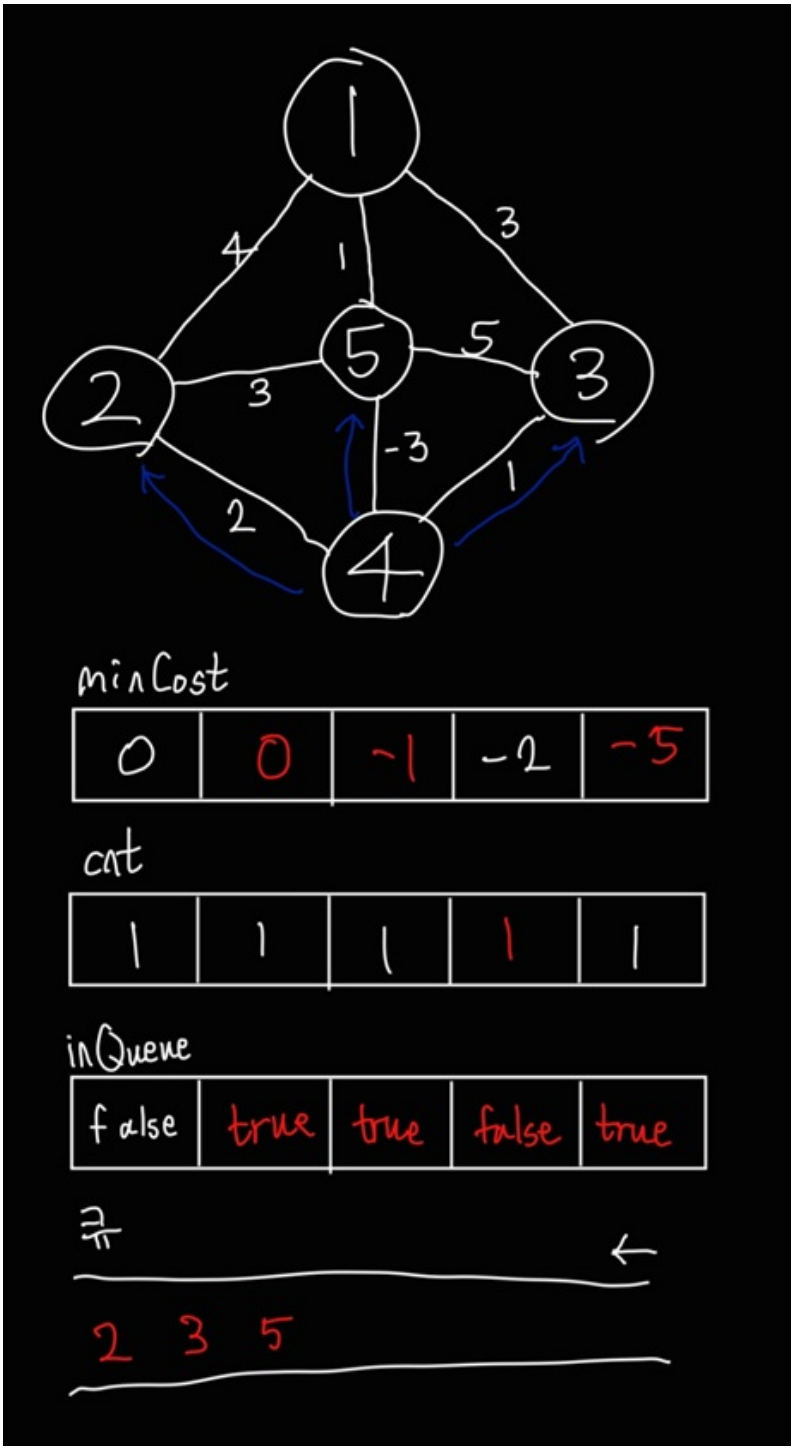
다음으로 큐의 맨 앞 원소인 **2번 정점**을 꺼낸다. 2번 정점에서 접근 가능한 정점은 **1번, 4번, 5번**이다. 이 중 거리가 단축되는 **4번 정점**의 거리를 **6**으로 갱신한다. 따라서 **4번 정점이 큐에 들어간다.**



다음으로 큐의 맨 앞 원소인 **3번 정점**을 꺼낸다. 3번 정점에서 접근 가능한 정점은 **1번, 4번, 5번**이다. 이 중 **4번 정점**의 거리가 **4**로 단축되지만 이미 큐에 들어 있으므로 **새로 큐에 들어가는 정점은 없다.**



다음으로 큐의 맨 앞 원소인 **5번 정점**을 꺼낸다. 5번 정점에서 접근 가능한 정점은 **1번, 2번, 3번, 4번**이다. 이 중 **4번 정점**의 거리가 **-2**로 단축되지만 이미 큐에 들어 있으므로 **새로 큐에 들어가는 정점은 없다.**



다음으로 큐의 맨 앞 원소인 **4번 정점**을 꺼낸다. 4번 정점에서 접근 가능한 정점은 **2번, 3번, 5번**이다. 이 중 **2번 정점**의 거리는 **0**, **3번 정점**의 거리는 **-1**, **5번 정점**의 거리는 **-5**로 단축된다. 따라서 **2번, 3번, 5번 정점이 큐에 들어간다.**

이 과정을 반복하다가 **큐가 비게 되면 더 이상 갱신될 정점이 없다는 의미이므로 알고리즘이 정상적으로 종료된다.** 또한 벨만-포드와 같은 원리로 하나의 정점이 n**번 이상 갱신되면**$(cnt == n)$ 해당 그래프에는 **음수 사이클이 존재한다고 판단하고 종료한다.**

연습 문제: 백준 11657번

```cpp
/** http://boj.kr/99e189479d7c4da1a50839057be86dc9 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;
const int MAX = 20001;

struct element {
    int pos, cost;
};

/**
 * minCost[i] : 시작 정점에서 i번 정점까지의 최단 거리
 * inQueue[i] : i번 정점이 큐에 들어있는지 확인하는 배열
 * cnt[i] : 음수 사이클을 찾기 위해 i번 정점으로의 최단거리가 몇 번 업데이트 됐는지
 ↪  기록하는 배열
 */
long long minCost[MAX];
int inQueue[MAX], cnt[MAX];
vector<vector<element>> conn(MAX);

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;
    while(m--) {
        int a, b, c; cin >> a >> b >> c;
        conn[a].push_back({b, c}); // 단방향 간선
    }

    /**
     * SPFA (Shortest Path Faster Algorithm)
     * - 우선순위 큐 대신 inQueue 배열과 큐를 사용
     */
    fill(minCost, minCost+MAX, INF); // 기본값 초기화
    queue<int> q; q.push(1); minCost[1]=0; // 시작 정점 넣기
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        inQueue[cur]=false;
        if(++cnt[cur]==n) { // 정점이 n번 업데이트 되는 경우 음수 사이클 존재
        ↪  (Bellman-Ford와 동일)
            cout << -1;
            return 0;
        }
        for(auto next:conn[cur]) {
            long long nextCost = minCost[cur] + next.cost;
            if(nextCost<minCost[next.pos]) { // 현재 경로로 다음 정점에 도달하는 비용이
            ↪  더 적다면
                minCost[next.pos] = nextCost; // 최단거리 업데이트
                if(!inQueue[next.pos]) { // 큐에 들어있지 않은 경우만 큐에 넣기
                    q.push(next.pos);
                    inQueue[next.pos]=true;
                }
            }
        }
    }

    for(int i=2;i<=n;i++) {
```

 CC BY-NC-SA 4.0

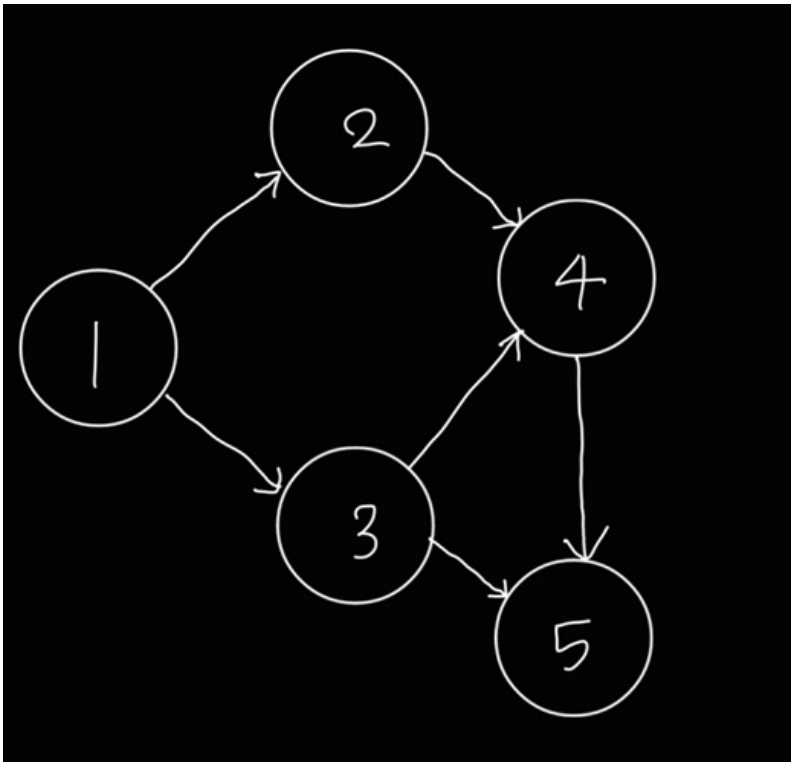
```
        if(minCost[i]==INF) cout << "-1\n";
        else cout << minCost[i] << '\n';
    }
}
```

3.3 DAG (Directed Acyclic Graph)

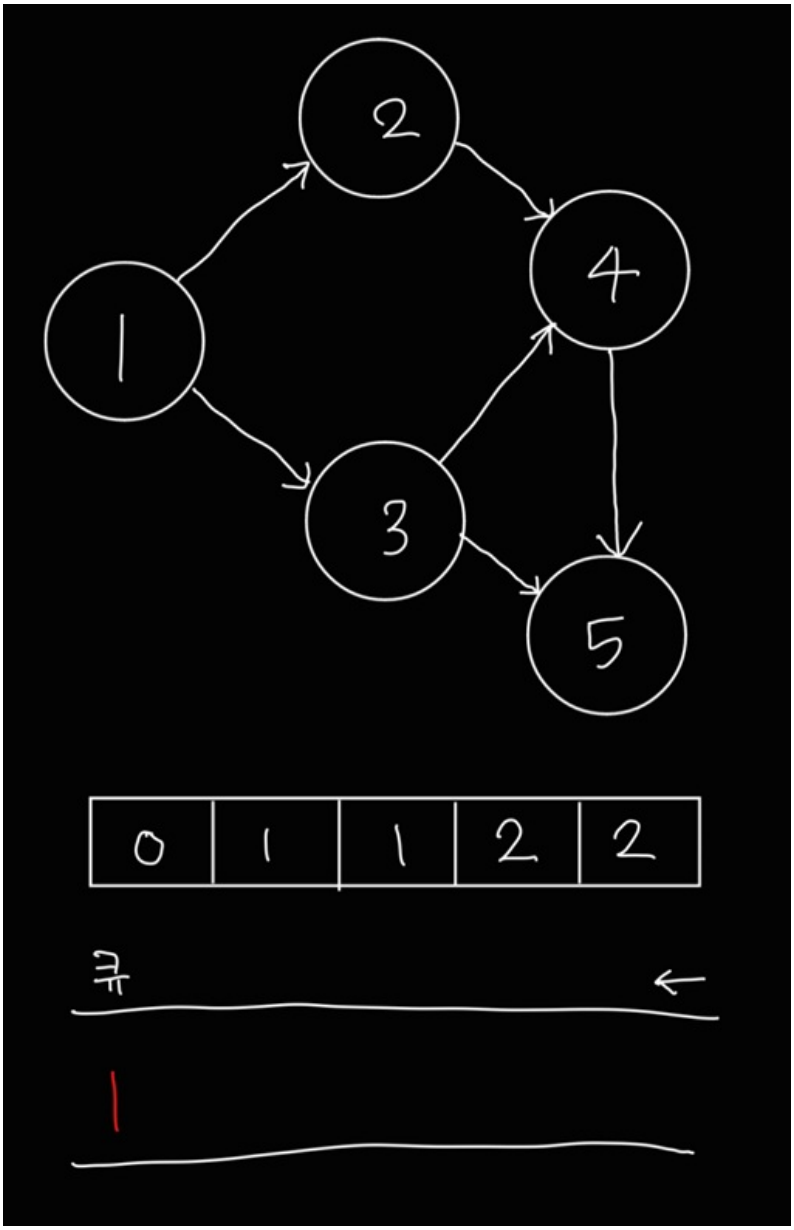
3.3.1 Kahn's Algorithm (Topological Sort, 위상 정렬) Gold III

방향성이 있고 사이클이 없는 그래프(DAG)에서 선행 관계를 만족하도록 정점을 나열하는 알고리즘

시간복잡도: $O(V + E)$ (V: 정점 수, E: 간선 수)

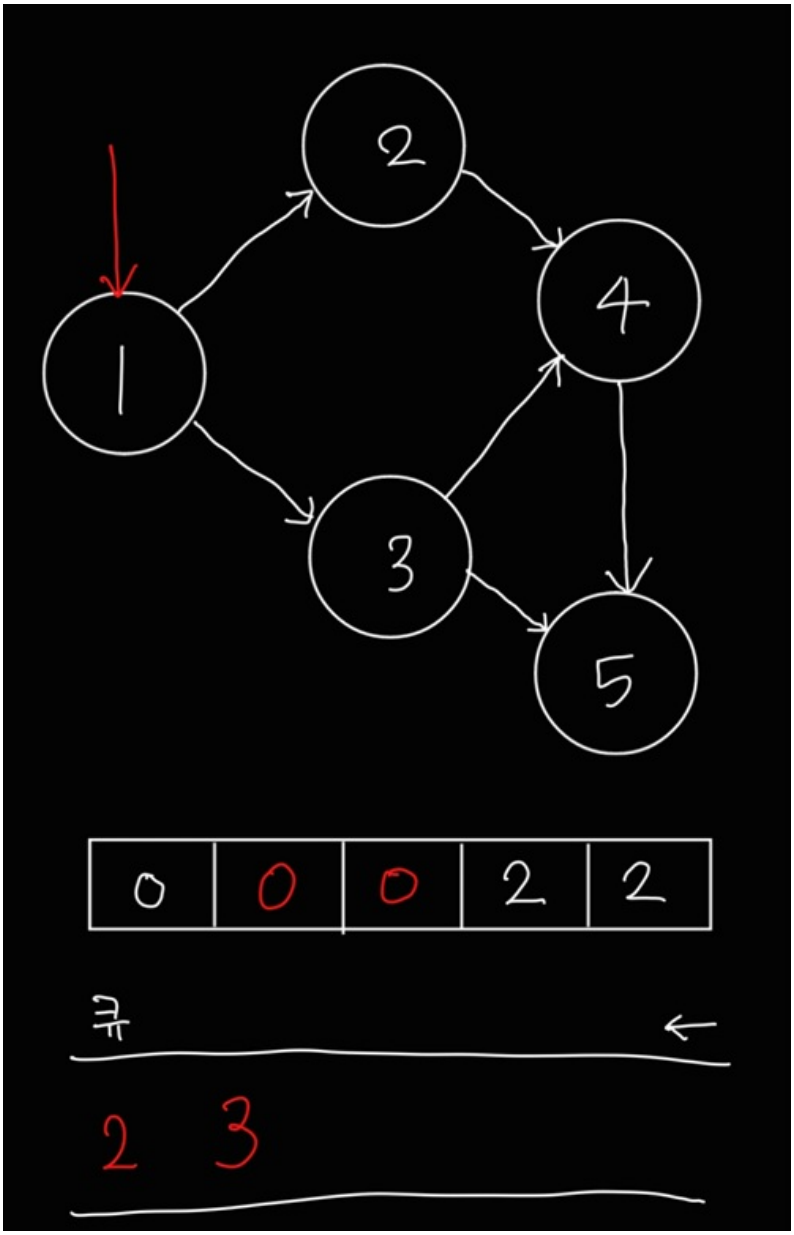


위와 같은 그래프가 있다고 하자.



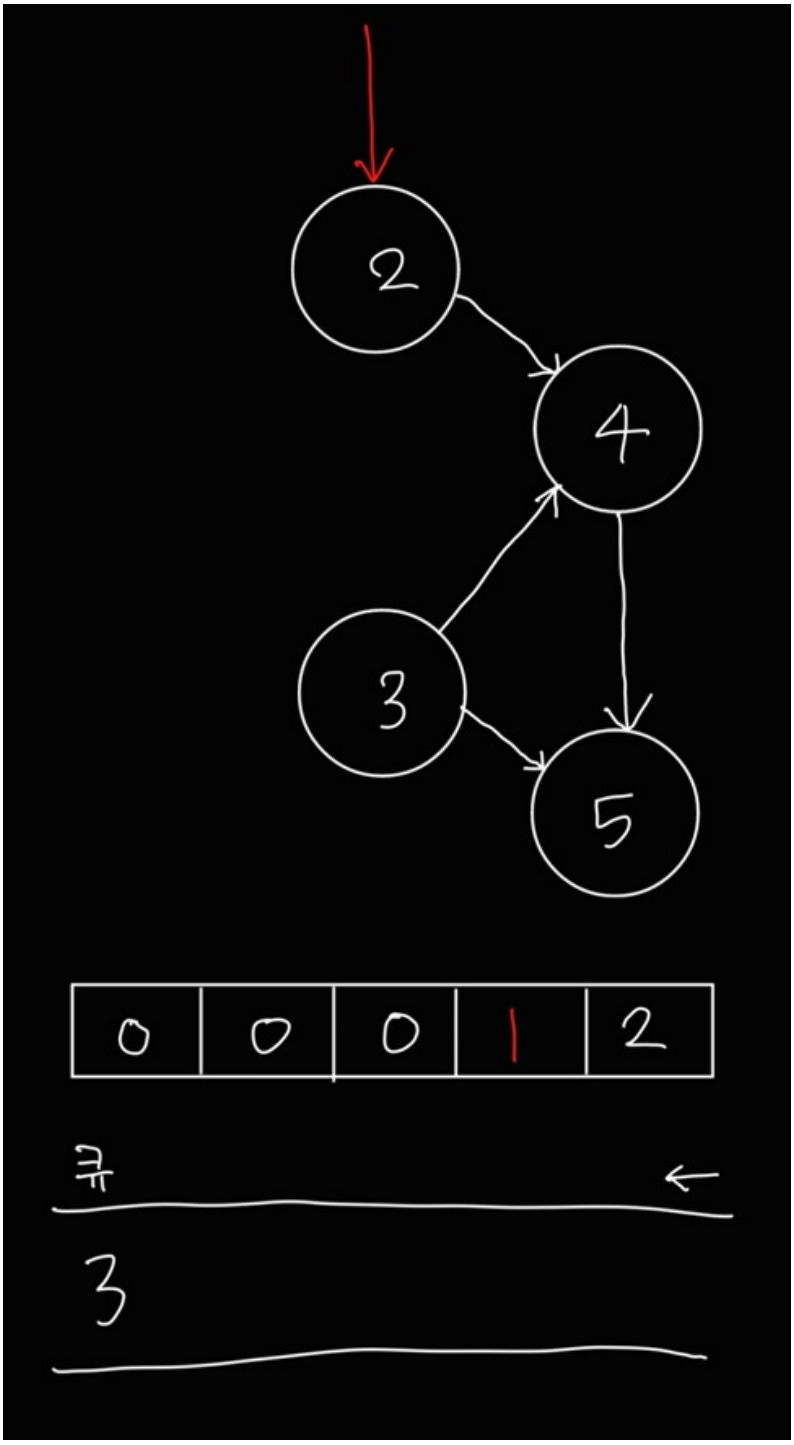
먼저 각 정점의 **진입 차수**를 계산한다. 이후 진입 차수가 **0인 모든 정점을 큐에 넣는다**. 초기 상태에서는 **1번 정점**이 큐에 들어간다.

 CC BY-NC-SA 4.0

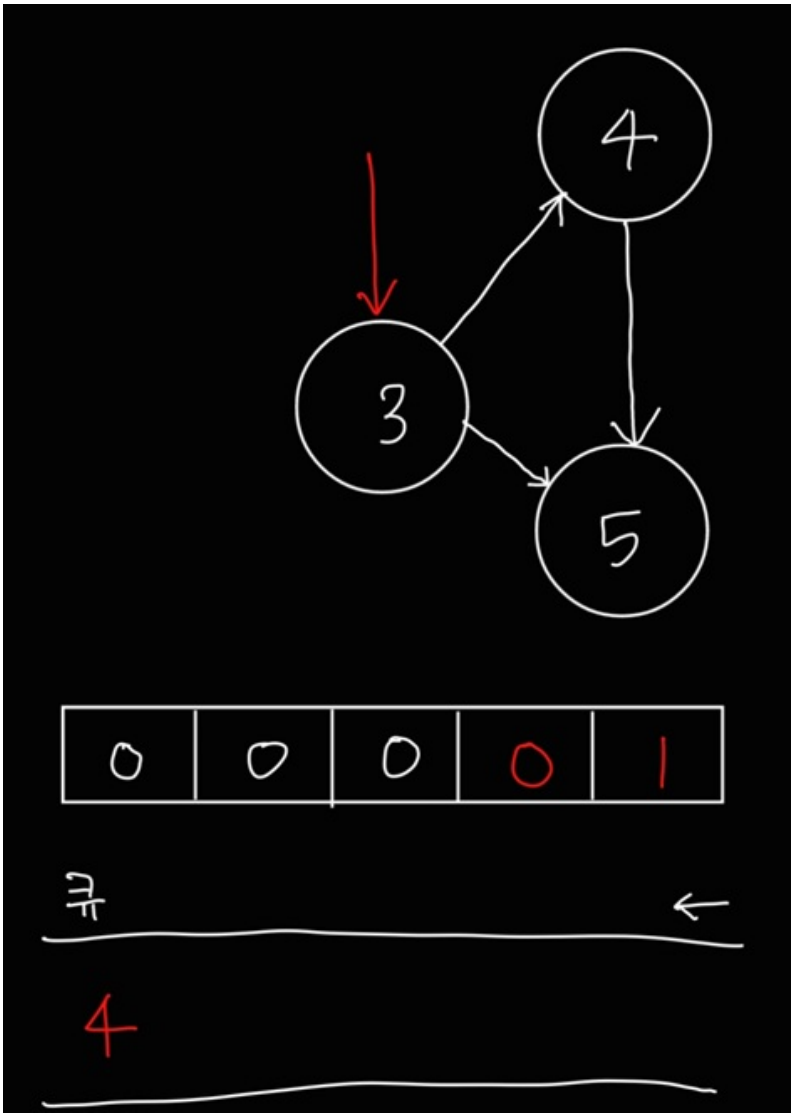


큐의 맨 앞에 있는 **1번 정점**을 꺼낸다. 1번 정점에서 갈 수 있는 **2번, 3번 정점**의 진입 차수를 각각 **1 감소시킨다**. 그 결과 **2번, 3번 정점의 진입 차수가 0이 되어 큐에 들어간다.**

 CC BY-NC-SA 4.0

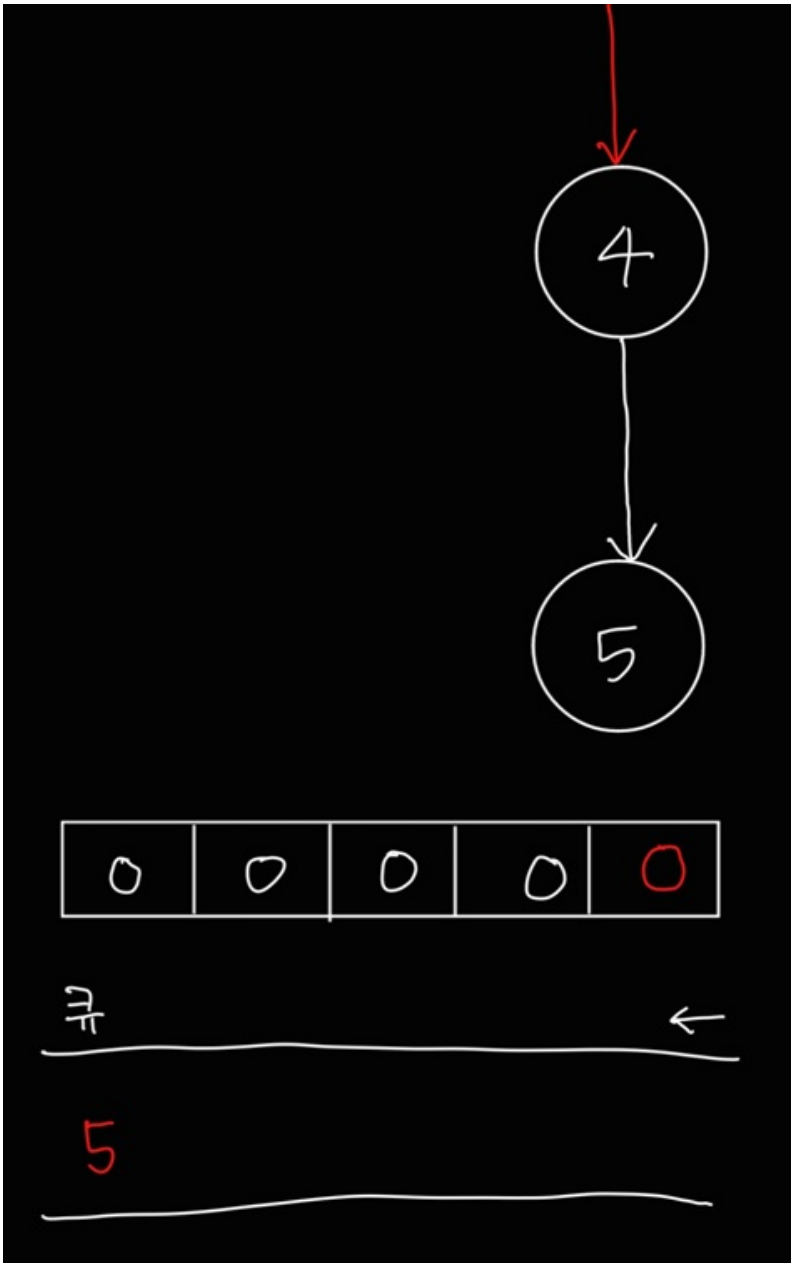


다음으로 큐의 맨 앞에 있는 **2번 정점**을 꺼낸다. 2번 정점에서 갈 수 있는 **4번 정점**의 진입 차수를 **1 감소시킨다**.

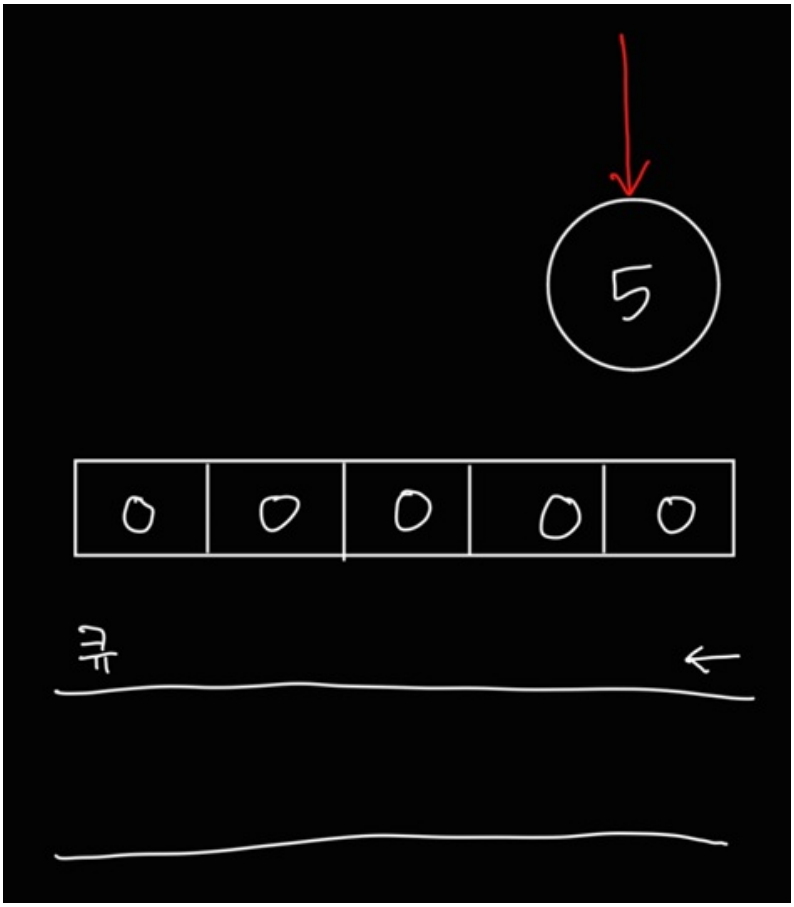


다음으로 큐의 맨 앞에 있는 **3번 정점**을 꺼낸다. 3번 정점에서 갈 수 있는 **4번, 5번 정점**의 진입 차수를 각각 **1 감소시킨다**. 그 결과 **4번 정점의 진입 차수가 0이 되어 큐에 들어간다.**

 CC BY-NC-SA 4.0



다음으로 큐의 맨 앞에 있는 **4번 정점**을 꺼낸다. 4번 정점에서 갈 수 있는 **5번 정점**의 진입 차수를 **1 감소시킨다**. 그 결과 **5번 정점의 진입 차수가 0이 되어 큐에 들어간다.**



다음으로 큐의 맨 앞에 있는 **5번 정점**을 꺼낸다. 5번 정점에서는 더 이상 갈 수 있는 정점이 없다. 다음 반복에서 **큐가 비게 되므로 알고리즘이 종료된다.**

이때 큐에서 꺼낸 정점의 순서가 바로 그래프의 **위상 정렬 결과**가 된다.

연습 문제: 백준 2252번

```
/** http://boj.kr/b66cea97849d422ebafedfee26b5643d 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int MAX = 32'001;

int inDegree[MAX]; // inDegree[i] : 정점 i으로 들어오는 간선의 개수
vector<vector<int>> conn(MAX);

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;
    while(m--) {
        int a, b; cin >> a >> b;
        inDegree[b]++; // 정점 b로 들어오는 간선의 개수 +1
        conn[a].push_back(b); // 정점 a에서 정점 b로의 간선 연결
    }

    /**
     * Kahn's Algorithm (Topological Sort)
```

```
     * 정점 i에 들어오는 간선이 없는 경우 큐에 추가
     * - i에 들어오는 간선이 없다는 뜻은 i보다 선행되야만 하는 정점이 없다는 뜻
     */
    queue<int> q;
    for(int i=1;i<=n;i++) {
        if(!inDegree[i]) {
            q.push(i);
        }
    }

    /**
     * 정점 cur에서 도착 가능한 바로 다음 정점의 inDegree 값을 1씩 감소
     * 다음 정점의 inDegree 값이 0이 되면 큐에 추가
     * - inDegree[next]==0 -> next보다 선행되야 하는 정점이 없다.
     * 이 문제에서는 사이클이 없지만 만약 사이클이 있다면 큐에 들어가는 총 정점 수가 n보다
     ↪  작음
     */
    while(!q.empty()) {
        int cur = q.front(); q.pop();
        cout << cur << ' ';
        for(int next:conn[cur]) {
            if(--inDegree[next]==0) {
                q.push(next);
            }
        }
    }
}
```

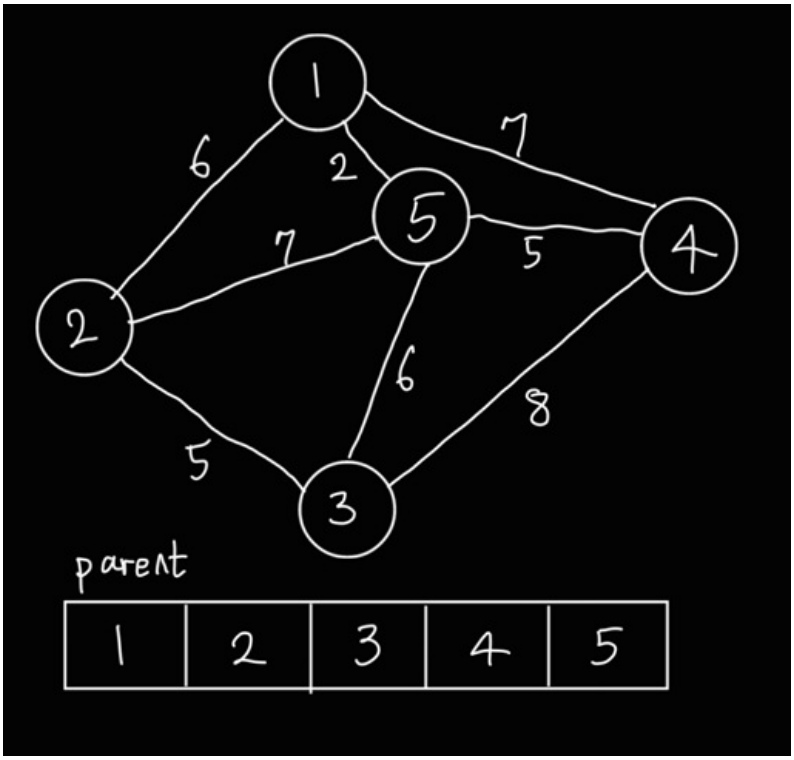
3.4 최소 스패닝 트리

3.4.1 Kruskal's Algorithm Gold IV

간선을 가중치 순으로 정렬한 뒤 사이클이 생기지 않는 간선만 선택하여 최소 스패닝 트리를 만드는 알고리즘

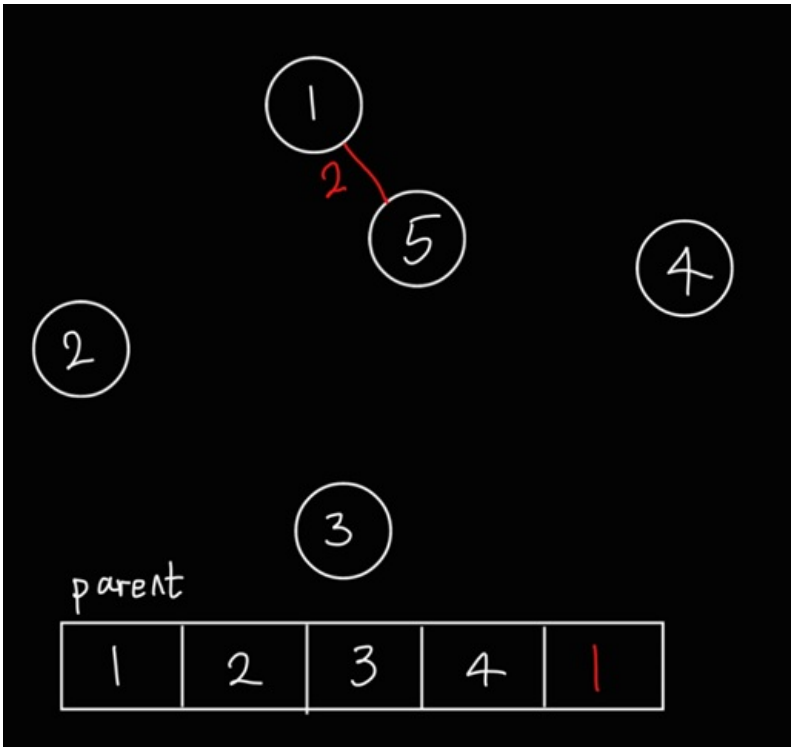
시간복잡도: $O(E \log E)$ (E: 간선 수)

 CC BY-NC-SA 4.0

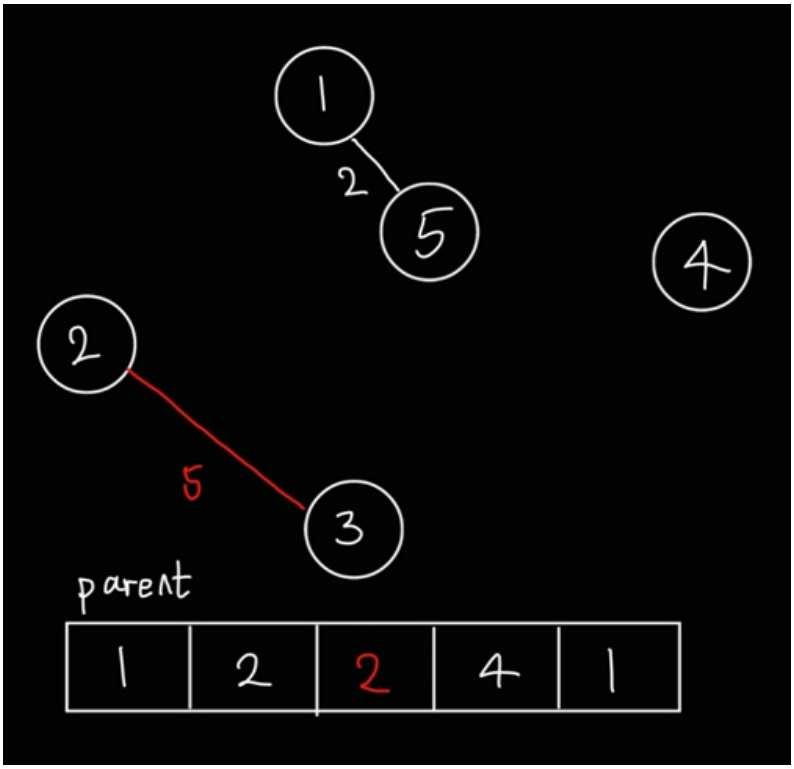


다음과 같은 그래프가 있다고 하자. 초기 상태에서는 모든 정점이 서로 다른 집합에 속하므로 집합의 개수는 총 5개이다.

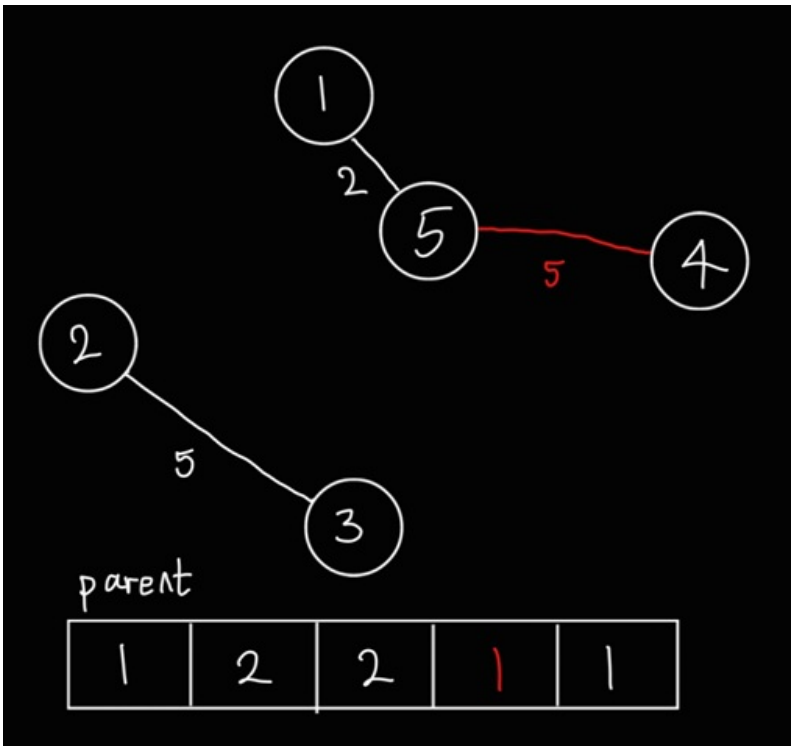
먼저 모든 간선을 가중치 기준으로 **오름차순 정렬**한다.



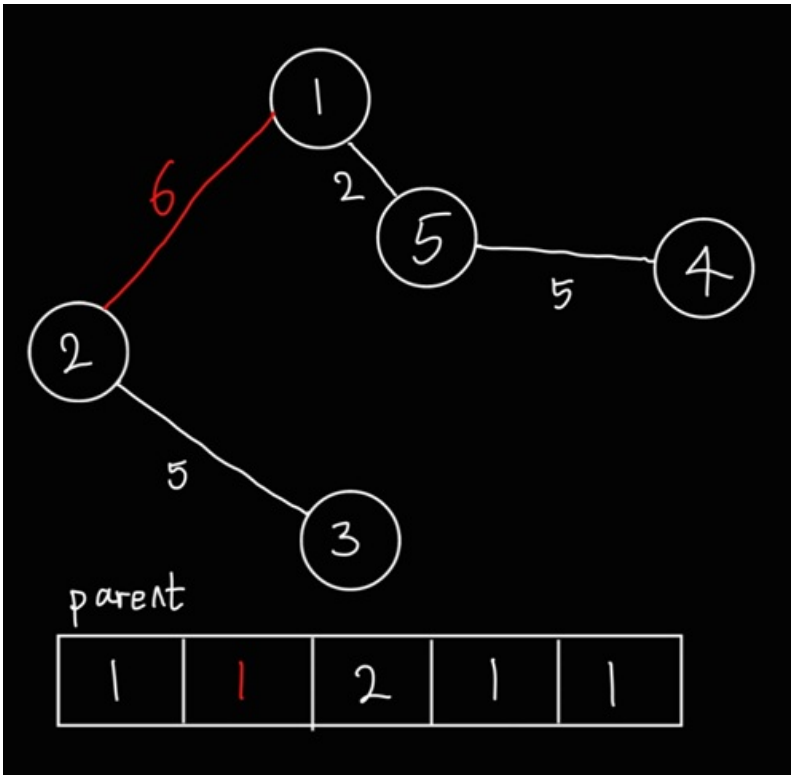
가중치가 가장 작은 간선은 **1번 정점과 5번 정점을 비용 2로 연결하는 간선**이다. 두 정점은 서로 다른 집합에 속하므로 이 간선을 선택하고 두 집합을 합친다. (DSU의 **Union 연산**)



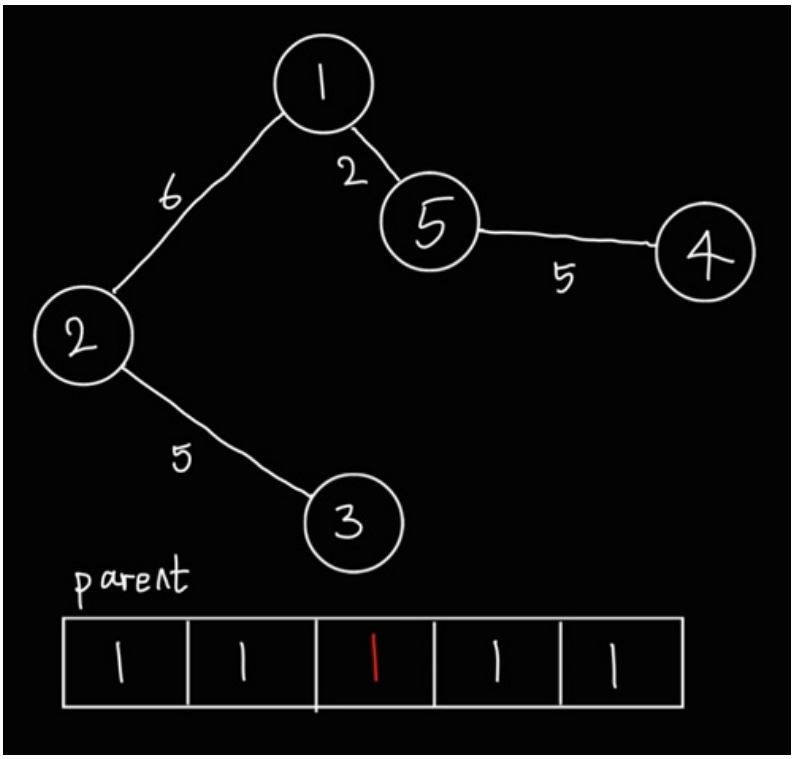
다음 간선은 **2번 정점과 3번 정점을 비용 5로 연결하는 간선**이다. 두 정점이 서로 다른 집합에 있으므로 이 간선도 선택하고 집합을 합친다.



다음 간선은 **4번 정점과 5번 정점을 비용 5로 연결하는 간선**이다. 이 두 정점 역시 서로 다른 집합에 속하므로 간선을 선택하고 집합을 합친다.



다음 간선은 **1번 정점과 2번 정점을 비용 6으로 연결하는 간선**이다. 두 정점이 서로 다른 집합에 있으므로 이 간선도 선택하고 집합을 합친다.



다음 간선은 **3번 정점과 5번 정점을 비용 6으로 연결하는 간선**이다. 그러나 두 정점은 이미 같은 집합에 속해 있으므로 이 간선을 선택하면 **사이클이 발생한다**. 따라서 이 간선은 선택하지 않는다.

이때 두 정점이 같은 집합에 속해 있는지 확인하기 위해 **Find 연산**을 수행한다.

 CC BY-NC-SA 4.0

이후 남아 있는 간선들도 모두 같은 집합에 속한 정점들을 연결하는 간선이므로 선택되지 않는다. 이렇게 선택된 간선들은 **최소 스패닝 트리(Minimum Spanning Tree)**를 이루게 된다.

연습 문제: 백준 1197번

```
/** http://boj.kr/71ab3ebe37394291bb957f50cd5b3228 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int MAX = 20'000;

int parent[MAX];

int _find(int x) {
    if(parent[x]==x) return x;
    return parent[x] = _find(parent[x]);
}

bool _union(int x, int y) {
    x = _find(x);
    y = _find(y);

    if(x==y) return false;
    if(x<y) parent[y]=x;
    else parent[x]=y;
    return true;
}

struct edge {
    int a, b, c;
    bool operator<(const edge e) const {
        return c < e.c;
    }
};

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int v, e; cin >> v >> e;
    vector<edge> edges;
    while(e--) {
        int a, b, c; cin >> a >> b >> c;
        edges.push_back({a-1, b-1, c});
    }
    for(int i=0;i<v;i++) parent[i]=i;

    /**
     * 크루스칼 알고리즘
     * 모든 정점을 최소 비용으로 연결하는 알고리즘
     * - 간선을 비용 기준으로 오름차순으로 정렬
     * - 순서대로 두 정점이 다른 집합에 있을 때만 DSU로 합치면서 비용 누적
     * - - v개의 정점은 v-1번의 합치기 연산 가능
     * - - 나머지 간선은 무시
     */
    int total=0;
    sort(edges.begin(), edges.end());
    for(auto e : edges) {
```

```
        if(_union(e.a, e.b)) {
            total += e.c;
        }
    }
    cout << total;
}
```

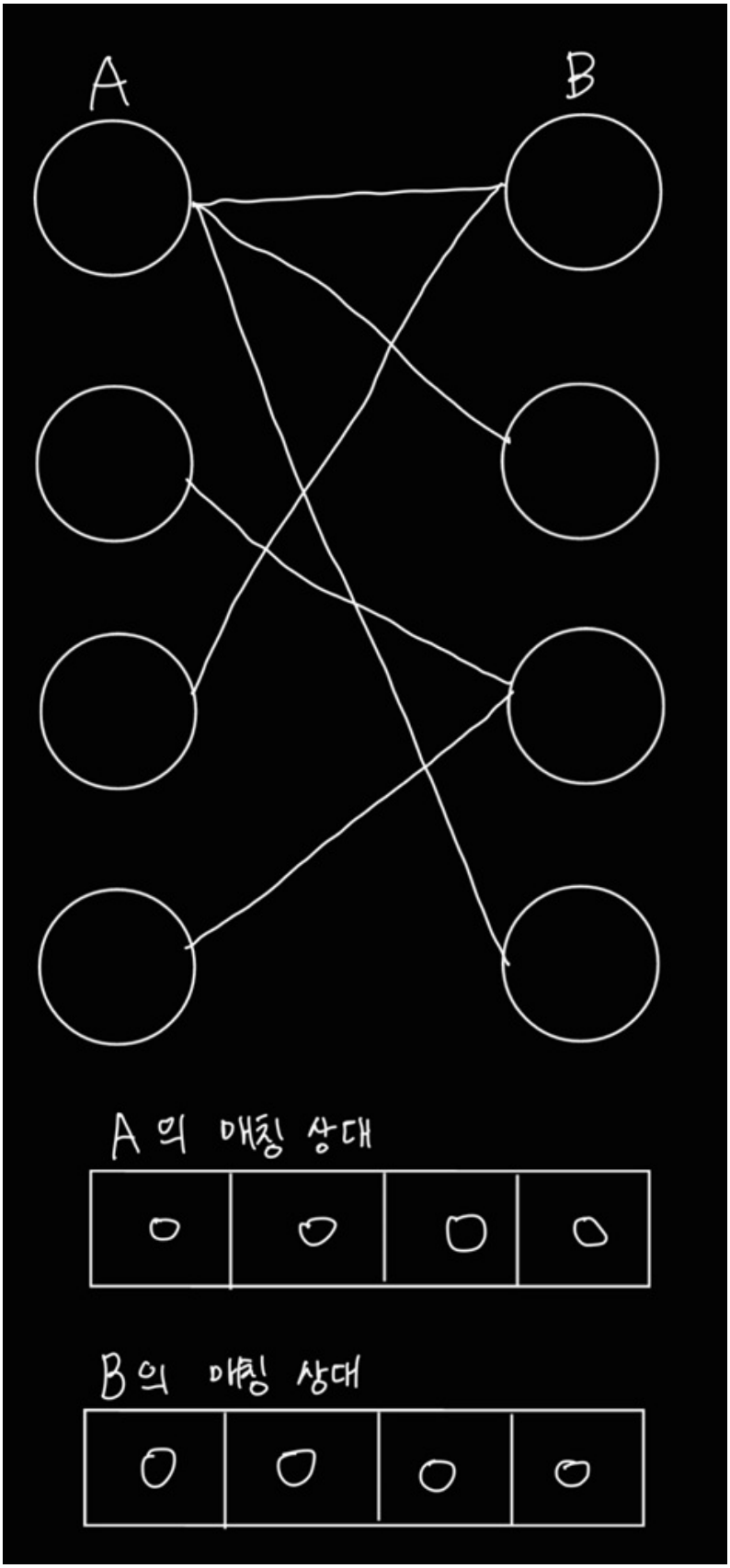
3.5 유량

3.5.1 Kuhn's Algorithm (Maximum Bipartite Matching, 이분 매칭) Platinum IV

이분 그래프에서 서로 겹치지 않는 간선의 최대 집합(최대 매칭)을 찾는 알고리즘

시간복잡도: $O(VE)$ (V: 왼쪽 그룹의 정점 수)

 CC BY-NC-SA 4.0

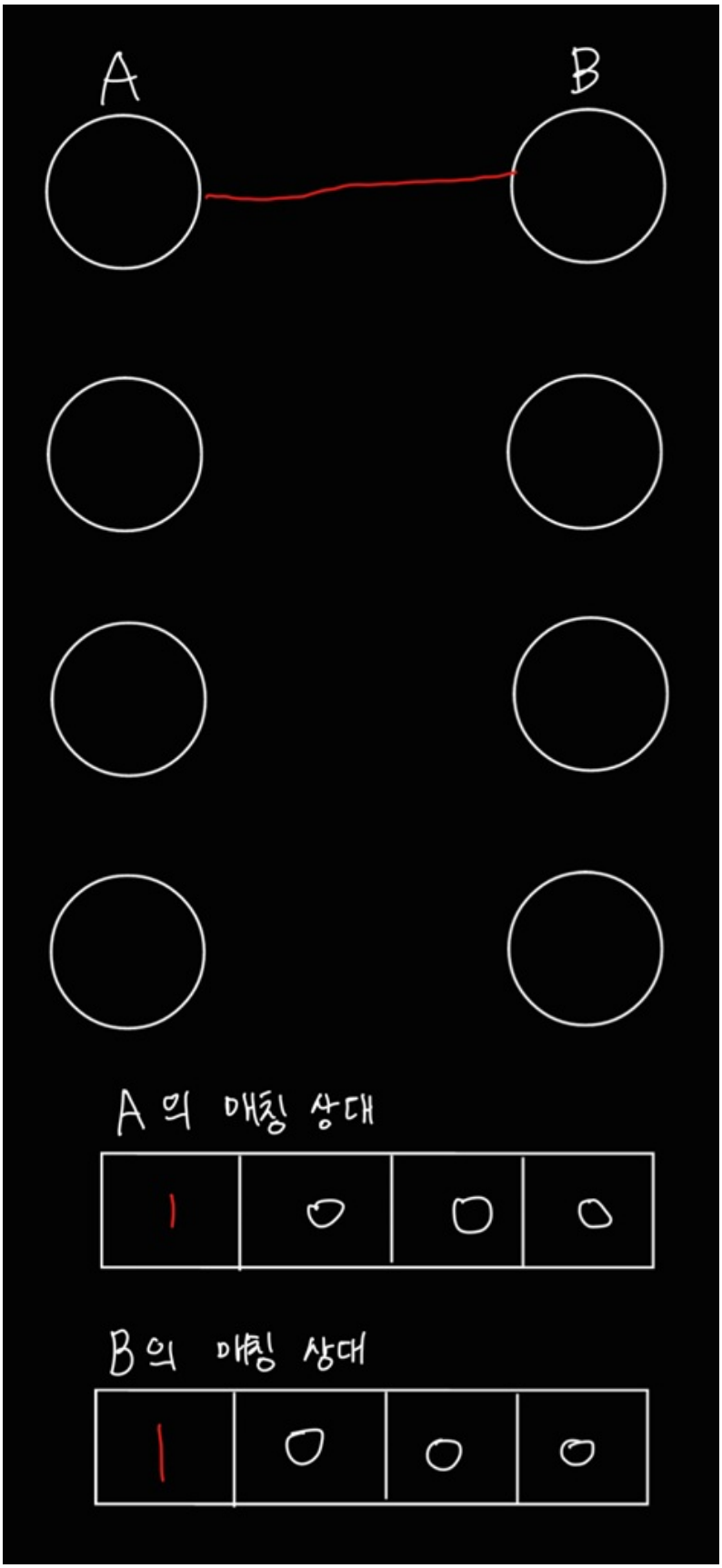


다음과 같은 이분 그래프가 있다고 하자. 이분 그래프는 정점을 왼쪽 그룹과 오른쪽 그룹으로 나누었을 때 간선이 항상 서로 다른 그룹 사이에만 존재하는 그래프이다.

초기 상태에서는 두 개의 배열을 사용해 각 정점의 매칭 상태를 관리한다고 가정한다. 이 그래프에서

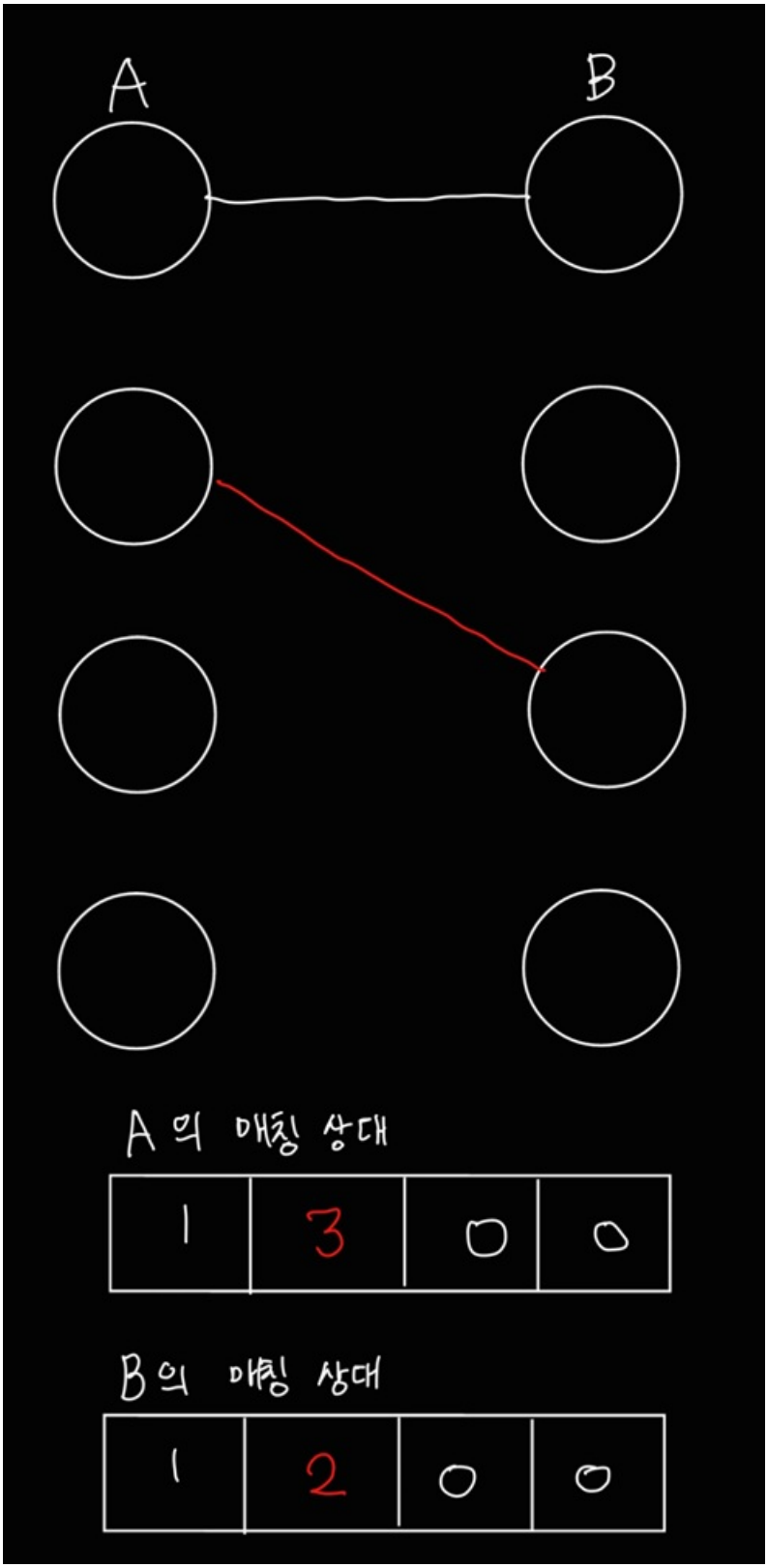
 CC BY-NC-SA 4.0

서로 겹치지 않는 왼쪽 정점–오른쪽 정점 쌍의 최대 개수를 구하는 것이 목표이다.

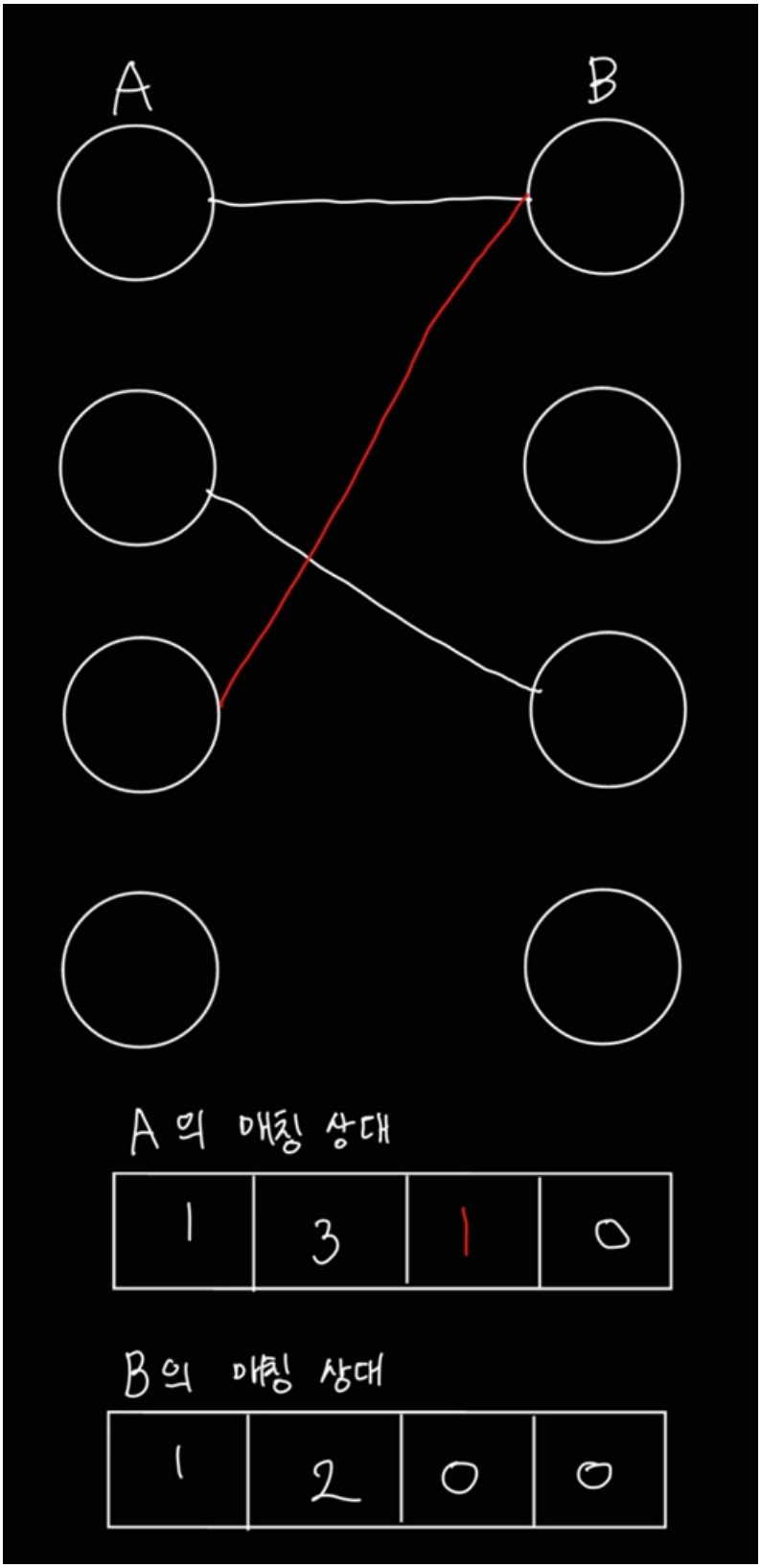


먼저 **A1**에 대해 연결 가능한 첫 번째 정점인 **B1**과 매칭을 시도한다. 현재 B1은 어떤 정점과도

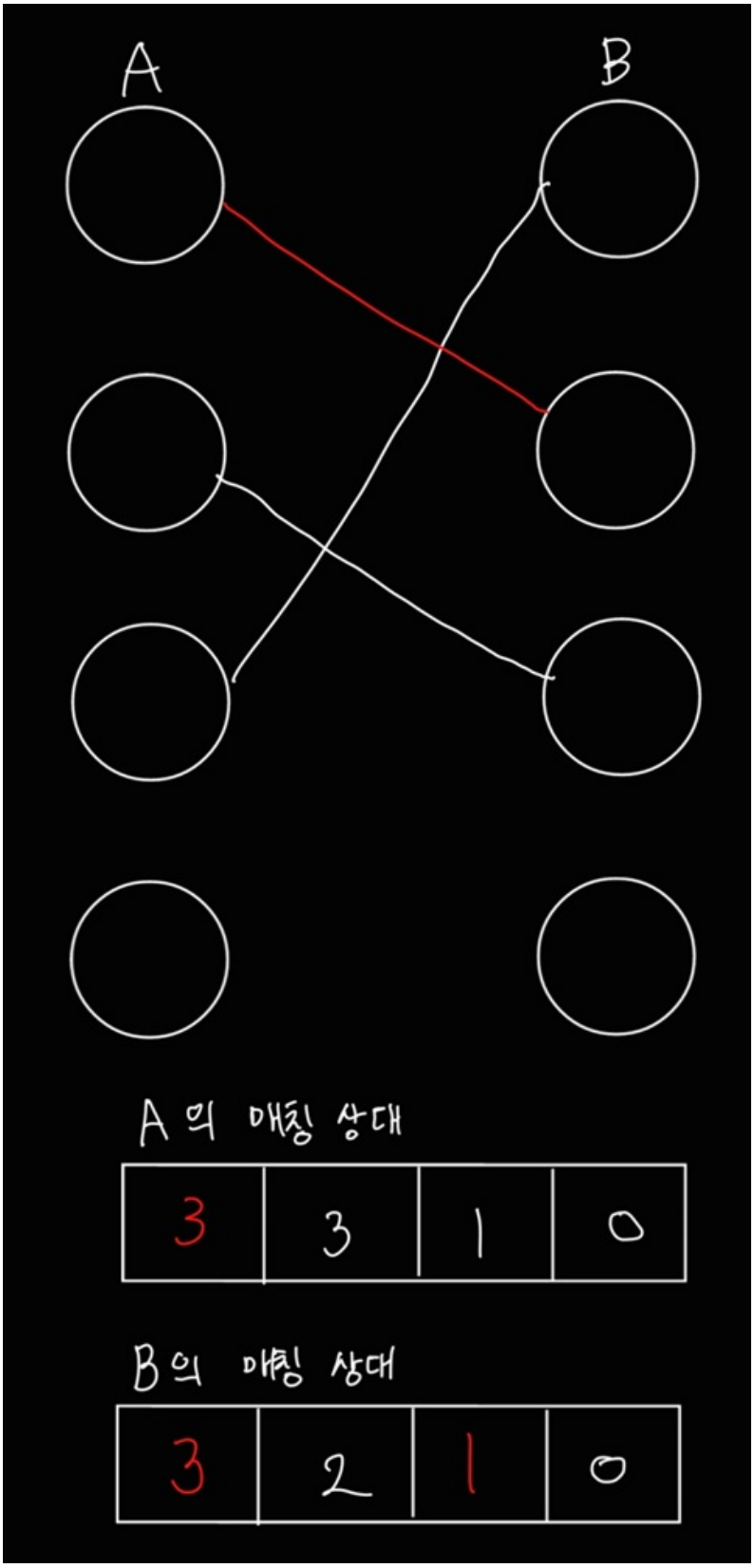
매칭되어 있지 않으므로 **A1–B1** 매칭이 생성된다.



다음으로 **A2**에 대해 연결 가능한 첫 번째 정점인 **B3**과 매칭을 시도한다. B3 역시 아직 매칭되어 있지 않으므로 **A2–B3** 매칭이 생성된다.

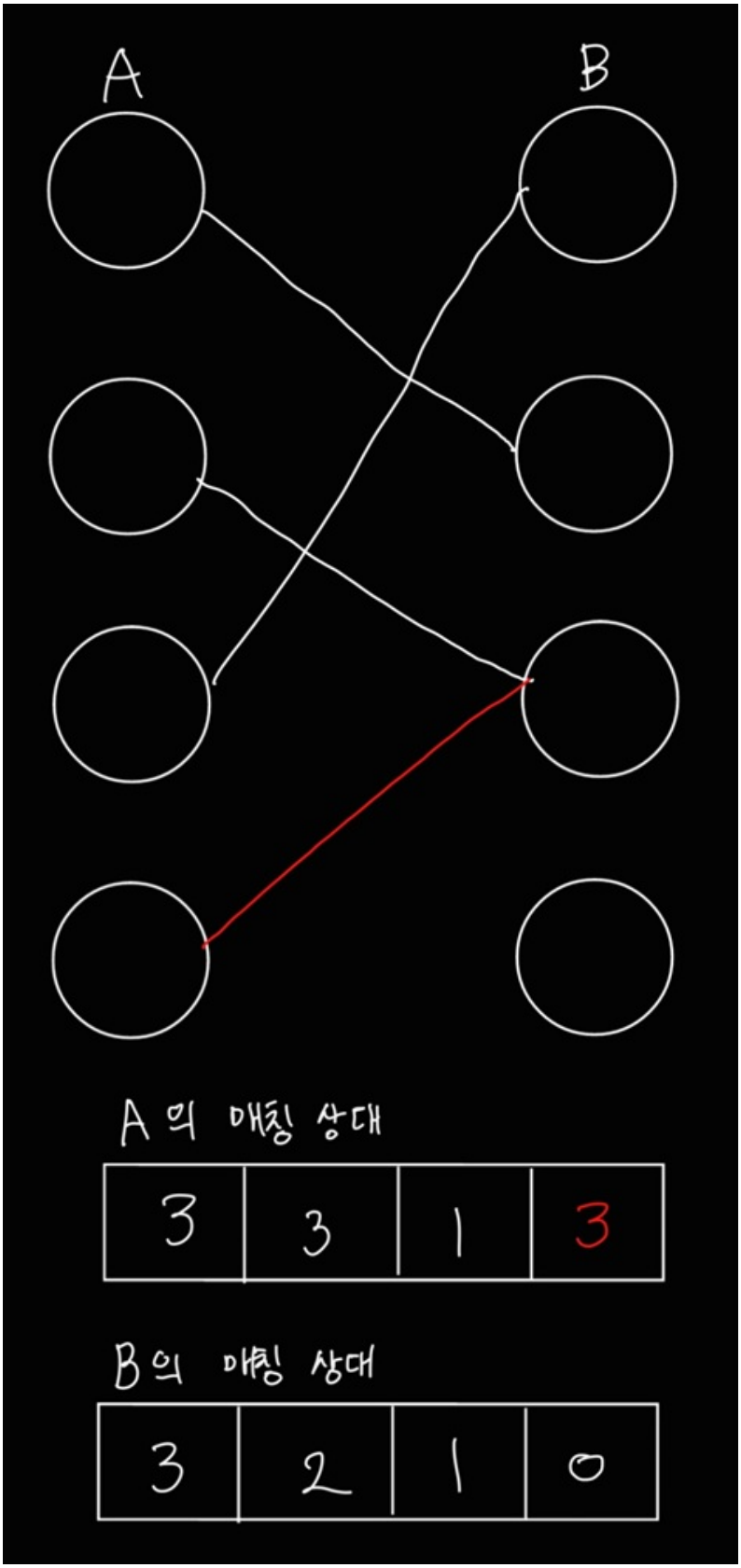


이제 **A3**에 대해 연결 가능한 첫 번째 정점인 **B1**과 매칭을 시도한다. 하지만 B1은 이미 **A1**과 매칭되어 있으므로 바로 매칭할 수 없다.



이 경우 기존 매칭을 재배치할 수 있는지 확인한다. B1과 연결된 **A1**이 다른 정점과 매칭될 수 있는지 탐색한다. A1은 **B2**와 연결할 수 있으므로 매칭을 **A1–B2**로 변경한다. 그 결과 **A3–B1** 매칭이 새롭게 가능해진다.

 CC BY-NC-SA 4.0



마지막으로 **A4**에 대해 연결 가능한 첫 번째 정점인 **B3**과 매칭을 시도한다. 그러나 B3은 이미 **A2**와 매칭되어 있다. A2가 다른 정점과 매칭될 수 있는지 확인하지만 가능한 정점이 없으므로 **A4는 매칭에 실패한다.**

이와 같이 Kuhn's Algorithm은 기존 매칭을 재배치하면서 새로운 매칭을 확장해 나가며 **최대 매**

 CC BY-NC-SA 4.0

칭을 찾는다.

연습 문제: 백준 11375번

```
/** http://boj.kr/2dc9d016c40d40508c1992d763a1b512 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

const int MAX = 1001; // 최대 정점 개수 (문제마다 다름)
int A[MAX], B[MAX]; // A : 왼쪽 정점이 선택한 오른쪽 정점 번호, B : 오른쪽 정점이 선택한
↪  왼쪽 정점 번호, 맨 처음에 -1로 한번 초기화
bool visited[MAX]; // 이미 이번 확인에서 방문한 정점인지 확인하기 위한 배열, 매 방문마다
↪  false로 초기화
vector<vector<int>> conn(MAX); // 왼쪽 정점 -> 오른쪽 정점으로의 간선 목록

/**
 * cur번째 왼쪽 정점이 아직 선택이 안된 오른쪽 정점과 새로 매칭이 되는지 확인.
 *
 * 만약 왼쪽 정점과 오른쪽 정점이 다음과 같이 간선이 있다면,
 * 1 -> 1
 * 1 -> 2
 * 2 -> 2
 *
 * main의 for문에서 dfs(1) 호출
 * 1 -> 1 살펴봄 : 첫 매칭 발생. (A[1]==1, B[1]==1)
 *
 * main의 for문에서 dfs(2) 호출
 * 2 -> 1 을 살펴보는데 B[1]에 이미 1이란 값이 들어있음. : 왼쪽 1번 정점을 다른 오른쪽
 ↪  정점과 매칭할 수 있는지 확인
 * 1 -> 1 살펴봄 : 오른쪽 1번 정점은 방금 방문해서 더 살펴볼 필요가 없음(visited[1]==true)
 * 1 -> 2 살펴봄 : 매칭 변경 가능 (1 -> 2로 변경, A[1]==2, B[2]==1)
 * 2 -> 1 다시 살펴봄 : 매칭 가능 (A[2]==1, B[1]==2)
 */
bool dfs(int cur) {
    visited[cur]=true;
    for(int next:conn[cur]) {
        if(B[next]==-1 || !visited[B[next]] && dfs(B[next])) {
            A[cur] = next;
            B[next] = cur;
            return true;
        }
    }
    return false;
}

int main(void) {
    cin.tie(0)->sync_with_stdio(0);
    int N, M; cin >> N >> M;

    /**
     * i번째 왼쪽 정점(직원 정점)이 j번째 오른쪽 정점(일 정점)과 간선이 있음.
     */
    for(int i=1;i<=N;i++) {
        int cnt; cin >> cnt;
        while(cnt--) {
            int j; cin >> j;
```

```
            conn[i].push_back(j);
        }
    }

    /**
     * 왼쪽 정점과 오른쪽 정점이 서로 어디에 연결되었는지 확인하기 위한 배열
     * A[i] : i번 왼쪽 정점이 A[i]번 오른쪽 정점과 연결됨.
     * B[i] : i번 오른쪽 정점이 B[i]번 왼쪽 정점과 연결됨.
     * 처음에는 연결이 안되어있어서 -1로 초기화 (꼭 -1일 필요는 없고 절대 등장하지 않은
     ↪  편한 숫자로 초기화하면 됨)
     */
    memset(B, -1, sizeof B);

    /**
     * cnt : 최대 매칭 수
     * i번 왼쪽 정점이 매칭할 수 있는지 확인
     * dfs(i)==true면, 새로 매칭 가능
     */
    int cnt=0;
    for(int i=1;i<=N;i++) {
        memset(visited, false, sizeof visited);
        if(dfs(i)) cnt++;
    }
    cout << cnt;
}
```

 CC BY-NC-SA 4.0